

LONG SEQUENCE TIME-SERIES FORECASTING PADA PASAR VALUTA ASING EUR/USD MENGGUNAKAN MODEL INFORMER

SKRIPSI

Oleh:

**MUHAMMAD FAQIH AJIPUTRA
2209106114**



**FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN**

**SAMARINDA
2025**

LONG SEQUENCE TIME-SERIES FORECASTING PADA PASAR VALUTA ASING EUR/USD MENGGUNAKAN MODEL INFORMER

SKRIPSI

Diajukan sebagai salah satu syarat untuk menyelesaikan pendidikan
pada Program Studi Strata 1 Informatika,
Fakultas Teknik, Universitas Mulawarman

Oleh:

**MUHAMMAD FAQIH AJIPUTRA
2209106114**



**FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN**

**SAMARINDA
2025**

PERNYATAAN KEASLIAN SKRIPSI

Saya menyatakan dengan sesungguhnya bahwa skripsi dengan judul:

LONG SEQUENCE TIME-SERIES FORECASTING PADA PASAR VALUTA ASING EUR/USD MENGGUNAKAN MODEL INFORMER

yang dibuat sebagai salah satu syarat untuk menyelesaikan pendidikan pada Program Studi S1 Informatika, Fakultas Teknik, Universitas Mulawarman, sejauh yang saya ketahui bukan merupakan tiruan atau duplikasi dari skripsi yang sudah dipublikasikan dan atau pernah dipakai untuk mendapatkan gelar kesarjanaan di lingkungan Universitas Mulawarman maupun di Perguruan Tinggi atau instansi manapun, kecuali bagian yang sumber informasinya dicantumkan sebagaimana mestinya.

Samarinda, Tgl bln thn

Materai

Muhammad Faqih Ajiputra

NIM. 2209106114

LONG SEQUENCE TIME-SERIES FORECASTING PADA PASAR VALUTA ASING EUR/USD MENGGUNAKAN MODEL INFORMER

Oleh:

Muhammad Faqih Ajiputra
2209106114

Telah diujikan pada Tgl Bln Tahun dan dinyatakan telah
memenuhi syarat

Samarinda, Tanggal Bulan Tahun

Disahkan oleh:

Pembimbing I,

Pembimbing II,

Rosmasari, S.Kom., M.T.
NIP 198509212019032017

Prof. Dr. Fahrul Agus, S.Si., M.T.
NIP 196909261994121002

Mengetahui,
Dekan Fakultas Teknik
Universitas Mulawarman

Prof. Dr. Ir. H. Tamrin, ST., MT., IPU., ASEAN Eng., APEC Eng
NIP197002272000121001

Skripsi ini penulis persembahkan untuk diri sendiri dan keluarga tercinta. Ucapan terima kasih yang tak terhingga ditujukan kepada kedua orang tua yang senantiasa memberikan doa, dukungan, dan kasih sayang tanpa syarat sepanjang perjalanan perkuliahan ini. Terima kasih kepada saudara-saudara dan sahabat yang selalu hadir memberikan semangat di saat suka maupun duka. Rasa terima kasih yang sebesar-besarnya juga disampaikan kepada dosen pembimbing yang telah dengan sabar meluangkan waktu dan ilmunya dalam membimbing penyelesaian skripsi ini, serta kepada dosen penguji yang telah memberikan kritik dan masukan berharga demi penyempurnaan penelitian ini. Terakhir, apresiasi ini layak diberikan kepada diri sendiri karena telah bertahan dan berjuang hingga saat ini.

Muhammad Faqih Ajiputra
NIM. 2209106114
Program Studi Informatika

Dosen Pembimbing
I. Rosmasari, S.Kom., M.T.
II. Prof. Dr.. Fahrul Agus, S.Si., M.T.

LONG SEQUENCE TIME-SERIES FORECASTING PADA PASAR VALUTA ASING EUR/USD MENGGUNAKAN MODEL INFORMER

ABSTRAK

Pasar valuta asing EUR/USD bersifat volatil, non-linear, dan memiliki dependensi jangka panjang sehingga peramalan harga menggunakan *sequence* panjang menjadi menantang. Penelitian ini membangun model Informer dengan mekanisme ProbSparse Self-Attention untuk meramalkan log return EUR/USD, merekonstruksinya menjadi harga close, serta menganalisis pengaruh panjang *input sequence* dan horizon prediksi terhadap kinerja model. Tahapan penelitian mencakup pengumpulan dan pembersihan data, pembentukan fitur RSI, ATR, dan log return, normalisasi, penyusunan data supervised learning menggunakan sliding window, pelatihan dengan optimizer Adam dan early stopping, rekonstruksi harga, serta evaluasi menggunakan MAE, MSE, dan MAPE. Data yang digunakan terdiri atas 95.544 observasi EUR/USD berinterval satu jam dari 7 April 2010 hingga 17 April 2026, dengan fitur Open, High, Low, Close, Volume, RSI, ATR, dan log return. Sebanyak 12 skenario kombinasi panjang *input sequence* dan horizon prediksi diuji, dengan hasil terbaik pada skenario S384-P1 yang memperoleh MAE 0,000654, MSE $1,06 \times 10^{-6}$, dan MAPE 0,058907%. Analisis menunjukkan bahwa horizon prediksi lebih memengaruhi kinerja model daripada panjang *input sequence*, ditandai dengan peningkatan rata-rata MAPE dari 0,058964% pada horizon 1 jam menjadi 0,092738% dan 0,125926% pada horizon 4 dan 8 jam. Hasil tersebut menunjukkan bahwa Informer efektif untuk peramalan EUR/USD jangka pendek, sedangkan prediksi banyak langkah masih memerlukan optimasi fitur, konfigurasi model, dan strategi pelatihan untuk mengurangi akumulasi kesalahan.

Kata kunci : *EUR/USD, Informer, long sequence time-series forecasting, log return*

Muhammad Faqih Ajiputra
NIM. 2209106114
Informatics Study Program

Supervisors
I. Rosmasari, S.Kom., M.T.
II. Prof. Dr.. Fahrul Agus, S.Si., M.T.

LONG SEQUENCE TIME-SERIES FORECASTING ON THE EUR/USD FOREIGN EXCHANGE MARKET USING THE INFORMER MODEL

ABSTRACT

The EUR/USD foreign exchange market is volatile, nonlinear, and characterized by long-term dependencies, making price forecasting with long sequences challenging. This study develops an Informer model using the ProbSparse Self-Attention mechanism to forecast EUR/USD log returns, reconstruct them into closing prices, and analyze the effects of input sequence length and forecast horizon on model performance. The research stages include data collection and cleaning, RSI, ATR, and log-return feature generation, normalization, supervised dataset formation using a sliding-window approach, model training with the Adam optimizer and early stopping, price reconstruction, and evaluation using MAE, MSE, and MAPE. The dataset consists of 95,544 hourly EUR/USD observations from April 7, 2010, to April 17, 2026, containing Open, High, Low, Close, Volume, RSI, ATR, and log-return variables. 12 combinations of input sequence lengths and forecast horizons were evaluated, with the S384-P1 scenario achieving the best performance, producing an MAE of 0.000654, an MSE of 1.06×10^{-6} , and a MAPE of 0.058907%. The analysis shows that the forecast horizon has a greater effect on model performance than the input sequence length, as the average MAPE increases from 0.058964% for the 1-hour horizon to 0.092738% and 0.125926% for the 4-hour and 8-hour horizons. These results indicate that Informer is effective for short-term EUR/USD forecasting, whereas multistep forecasting still requires feature optimization, model configuration, and improved training strategies to reduce accumulated errors.

Keywords: EUR/USD, Informer, long sequence time-series forecasting, log return

KATA PENGANTAR

Puji syukur kepada Allah SWT, Tuhan Yang Maha Esa sehingga dapat menyelesaikan proposal skripsi dengan judul **LONG *SEQUENCE* TIME-SERIES FORECASTING PADA PASAR VALUTA ASING EUR/USD MENGGUNAKAN MODEL INFORMER**. Proposal ini disusun sebagai salah satu tahapan dalam menyelesaikan skripsi pada Fakultas Teknik, Universitas Mulawarman.

Oleh karena itu, pada kesempatan ini kami ingin mengucapkan terima kasih kepada semua pihak yang telah mendukung serta membantu saya selama proses penyusunan proposal skripsi, kepada:

1. Orang tua dan Saudara-saudara saya atas do'a, bimbingan serta kasih sayangnya.
2. Bapak Prof. Dr. Ir. H. Tamrin, S.T., M.T., IPU., APEC Eng., selaku Dekan Fakultas Teknik Universitas Mulawarman, atas fasilitas dan dukungan yang diberikan selama proses perkuliahan. Nama dan gelar akademik Koordinator Prodi selaku K Program Studi Informatika
3. Bapak Awang Harsa Kridalaksana, S.Kom., M.Kom., selaku Koordinator Program Studi Informatika, atas bimbingan dan arahannya selama penyusunan proposal ini.
4. Ibu Rosmasari, S.Kom., MT, selaku Dosen Pembimbing I, yang telah memberikan arahan, saran, dan motivasi dalam penyusunan proposal ini.
5. Bapak Prof. Dr. Fahrul Agus, S.Si., M.T., selaku Dosen Pembimbing II, yang telah memberikan bimbingan dan masukan yang sangat berharga bagi penyempurnaan penelitian ini. Segenap Dosen Program Studi Informatika, yang telah memberikan ilmu pengetahuan selama mengikuti perkuliahan.
6. Segenap Dosen Program Studi Informatika, yang telah memberikan ilmu pengetahuan selama mengikuti perkuliahan.
7. Rekan-rekan seperjuangan yang terus memberikan dukungan semangat demi terselesainya tugas ini.

Saya menyadari bahwa proposal skripsi ini tidak luput dari berbagai kekurangan. Oleh karena itu, semua kritik dan saran yang bersifat memperbaiki demi kesempurnaan sangat diharapkan.

Samarinda,..... 2026

Penulis

DAFTAR ISI

	<i>halaman</i>
HALAMAN JUDUL	i
PERNYATAAN KEASLIAN SKRIPSI.....	ii
HALAMAN PENGESAHAN.....	iii
ABSTRAK	v
<i>ABSTRACT</i>	vi
KATA PENGANTAR.....	vii
DAFTAR ISI	viii
DAFTAR TABEL	xi
DAFTAR GAMBAR.....	xii
DAFTAR LAMPIRAN.....	xiii
DAFTAR ISTILAH/LAMBANG	xiv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	2
1.3 Batasan Masalah	2
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	3
1.6 Kontribusi Penelitian	4
BAB II TINJAUAN PUSTAKA	5
2.1 Penelitian Terkait.....	5
2.2 Pasar Valuta Asing (FOREX).....	17
2.2.1 Log Return.....	18
2.2.2 Relative Strength Index (RSI)	18
2.2.3 Average True Range (ATR).....	20
2.2 Machine Learning dan Deep Learning	21
2.3.1 Machine Learning	21
2.3.2 Deep Learning	22
2.4 Informer	22
2.4.1 Probsparse Self-Attention Layer	24

2.4.2	Distilling Layer	25
2.4.3	Feed Forward Network (FFN).....	28
2.4.4	Masked ProbSparse Self-Attention Layer.....	29
2.4.5	Cross-Attention Layer	30
2.4.6	Linear Projection Layer.....	31
2.5	Pelatihan Model	31
2.5.1	Adam (Adaptive Moment Estimation)	32
2.6	Evaluasi Model	34
2.6.1	Mean Absolute Error (MAE)	35
2.6.2	Mean Squared Error (MSE)	35
2.6.3	Mean Absolute Percentage Error (MAPE).....	36
BAB III METODOLOGI PENELITIAN		37
3.1	Tahapan Pelaksanaan Penelitian	37
3.2	Pengumpulan Data	39
3.3	Perancangan Data	40
3.4	Perancangan Proses/Algoritma	41
3.5	Perancangan Pengujian	45
BAB IV HASIL DAN PEMBAHASAN		48
4.1	Penerapan/Pengolahan Data	48
4.1.1	Pengumpulan dan Penyaringan Data.	48
4.1.2	Validasi Kualitas Data	53
4.1.3	Pembagian Data	55
4.1.4	Normalisasi Data dan Formasi <i>Sliding Window</i>	58
4.2	Implementasi Proses Model Informer.....	61
4.2.1	Konfigurasi Arsitektur Model Informer	62
4.2.2	Konfigurasi Pelatihan Model dan Proses Pelatihan Model	70
4.3	Hasil Pengujian	75
4.4	Pembahasan	80
BAB V KESIMPULAN DAN SARAN.....		87
5.1	Kesimpulan	87
5.2	Saran	88
DAFTAR PUSTAKA.....		90

LAMPIRAN 93

DAFTAR TABEL

Halaman

Tabel 2. 1 Tabel Perbedaan Penelitian.....	13
Tabel 3. 1 Skema struktur data EUR/USD	40
Tabel 3. 2 Konfigurasi perancangan model	46
Tabel 3. 3 Konfigurasi hyperparameter model	46
Tabel 4. 1 Sepuluh Baris Pertama Data Mentah EUR/USD.....	49
Tabel 4. 2 Sepuluh Baris Pertama Pada Data yang Sudah Diberi Penamaan Kolom.....	51
Tabel 4. 3 Dua Puluh Baris Pertama Data EURUSD Setelah Diproses	53
Tabel 4. 4 Hasil Validasi Kualitas Data.....	54
Tabel 4. 5 Pembagian Dataset Untuk Model	57
Tabel 4. 6 Skenario Panjang Sequence yang Akan Diuji	61
Tabel 4. 7 Konfigurasi Arsitektur Model Informer	62
Tabel 4. 8 Konfigurasi Hyperparameter Pelatihan	70
Tabel 4. 9 Ringkasan Pelatihan Setiap Skenario	74
Tabel 4. 10 Contoh Rekonstruksi Harga Close.....	76
Tabel 4. 11 Hasil Evaluasi Seluruh Skenario	77
Tabel 4. 12 Rata-rata Hasil Evaluasi Berdasarkan Panjang Prediksi	79

DAFTAR GAMBAR

	Halaman
Gambar 2. 1 Arsitektur Informer	23
Gambar 3. 1 Tahapan Pelaksanaan Penelitian	37
Gambar 3. 2 Contoh 10 data pertama EUR/USD yang telah diunduh	39
Gambar 3. 3 Flowchart Perancangan Proses/Algoritma	41
Gambar 3. 4 Arsitektur Model Informer	43
Gambar 4. 1 Tampilan Halaman Pengambilan Data forexsb.com	48
Gambar 4. 2 Konfigurasi Data yang Diunduh	49
Gambar 4. 3 Kode Program Penamaan Kolom dan Penambahan RSI, ATR, dan Log Return.....	50
Gambar 4. 4 Dua Puluh Baris Pertama Kolom Log Return, RSI, dan ATR.....	52
Gambar 4. 5 Kode Program Pembagian Data Cadangan dan Model.....	56
Gambar 4. 6 Kode Program Implementasi Standard Scaler dan Time Feature Extraction	58
Gambar 4. 7 Kode Program Implementasi Sliding Window Pada Formasi Dataset	60
Gambar 4. 8 Implementasi ProbSparse Self-Attention dan Attention Layer	65
Gambar 4. 9 Kode Program Implementasi Encoder, Decoder, dan Distilling.....	68
Gambar 4. 10 Kode Program Implementasi Arsitektur Model Informer	70
Gambar 4. 11 Kode Program Konfigurasi Hyperparameter Pelatihan	71
Gambar 4. 12 Kode Program Implementasi Early Stopping	72
Gambar 4. 13 Kode Program Implementasi Skenario Pelatihan	73
Gambar 4. 14 Seratus Prediksi Sampel Dengan Panjang Prediksi 1 Jam.....	82
Gambar 4. 15 Seratus Prediksi Sampel Dengan Panjang Prediksi 4 Jam.....	83
Gambar 4. 16 Seratus Prediksi Sampel Dengan Panjang Prediksi 8 Jam.....	83

DAFTAR LAMPIRAN

	<i>Halaman</i>
Lampiran 1 Source Code Model Informer	93
Lampiran 2 Source Code Pemrosesan Dataset	93
Lampiran 3 Dataset Mentah	94
Lampiran 4 Dataset Terproses	94
Lampiran 5 Hasil Pengujian	95

DAFTAR ISTILAH/LAMBANG

Arti

Contents

<i>Adaptive Scaling</i>	Penskalaan data time-series secara dinamis sesuai volatilitas lokal.
<i>Attention</i>	Mekanisme fokus pada bagian penting <i>sequence input</i> untuk menangkap dependensi jangka panjang.
<i>Average Pooling</i>	Downsampling dengan menghitung rata-rata nilai dalam window.
<i>Back-propagation</i>	Propagasi balik kesalahan untuk memperbarui bobot selama pelatihan.
<i>Batch size</i>	Jumlah sampel yang diproses sekaligus dalam satu iterasi pelatihan.
<i>Bias</i>	Parameter penyesuaian output neuron yang independen dari <i>input</i> .
<i>Causal mask</i>	Topeng yang mencegah model melihat data masa depan saat decoding.
<i>Dot-product</i>	Perkalian titik antara query dan key pada mekanisme attention.
<i>Downsample</i>	Pengurangan panjang <i>sequence</i> untuk menekan kompleksitas komputasi.
<i>Dropout rate</i>	Probabilitas neuron yang dimatikan secara acak saat pelatihan.
<i>Early stopping</i>	Penghentian pelatihan otomatis saat validasi tidak membaik.
<i>Forecasting</i>	Prediksi nilai masa depan berdasarkan data deret waktu historis.
<i>Forward pass</i>	Perhitungan maju dari <i>input</i> hingga output model.
<i>Generative style</i>	Cara decoding Informer yang menghasilkan seluruh <i>sequence</i> output sekaligus.
<i>Gradient descent</i>	Algoritma optimasi yang memperbarui bobot menuju minimum loss.
<i>Gradient</i>	Turunan fungsi loss yang menunjukkan arah perbaikan bobot.
<i>Gradient Flow</i>	Aliran gradien selama back-propagation pada model deep learning.

<i>Hyperparameter</i>	Parameter yang ditentukan sebelum pelatihan (bukan dipelajari dari data).
<i>Layer Unit</i>	struktural dalam arsitektur neural network (<i>encoder/decoder</i>).
<i>Learning rate</i>	Ukuran langkah pembaruan bobot pada setiap iterasi.
<i>Logarithmic return</i>	target prediksi yang stasioner.
<i>Loss function</i>	Fungsi pengukur kesalahan prediksi model (MAE, MSE, MAPE).
<i>Max Pooling</i>	Downsampling dengan mengambil nilai maksimum dalam window.
<i>Missing value</i>	Nilai data yang hilang dan harus ditangani sebelum pelatihan.
<i>Noise</i>	Gangguan acak pada data yang tidak mengandung pola.
<i>Outliers</i>	Nilai ekstrem yang jauh dari pola data umum.
<i>Over-the-counter</i>	Pasar forex desentralisasi tanpa bursa terpusat.
<i>Overfitting</i>	Model terlalu menyesuaikan data latih sehingga buruk pada data baru.
<i>Over-prediction</i>	Prediksi lebih tinggi daripada nilai aktual.
<i>Random shuffling</i>	Pengacakan urutan data latih untuk mengurangi bias.
<i>Sequence</i>	Deret data berurutan (<i>input/output</i> model).
<i>Stack</i>	Penumpukan beberapa layer yang sama secara berurutan.
<i>Stride</i>	Jarak pergeseran window saat pooling atau convolution.
<i>Tick</i>	Unit terkecil pergerakan harga forex.
<i>Timestep</i>	Satuan waktu diskrit dalam deret waktu.
<i>Trading</i>	Aktivitas jual-beli mata uang pada pasar forex.
<i>Tuning</i>	Penyesuaian hyperparameter untuk performa optimal.
<i>Under-prediction</i>	Prediksi lebih rendah daripada nilai aktual.
<i>Weight</i>	Bobot koneksi antar neuron yang dipelajari selama pelatihan.
<i>Window</i>	Segmen data berukuran tetap sebagai <i>input</i> model.
<i>Closing price</i>	Harga terakhir suatu aset pada akhir interval waktu tertentu.
<i>Cross-attention</i>	Mekanisme perhatian yang menghubungkan informasi dari <i>encoder</i> dengan <i>decoder</i> .
<i>Dataset</i>	Kumpulan data yang digunakan dalam pelatihan, validasi, dan pengujian model.

<i>Decoder</i>	Bagian model yang menghasilkan prediksi berdasarkan informasi dari <i>encoder</i> .
Distilling layer	Lapisan Informer yang mengurangi panjang sequence dengan mempertahankan informasi penting.
Embedding	Proses mengubah data masukan menjadi representasi numerik yang dapat diproses model.
<i>Encoder</i>	Bagian model yang mempelajari pola dan representasi data historis.
Epoch	Satu siklus ketika seluruh data latih telah diproses oleh model.
Input sequence	Deret data historis yang digunakan sebagai masukan model.
Optimizer	Algoritma yang memperbarui parameter model untuk mengurangi nilai kesalahan.
Output sequence	Deret nilai masa depan yang dihasilkan atau diprediksi oleh model.
Random seed	Nilai awal yang menjaga hasil proses acak agar dapat direproduksi.
Sliding window	Metode pembentukan sampel dengan menggeser jendela data secara bertahap.
Supervised learning	Pembelajaran model menggunakan data masukan yang memiliki nilai target.
Test set	Bagian data yang digunakan untuk mengevaluasi kinerja akhir model.
Time series	Data yang disusun secara berurutan berdasarkan waktu.
Timeframe	Interval waktu yang digunakan pada setiap observasi data.
Training set	Bagian data yang digunakan untuk mempelajari dan memperbarui parameter model.
Validation set	Bagian data yang digunakan untuk memantau kinerja model selama pelatihan.

DAFTAR SINGKATAN

Arti

Contents

ATR	<i>Average True Range</i>
EUR/USD	<i>Euro pada US Dollar</i>
FOREX	<i>Foreign Exchange</i>
H1	<i>Timeframe 1 Jam</i>
LSTF	<i>Long Sequence Time-Series Forecasting</i>
LSTM	<i>Long Short-Term Memory</i>
MAE	<i>Mean Absolute Error</i>
MAPE	<i>Mean Absolute Percentage Error</i>
MSE	<i>Mean Squared Error</i>
OHLCV	<i>Open-High-Low-Close-Volume</i>
OTC	<i>Over-the-Counter</i>
RSI	<i>Relative Strength Index</i>
FFN	<i>Feed Forward Network</i>
RNN	<i>Recurrent Neural Network</i>
CNN	<i>Convolutional Neural Network</i>
GRU	<i>Gated Recurrent Unit</i>
RMSE	<i>Root Mean Squared Error</i>
NRMSE	<i>Normalized Root Mean Squared Error</i>
SVR	<i>Support Vector Regression</i>
RF	<i>Random Forest</i>
SVM	<i>Support Vector Machine</i>
MLP	<i>Multilayer Perceptron</i>
TFT	<i>Temporal Fusion Transformer</i>
ARIMA	<i>AutoRegressive Integrated Moving Average</i>
GPU	<i>Graphics Processing Unit</i>
ADF	<i>Augmented Dickey–Fuller</i>
EMA	<i>Exponential Moving Average</i>

MACD	<i>Moving Average Convergence Divergence</i>
SMA	<i>Simple Moving Average</i>
CCI	<i>Commodity Channel Index</i>
ALFA	<i>Attention-Based LSTM for Forex</i>
ReLU	<i>Rectified Linear Unit</i>
GeLU	<i>Gaussian Error Linear Unit</i>
ELU	<i>Exponential Linear Unit</i>

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pasar Valuta Asing atau Foreign Exchange Market (Forex Market) Adalah pasar keuangan di mana mata uang saling diperjualbelikan. Dengan volume transaksi lebih dari 5 triliun dolar AS. Membuatnya menjadi pasar keuangan terbesar di dunia (Yıldırım et al., 2021). Dengan Volume yang tinggi tersebut, menyebabkan volatilitas yang tinggi pada pergerakan harganya. Karena volatilitas pergerakan harga yang tinggi, menyebabkan analisis teknikal yang menyulitkan bagi beberapa orang, khususnya bagi retail trader. Menurut Ayitey Junior et al. (2023) hanya 2% retail trader yang sukses dalam melakukan prediksi pergerakan mata uang di Forex Market, menjadikannya hal yang sangat menantang. Karena itu model Machine Learning dan Deep Learning menjadi sangat populer dalam melakukan prediksi pergerakan pasar.

Forex memiliki dependensi terhadap faktor makroekonomi seperti suku bunga, inflasi, geopolitik, dll. Karena dependensi tersebut, menyebabkan forex memiliki volatilitas, non-linearitas, serta trend yang menyebabkan prediksi jangka panjang menyulitkan (Mach et al., 2025). Karena saat ingin melakukan prediksi jangka panjang, membutuhkan konteks data historis yang panjang juga.

Dengan berkembangnya teknologi kecerdasan buatan, banyak metode yang dapat digunakan untuk melakukan prediksi pergerakan harga. Dari berbagai model, Model Long Short Term Memories(LSTM) menjadi model yang sering digunakan untuk melakukan prediksi harga. Tetapi model LSTM kurang baik saat melakukan prediksi deretan waktu panjang. Menurut (Zhou et al., 2021) saat model LSTM melakukan prediksi pada perubahan temperatur Gardu Induk pada jangka waktu pendek (12 poin, 0.5 hari) hingga jangka waktu yang panjang (480 poin, 20 hari), performa LSTM mengalami penurunan ketika melakukan prediksi lebih dari 48 poin atau 2 hari. Di mana MSE mengalami lonjakan yang signifikan, dan kecepatan inferensi mengalami penurunan yang tajam, dan model LSTM mengalami kegagalan.

LSTM juga memiliki beberapa keterbatasan, walaupun LSTM di desain untuk menangkap korelasi data berurutan yang lebih panjang dibanding arsitektur RNN, tetapi gagal untuk mengingat informasinya. LSTM tidak memiliki kemampuan untuk melakukan revisi pada historis *forget gate* nya, yang membatasi fleksibilitasnya untuk beradaptasi terhadap perubahan data (Kong et al., 2025). Maka dari itu LSTM mengalami penurunan saat *input sequence* diperpanjang, di mana memperpanjang *input sequence* dapat memberikan konteks terhadap data lebih baik.

Arsitektur model Transformer dapat mengatasi kelemahan dari LSTM, dengan mekanisme self-attention arsitektur Transformer dapat memberikan konteks pada data lebih baik. Tetapi kelemahan dari arsitektur Transformer adalah komputasinya yang sangat berat dibandingkan dengan model LSTM. Mekanisme dari self-attention pada model Transformer menyebabkan fitur waktu dan penggunaan memori yang banyak, sehingga kompleksitasnya menjadi $O(L^2)$ (Zhou et al., 2021). Maka dari itu melakukan prediksi dengan window yang banyak akan menyebabkan meningkatnya penggunaan memori.

Oleh sebab itu, penelitian ini mengangkat judul Long *Sequence* Time-Series Forecasting (LSTF) Pada Pasar Mata Uang Asing Menggunakan Model Informer pada data pasar valuta asing EUR/USD. Hal ini bertujuan agar dapat mengetahui bagaimana model Informer melakukan prediksi harga EUR/USD dengan waktu deret yang panjang.

1.2 Rumusan Masalah

Berdasarkan latar belakang penelitian, maka yang menjadi rumusan masalah dalam penelitian ini adalah :

1. Bagaimana menerapkan model Informer untuk melakukan Forecasting dengan *Sequence* yang panjang pada dataset Forex EUR/USD.
2. Bagaimana kinerja model Informer dalam melakukan forecasting dengan panjang *input* dan panjang output yang berbeda.

1.3 Batasan Masalah

Penelitian ini disusun berdasarkan data-data yang diperoleh. Luasnya bidang yang dihadapi membuat penelitian ini harus dibatasi. Ruang lingkup masalah dibatasi pada:

1. Data yang digunakan dalam penelitian Adalah data historis pasar valuta asing dengan pasangan mata uang EUR/USD.
2. Timeframe yang digunakan adalah H1 (1 Jam).
3. Target penelitian ini adalah nilai Log return pada data Closing price.
4. *Input Sequence* yang di-uji pada interval 48, 96, 192, dan 384.
5. *Output Sequence* yang di-uji pada interval 1, 4, dan 8.
6. Sampling factor yang diuji pada nilai 5.

1.4 Tujuan Penelitian

Tujuan yang ingin dicapai dalam penelitian ini adalah:

1. Membangun model Informer untuk melakukan forecasting pada Log return pada dataset forex EUR/USD.
2. Menganalisis performa model Informer dalam melakukan forecasting dengan panjang *input* dan panjang output yang berbeda.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat pada berbagai pihak, khususnya:

1. Penulis

Manfaat penelitian bagi penulis adalah untuk mengembangkan wawasan penulis dalam merancang dan mengembangkan algoritma Informer dalam melakukan prediksi deret panjang pada pasar valuta asing dengan pasangan mata uang EUR/USD.

2. Mahasiswa

Penelitian ini dapat memberikan pengetahuan kepada mahasiswa bagaimana algoritma Informer diterapkan dalam melakukan peramalan deret waktu panjang dan menjadi referensi khususnya bagi mahasiswa dibidang Informatika yang dapat membantu proses pembelajaran.

3. Instansi/Lembaga/Perusahaan

Manfaat penelitian ini bagi instansi/Lembaga/perusahaan adalah dengan adanya hasil analisa long *sequence* time series forecasting pada pasar valuta asing EUR/USD

diharapkan dapat membantu Instansi/Lembaga/Perusahaan untuk melakukan analisis pada pasar valuta asing EUR/USD.

1.6 Kontribusi Penelitian

Kontribusi utama dalam penelitian ini adalah pengembangan model Informer untuk melakukan peramalan deret waktu panjang yang secara spesifik dilatih untuk karakteristik pasar valuta asing dengan pasangan mata uang EUR/USD.

BAB II

TINJAUAN PUSTAKA

2.1 Penelitian Terkait

Dalam rangka mendukung penelitian ini, maka dilakukan kajian dengan mempelajari penelitian-penelitian terkait yang telah dilakukan sebelumnya. Daftar penelitian terkait antara lain:

1. **Metode yang digunakan:** SVR, LSTM, Transformer, Informer, Autoformer, dan NSTransformer. **Temuan Penelitian:** Penelitian ini menggunakan 3 Indeks Saham Utama yaitu: S&P 500 (SPX), Hang Seng Index (HSI), dan China Securities Index (CSI 300). Dari ketiga Indeks saham Utama tersebut dimiliki 6 fitur yaitu OHLCV (Opening price, Highest price, Lowest price, Closing price, Trading Volume) pada timeframe daily dan didapatkan 4295 bar. Preprocessing data tersebut melibatkan Z-score normalization dan dibagi dengan rasio 7:1:2, didapatkan 3005 Train, 430 validation, dan 860 test. Model dilatih dengan Adam optimizer pada batch size 32 dengan 20 epochs. Model memiliki *input* 5 hari sebelum pada *encoder*, lalu 2 hari sebelum pada *decoder* dengan output 1 timestep. Lalu model dievaluasi dengan MAE dan RMSE dengan stoploss pada MAE saat tidak terjadi peningkatan selama 3 kali berturut. Didapatkan bahwa NSTransformer menjadi model terbaik secara keseluruhan dengan MAE 0.04304 dan RMSE 0.05891 pada CSI300, MAE 0.07662 dan RMSE 0.10282 pada HSI, dan MAE 0.08563 dan RMSE 0.12087 pada SPX. Di mana NSTransformer berhasil menurunkan MAE sebesar 57.53% dan RMSE sebesar 54.76% dibanding Vanilla Transformer pada prediksi CSI (Wu, 2023)
2. **Metode yang digunakan:** Random Forest (RF) dan Support Vector Regression (SVR) **Temuan Penelitian:** Dalam penelitian ini, model dilatih dengan menggunakan 12 pasang mata uang euro untuk melakukan prediksi perubahan nilai tukar pada 3 pasang mata uang euro yaitu: EUR/AUD, EUR/GBP, dan EUR/PLN pada horizon 1, 5, 10, 15, 30, 45, dan 60 menit. Data dalam penelitian ini memiliki

12 kolom dan 121,673 baris yang melalui data preprocessing dengan missing-value imputation dan min-max normalization dengan range [0,1]. Data di dibagi menjadi 80% untuk train, 20% untuk testing, dan 5-fold cross-validation pada training set digunakan untuk melakukan tuning. Didapatkan parameter terbaik pada SVR adalah $\epsilon = 0.01$, kernel scale = 0.1, dan box constraint = 20, lalu parameter random forest terbaik adalah minimum leaf size = 20, maximum number of split = 100, dan sampled variable = 50. Di Penelitian ini menguji 4 kasus berbeda dengan *input* yang sama yaitu: 1). Random Forest. 2). Support Vector Regression. 3). Average RF+SVR 4). Stacked RF+SVR. Didapatkan bahwa kasus ke 4 yaitu Stacked RF+SVR mendapatkan hasil keseluruhan terbaik, dengan metrik MSE, MAE, dan MAPE terendah dibandingkan 3 kasus lainnya. Lalu didapatkan juga bahwa akurasi terbaik pada horizon 1 menit karena kian memburuk saat horizon prediksi ditingkatkan. Pada pair EUR/USD dengan horizon +1 menit dan Stacked RF+SVR didapatkan nilai MSE 0.000006, MAE 0.001622, dan MAPE 0.247237%, dan walau horizon ditingkatkan menjadi 60+ menit metrik evaluasi mendapatkan yang terbaik dengan nilai MSE 0.000390, MAE 0.014743, dan MAPE 2.247728%. Menggabungkan model RF dan SVR menjadi stacked model adalah pilihan yang baik karena nilai ANOVA p-values di antara 0.0103 dan 0.0195 (Jazayeri, 2024).

3. **Metode yang digunakan:** MLP, CNN, RNN, LSTM, CNN-LSTM, dan CNN-TLSTM. **Temuan penelitian:** Penelitian ini menggunakan data USD/CNY pada timeframe harian dengan periode waktu dari 2 januari 2006 hingga 30 October 2020 untuk melakukan prediksi Harga penutupan. Terdapat 8 fitur yang digunakan sebagai berikut: 1. Open Price. 2. High Price. 3. Low Price. 4. Close Price. 5. Closing Price Shanghai Composite Index. 6. Closing Price Nasdaq Index. 7. Closing Price Hang Seng Index. 8. Closing Price Dow Jones Industrial Average. Sebelum melakukan melatih model, dilakukan ADF stationary test pada data mentah untuk melakukan First-order differencing agar data non-stasioner menjadi stasioner. Lalu model menggunakan 5 time step sebagai *input* untuk melakukan prediksi 1 step ke depan. Didapatkan parameter paling optimal untuk model CNN-TLSTM adalah dengan 60 convolutional filters, 2 kernel size, sigmoid activation pada convolution layer, valid padding, 1 pooling size, 60 hidden units pada

TLSTM, tanh activation pada TLSTM, Adam optimizer, learning rate sebesar 0.001, MAE untuk *loss function*, dilatih dengan 50 epochs, dan dengan batch size 60. Ke-enam model perbandingan dilatih sebanyak 50 kali untuk didapatkan model terbaiknya, yang digunakan untuk tes prediksi. Hasil akhir menunjukkan bahwa model terbaik adalah CNN-TLSTM dengan menunjukkan peningkatan performa dibandingkan model CNN-LSTM dengan berhasil menurunkan nilai MAPE dari 0.20728 menjadi 0.18945 lalu MSE dari 0.00042 menjadi 0.00038 dan dengan nilai R^2 0.98950, dengan nilai tersebut membuktikan hasil modifikasi gate TLSTM menjaga informasi penting dari data sehingga prediksi model menjadi lebih akurat (Wang et al., 2021).

4. **Metode yang digunakan:** Transformer, Informer, dan Temporal Fusion Transformer (TFT). **Temuan penelitian:** Penelitian ini menggunakan 4 pasang mata uang yaitu NZD/USD, NZD/CNY, NZD/GBP, dan NZD/AUD pada timeframe harian. Didapatkan setiap pasang mata uang dengan periode 3 januari 2005 hingga 2 february 2024 sebanyak 4980 sample yang setiap sample terdapat Open price, High price, Low price, Close price, dan floating-price. Dengan Close price sebagai target, data dibagi menjadi 80% untuk training dan 20% untuk testing. Untuk model Transformer, digunakan *Input window* sebesar 7, batch size 100, learning rate 0.0005, dan 150 epoch. Pada model Informer, parameter yang digunakan adalah *Input window* sebesar 64, prediction length 5, batch size 128, learning rate 0.00005 dan 60 epochs. Lalu untuk model TFT, parameter yang digunakan adalah learning rate 0.001, hidden size 32, attention head size 1, output size 8, batch size 128, dan 150 epoch, dengan early stop menggunakan framework Pytorch-Lightning. Ketiga model menggunakan evaluasi metrik yaitu, RMSE, MAE, MAPE, dan R^2 . Secara rata-rata dari keempat dataset, Transformer mendapatkan nilai RMSE 0.0329, MAPE 0.0097, MAE 0.0160, dan R^2 sebesar 0.8922. Pada model Informer nilai evaluasi metrik nya adalah RMSE 0.0201, MAPE 0.0095, MAE 0.0160, dan R^2 sebesar 0.8893. Lalu pada model TFT sebagai terbaik secara keseluruhan mendapatkan nilai RMSE 0.00111, MAPE 0.0055, MAE 0.0043, dan R^2 0.9499. Model TFT juga memiliki performa sangat baik pada pasangan mata uang NZD/USD dan NZD/CNY, Sedangkan model

Informer memiliki proses pelatihan yang lebih cepat, dibuktikan dengan hanya membutuhkan 60 epochs untuk mencapai prediksi yang stabil (Zhao & Yan, 2024).

5. **Metode yang digunakan:** RNN, LSTM, GRU. **Temuan Penelitian:** Penelitian ini melakukan prediksi volatilitas pada pasar valuta asing. Data yang digunakan pada pelatihan adalah pasangan mata uang EUR/USD, GBP/USD, USD/CAD, dan USD/CHF pada timeframe 4 jam dengan periode waktu dari 28 Agustus 2014 hingga 29 Desember 2023. Target dari penelitian ini adalah Range-Based Volatility yang didapatkan dengan $\log(\text{high}) - \log(\text{low})$. Digunakan juga 2 fitur kompleksitas tambahan yaitu Hurst Exponent dan Fuzzy Entropy yang dihitung dengan 1024 sampel data dari interval sebelumnya. Penelitian ini menggunakan 4 pengujian dengan jumlah fitur yang berbeda yaitu: 1. Kasus 1: Range-Based Volatility. Kasus 2: Ranged-Based Volatility, High price, dan Low price. Kasus 3: Range-Based Volatility, Hurst Exponent, dan FuzzyEn. Kasus 4: Ranged-Based Volatility, High price, Low price, Hurst Exponent, dan FuzzyEn. Setiap kasus memiliki target prediksi yang sama yaitu Ranged-Based Volatility. Data melalui preprocessing dengan normalisasi data dengan interval $[0,1]$, lalu pembagian 60% untuk data training dan 40% untuk data testing. Hyperparameter yang digunakan adalah three-fold grid-search cross-validation pada fungsi tanh activation, Adam optimaizer, batch size (16/32/64), epochs (10/50/100), dan jumlah neuron (8/16/32/64). Lalu performa model akan diuji dengan 4 metrik evaluasi yaitu MAE, RMSE, NRMSE, dan Directional Symmetry. Hasil akhir menunjukkan bahwa model LSTM dan GRU secara konsisten melampaui perfoma model GRU, dan dengan menambahkan 2 fitur kompleksitas tambahan Hurst Exponent dan Fuzzy Entropy berhasil meningkatkan perfoma semua ketiga model yang diuji. Seperti pada pasangan mata uang EUR/USD, dengan model LSTM pada pengujian kasus 4 berhasil meningkatkan hasil MAE 11%, RMSE 26%, NRMSE 37%, dan sebanyak 9% pada Directional Symmetry. Yang menunjukkan bahwa model LSTM dan GRU yang dilengkapi dengan fitur kompleksitas menjadi pilihan yang paling efektif dalam melukan prediksi volatilitas pada pasar valuta asing (Zitis et al., 2024).
6. **Metode yang digunakan:** Dual-input LSTM **Temuan Penelitian:** Penelitian ini membandingkan Dual-input LSTM dengan F-LSTM, T-LSTM, dan FT-LSTM

dalam melakukan prediksi Closing price dari pasangan mata uang EUR/USD menggunakan indikator teknikal dan fundamental dengan 2 cabang arsitektur. Data mata uang EUR/USD terdiri dari High price, Low price, dan close price dalam periode 3 Januari 2000 hingga 14 Maret 2025 dengan interval harian, yang didapatkan total hari sebanyak 6575. Lalu indikator fundamental sebanyak 6, yang terdiri dari: EMA, MACD, RSI, Upper Bollinger Band, Lower Bollinger Band, dan ATR. Karena MACD membutuhkan periode warm-up, 25 sample pertama akan di hilangkan, menyisakan 6550 sample dengan periode 7 Februari 2000 ke depan. Data dibagi menjadi 70% untuk training, 15% untuk validation, dan 15% untuk testing. Setiap cabang memiliki *input* 10 x 6 *sequence*, maka dari itu model menggunakan 10 hari historical pada kedua sisi fundamental dan teknikal dan memprediksi closing price 1 hari ke depan. Arsitektur dalam penelitian ini memiliki 3 layers LSTM dengan 6 unit di setiap cabangnya lalu digabungkan, lalu 2 dense layers dengan 12 dan 3 unit ReLU, dan diakhiri dengan 1 output neuron linear. Setelah melewati validasi, training menggunakan Adam optimizer, MAE, Huber loss, 600 epochs, dan 64 batch size, dan model dilatih sebanyak 100 kali dengan random seeds. Dual-*input* LSTM menjadi yang terbaik dari ketiga model alternatif lainnya. Dengan mendapatkan nilai test RMSE sebesar 0.0256 +- 0.0372 dan nilai test MAE sebesar 0.0211 +- 0.0316, dibandingkan dengan 0.0332/0.0278 pada T-LSTM, 0.0499/0.0393 pada FT-LSTM, dan 0.2014/0.1528 pada F-LSTM. Dibandingkan dengan model kedua terbaik yaitu T-LSTM, Dual-*input* LSTM berhasil menurunkan nilai test MAE sekitar 24% dan nilai test RMSE sekitar 23%. Model juga di uji dengan rolling forecasting (prediksi berulang secara real-time) menunjukkan nilai RMSE 0.0062 dan MAE 0.0047 pada 50 titik forecast acak, dan juga stabil saat melakukan prediksi 3 hari ke depan (Saghafi et al., 2025).

7. **Metode yang digunakan:** Linear Regression, Random Forest, SVM, LSTM, and ARIMA **Temuan Penelitian:** Penelitian ini melakukan prediksi tren pasar valuta asing dengan pasangan mata uang EUR/USD, GBP/USD, dan USD/NGN menggunakan data historis dalam periode waktu 2015 hingga 2025 dengan interval harian. Data berisikan OHLC, Trading volume, dan indikator ekonomi makro yaitu interest-rate differentials, inflation indicies, dan market-sentiment scores.

Preprocessing data dilakukan dengan Forward-fill untuk mengisi data yang hilang dengan nilai sebelumnya, lalu capping outliers untuk membatasi nilai ekstrim di persentil ke-99, lalu Min-max normalization dengan interval $[0, 1]$, dan menghitung indikator teknikal yaitu RSI, MACD, SMA, EMA, dan Bollinger Bands, dan data dibagi 80% untuk data train dan 20% untuk data test, lalu 30 hari sliding window pada model sequential. Untuk LSTM, memiliki 30 timestep dan output binary dari arah harga besoknya (naik = 1 atau turun = 0). Lalu model LSTM memiliki 2 stacked layer dengan unit sebesar 64 dan 32, dropout 0.2, dan dense sigmoid layer pada outputnya. Pelatihan model dilakukan dengan pengaturan Adam optimizer dengan learning rate 0.001, lalu *loss function* dengan Binary cross-entropy, 50 epochs, dan 64 pada batch size. Model di evaluasi dengan 4 evaluasi metrik yaitu MAE, RMSE, Directional Accuracy, dan simulated profitability. Hasil akhir menunjukkan LSTM menjadi model terbaik dengan nilai MAE 0.0018, RMSE 0.0024, Directional Accuracy 82.3%, dan simulated profit sebesar 15.6%. Diikuti dengan dengan model Random Forest nilai evaluasi metrik sebesar 0.0021, 0.0028, 78.7%, dan 12.4%, SVM dengan 0.0025, 0.0031, 75.2%, dan 9.8%, ARIMA dengan 0.0034, 0.0045, 68.5%, dan 6.1%. Penelitian ini menyimpulkan bahwa LSTM menjadi model terbaik dari model lain yang diuji karena lebih baik dalam menangkap ketergantungan temporal dan memberikan profitabilitas dan stabilitas terbaik saat pasar sedang volatile disbanding model lainnya (Chika-Obi, 2026)

8. **Metode yang digunakan:** Transformer dengan time embeddings, LSTM, ARIMA.
Temuan Penelitian: Penelitian ini melakukan prediksi Closing Bid Return pada 3 pasang mata uang yaitu EUR/USD, USD/JPY, dan GBP/USD. Data yang digunakan sebanyak 1273 *input* yang terdiri dari 433 indikator fundamental dan 840 indikator teknikal dalam periode 1 November 2020 hingga 31 Januari 2022 dengan interval Waktu 10 menit. Pada tahap preprocessing, forward-filling dilakukan pada data yang kosong, lalu mengubah data Harga menjadi return untuk meningkatkan ke-stasionerannya, setelah itu data di normalisasi dengan interval $[0, 1]$, dan data dibagi menjadi 80% untuk training set, 10% untuk validation set, dan 10% untuk test set. Model transformer memiliki Panjang *sequence* 128 dalam 10 menit frekuensi, dengan batch size sebesar 32 dan 1273 fitur mentah. Setelah digabung

dengan Time Embeddings, *input* dari *encoder* menjadi 1275. Pada block Attention, memiliki K, Q, dan V dengan dimensi 256 dan 12 *attention heads*. Pelatihan dilakukan dengan 100 epochs dan dengan early stopping, dengan Adam optimizer dan learning rate 0.0001, model menggunakan MSE loss dan Identity activation function pada outputnya. Pada model LSTM, digunakan 5 layer LSTM dengan batch normalization dan tanh activation, 25% dropout untuk *input* univariate dan 50% untuk *input* multivariate, 128 neuron pada setiap layer di *input* univariate dan 256 neuron pada *input* multivariate, dan memiliki learning rate dari 0.001 hingga 0.00001. Hasil akhir menunjukkan bahwa model Transformer melampaui model LSTM pada performa tradingnya, dibuktikan dengan nilai rata-rata return 18.7% melawan 6.8% pada LSTM, dan dengan nilai rata-rata Sharpe ratio 4.4 melawan 1.2 pada model LSTM. Penelitian ini menunjukkan bahwa hanya multivariate Transformer yang sangat diuntungkan Ketika ditambahkan fitur-fitur cross-sectional (Fischer et al., 2024).

9. Metode yang digunakan: ALFA (Attention-based LSTM for Forex) Temuan Penelitian: Penelitian ini bertujuan untuk melakukan prediksi closing rate menggunakan data OHLC dari pasangan mata uang EUR/USD, USD/JPY, dan GBP/JPY. Data yang digunakan berasal dari periode Waktu Maret 2014 hingga April 2024. Data dibagi menjadi 3 bagian dengan rasio 60% training set, 20% validation set, dan 20% testing set. Window size yang digunakan adalah $T = \{2, 4, 8, 16, 32\}$ untuk menemukan Panjang *input* terbaik. *Input* diuji dengan beberapa kasus yaitu, 1 fitur saja, fitur berpasangan, 4 fitur yang berisi OHLC, dan indikator teknikal yang terdiri dari SMA, MACD, ROC, RSI, Bollinger Bands, dan CCI. Arsitektur ALFA memiliki LSTM dengan 76 unit yang diikuti dengan unit attention sebanyak 100, lalu diakhiri dengan dense layer sebanyak 1 unit di outputnya. Penelitian ini menunjukkan bahwa model ALFA menjadi yang terbaik hampir pada setiap kasus dan mendapatkan nilai MAE terbaik pada mayoritas kombinasi fitur dan pasangan mata uang. Dengan mendapatkan nilai MAE sebesar 0.000887 pada pasangan EUR/USD dengan fitur Low Price dan Close Price, lalu nilai MAE sebesar 0.000176 pada pasangan USD/JP dengan fitur High Price dan Open Price. Visualisasi pada Attention Weight mengkonfirmasi bahwa model berhasil

memberikan prioritas tertinggi pada closing price dengan benar. Pada analisis kompleksitas model menunjukkan bahwa model ALFA berhasil mempertahankan keseimbangan dengan baik. Dengan 4.093 parameter dan 0.473 ms pada rata-rata inference time pada setiap sampel, lebih cepat disbanding dengan Transformer dengan Waktu 2.269 dan TCN dengan 4.272 ms sembari mengungguli keduanya dalam akurasi. Dalam simulasi trading yang dilakukan dalam periode 10 Juni 2024 hingga 6 September 2024, model alfa berhasil menghasilkan return sebesar 8.79% hingga 19% setelah setelah biaya di mana trader forex professional bisa menghasilkan return sebesar 5% hingga 30%. Di mana hasil simulasi ini mendemostrasikan keefektifan praktikal dari model ALFA dalam menangani berbagai level volatilitas dan kemampuan untuk menyaring fitur yang tidak relevan dengan mekanisme Attention dalam melakukan prediksi pada forex rate (Ghahremani & Nguyen, 2025).

10. Metode yang digunakan: Informer Temuan penelitian: Penelitian ini melakukan prediksi terhadap Intra-day Closing price pada frekuensi 1 menit dan 5 menit di 4 Index saham. Data yang digunakan adalah 4 index saham yaitu Hong Kong Hang Seng Index (HSI), NASDAQ Index (IXIC), Tencent Holdings Ltd, dan Apple Inc (AAPL), di mana setiap dataset memiliki 65.358–77.264 poin pada frekuensi 1 menit dengan periode 3 Januari 2022 hingga 12 Desember 2022, dan 28.248–33.384 poin pada frekuensi 5 menit dengan periode 5 April 2022 hingga 12 Desember 2022. *Input* feture terdiri dari Open price, High price, Low price, Close price, Volume, dan turnover ratio, dengan closing price sebagai target univariate. Data dibagi menjadi 70% untuk train, 20% untuk testing, dan 10% untuk data validasi. Preprocessing yang dilakukan adalah min-max normalization ke [0, 1]. Untuk model Informer, hyperparameter yang digunakan adalah batch size, 32, panjang *input sequence* 95, *guiding sequence* 48, Panjang prediksi 24, 20 epochs, learning rate dinamis yang dimulai dari 0.00004 dengan GeLU activation, 0.05 dropout, dan early stop dengan 3 validation loss. Hasil dari penelitian ini menunjukkan model Informer menghasilkan nilai MAE, RMSE, dan MAPE terendah secara konsisten di setiap dataset dan frekuensi yang diuji. Seperti contoh di HSI pada frekuensi 1 menit, Informer berhasil mendapatkan nilai MAE 0.0174,

RMSE 0.0305, dan MAPE sebesar 0.0779, lebih baik dibanding model BERT dengan nilai MAE 0.0272, model Transformer dengan nilai MAE 0.0328, dan pada model LSTM dengan nilai MAE 0.0485. Pada inpeksi visual dari 250 poin mengkonfirmasi bahwa model informer superior dalam menangani lonjakan volatilitas yang tajam, khususnya pada saat pasar dibuka dan ditutup di mana volume melonjak secara signifikan. Studi ablasi dengan menghilangkan mekanisme global time-stamp menghasilkan penurunan performa pada model, membuktikan bahwa mekanisme tersebut memiliki peran yang kritis dalam menangkap pola intra-day (Lu et al., 2023).

Tabel 2. 1 Tabel Perbedaan Penelitian

No	Peneliti	Hasil Penelitian	Perbedaan
1	Yumin Wu (2023)	Membandingkan 6 model peramalan yaitu SVR, LSTM, Transformer, Informer, Autoformer, NSTransformer. Hasil menunjukkan NSTransformer menurunkan nilai MAE dan RMSE lebih dari 50% dibandingkan dengan Transformer pada data CSI300.	Penelitian ini mengimplementasikan model Informer pada mata uang EUR/USD. Penelitian ini membandingkan pengaruh panjang <i>input sequence</i> dan panjang <i>output sequence</i> .
2	Kian Jazayeri (2024)	Mengevaluasi model Random Forest, Support Vector Regression, Average RF+SVR, dan Stacked RF+SVR untuk melakukan prediksi pada pair EUR. Hasil penelitian menunjukkan jika Stacked RF+SVR secara konsisten mendapatkan nilai MSE, MAE, dan MAPE terendah di setiap horizon forecastingnya. Akurasi dari prediksi juga menurun saat horizon forecasting diperpanjang.	Penelitian ini menggunakan arsitektur Informer dibandingkan arsitektur konvensional dan berfokus pada performa <i>sequence</i> panjang pada data EUR/USD.

No	Peneliti	Hasil Penelitian	Perbedaan
3	Jingyang Wang, Xiaoxiao Wang, Jiazheng Li, Haiyao Wang (2021)	Perbandingan model MPL, CNN, RNN, LSTM, CNN-LSTM, dan CNN-TLSTM dalam melakukan prediksi pada harga close USD/CNY harian. Hasil penelitian menunjukkan model CNN-TLSTM memiliki performa terbaik dengan menurunkan MAPE dan MSE.	Penelitian ini berfokus pada melakukan evaluasi model Informer dalam melakukan prediksi EUR/USD dengan panjang <i>sequence</i> yang berbeda.
4	Lu Zhao, Wei Qi Yan (2024)	Perbandingan model Transformer, Informer, dan Temporal Fusion Transformer (TFT) dalam melakukan prediksi 4 mata uang asing berbasis New Zealand Dolar (NZD). Hasil penelitian menunjukkan jika model TFT model terbaik dalam akurasi, sedangkan model informer terbaik dalam kecepatan pelatihan modelnya.	Penelitian ini berfokus pada evaluasi arsitektur model Informer dan perbedaan <i>sequence</i> yang digunakan pada model.
5	Pavlos I. Zitis, Stelios M. Potirakis, Alex Alexandridis (2024)	Penelitian ini melakukan prediksi volatilitas dengan RNN, LSTM, dan GRU dengan fitur kompleksitas Hurst Exponent dan Fuzzy Entropy. Hasil penelitian menunjukkan jika penambahan fitur tersebut meningkatkan performa setiap model yang diuji dengan model LSTM dan GRU mengungguli model RNN standar pada nilai evaluasi metrik MAE, RMSE, NRMSE, dan Directional Symmetry.	Penelitian ini melakukan prediksi pada Log return yang akan direkonstruksi menjadi harga Close dan mengevaluasi model Informer yang berarsitektur Transformer.

No	Peneliti	Hasil Penelitian	Perbedaan
6	Abolfazl Saghafi, Maryam Bagherian, Farhad Shokoohi (2025)	Menerapkan arsitektur <i>Dual-Input</i> LSTM yang menggabungkan <i>input</i> indikator teknikal dan indikator fundamental dalam melakukan prediksi mata uang EUR/USD. Hasil penelitian menunjukkan penggunaan 2 cabang paralel model LSTM dapat mengungguli model F-LSTM, T-LSTM, dan FT-LSTM dengan menurunkan nilai RMSE dan MAE dengan tetap mempertahankan performa prediksi multi-day dengan stabil.	Penelitian ini berfokus pada penggunaan 1 model informer menggunakan fitur <i>input</i> OHLCV, RSI, ATR, dan Log return dalam melakukan prediksi pada Log return yang akan direkonstruksi menjadi harga Close.
7	Kpanuku Chika-Obi (2026)	Membandingkan Linear Regression, Random Forest, Support Vector Machine, LSTM, dan ARIMA untuk melakukan prediksi arah pergerakan harga dengan menggunakan harga historis, indikator teknikal, dan ekonomi makro. Hasil menunjukkan model LSTM mendapatkan akurasi terbaik, directional accuracy terbaik, dan simulasi trading terbaik, menunjukkan jika model LSTM dapat menangkap dependensi sementara pasar dengan baik.	Penelitian ini berfokus pada evaluasi arsitektur model Informer dan perbedaan <i>sequence</i> yang digunakan pada model dalam melakukan prediksi Log return yang akan direkonstruksi menjadi harga Close.
8	Tizian Fischer, Marius Sterling, Stefan Lessmann (2024)	Membandingkan model Transformer dengan time embedding pada model LSTM dan ARIMA menggunakan data pasar valuta asing multivariat	Penelitian ini berfokus pada melakukan evaluasi model Informer dalam melakukan prediksi EUR/USD dengan panjang <i>sequence</i> yang berbeda untuk mengetahui dapat perbedaan <i>sequence</i> .

No	Peneliti	Hasil Penelitian	Perbedaan
9	Shahram Ghahremani, Uyen Trang Nguyen (2025)	Menggunakan model Attention-based LSTM for Forex (ALFA) untuk melakukan prediksi pada banyak pasar valuta asing. Hasil penelitian menunjukkan jika model mendapatkan nilai MAE terendah di setiap kombinasi fitur yang berbeda dengan tetap mempertahankan kompleksitas komputasi dan simulasi trading yang kuat.	Penelitian ini befokus pada penggunaan mekanisme Probsparse Self-Attention dan pengaruh perbedaan panjang <i>sequence</i> yang digunakan.
10	Lu Zhao, Wei Qi Yan (2023)	Menggunakan model informer dalam melakukan prediksi harga saham intraday menggunakan data dengan interval 1 menit dan menit pada 4 index saham yang berbeda. Setelah min-max normalization dan optimisasi hyperparameter, model dapat secara konsisten mendapatkan nilai MAE, RMSE, dan MAPE terendah dibandingkan model LSTM, BERT, dan Transfomer dengan secara efektif menangani peramalan dengan <i>sequence</i> panjang dan volatilitas yang seacara tiba-tiba dengan mekanisme ProbSparse Self-Attention.	Penelitian ini berfokus pada melakukan prediksi pada Log return yang akan direkonstruksi menjadi harga close dari mata uang EUR/USD dan pengaruh panjang <i>sequence</i> yang berbeda.

Perbedaan dengan Penelitian Sebelumnya

Berdasarkan referensi yang terkait, maka didapatkan perbedaan utama terhadap penelitian terdahulu pada penggunaan studi kasus yang lebih spesifik. Sebagian besar penelitian di atas menggunakan *input sequence* yang tidak lebih dari 48, penelitian ini menggunakan *input sequence* eksperimental dengan nilai 48, 96, 192, dan 384 agar model

lebih memahami konteks dari data historis yang lebih panjang untuk melakukan prediksi data pasar valuta asing yang bersifat volatil.

Perbedaan berikutnya terletak pada tujuan penelitian. Penelitian terdahulu umumnya berfokus pada perbandingan antar model dalam melakukan prediksi terhadap data pasar, tetapi penelitian ini melakukan uji parameter terhadap model Informer itu sendiri. Penelitian ini bertujuan untuk mengembangkan model informer dengan menguji beberapa parameter untuk mendukung model dalam melakukan prediksi pada data pasar. Namun penelitian ini tidak mencakup pada implementasi pada sistem/deployment secara teknis, melainkan menitikberatkan pada kinerja model.

Selain itu, penelitian ini melakukan prediksi pada nilai Log Return dari Closing Price pada data pasar. Pendekatan ini memberikan kemudahan model dalam melakukan prediksi pada data pasar yang bersifat non stasioner, karena dengan memilih Log Return dari Closing Price maka dapat meminimalisir sifat non stasionernya.

2.2 Pasar Valuta Asing (FOREX)

Pasar valuta asing (Forex) adalah pasar global yang terdesentralisasi dan beroperasi secara *Over-the-counter* (OTC) di mana di dalamnya terjadi pertukaran mata uang yang membuat terjadinya perubahan nilai tukar antar mata uang di dunia dan memfasilitasi transaksi internasional pada perdagangan barang, jasa, dan aset finansial (Chaboud et al., 2023). Tidak seperti pasar tersentralisasi, Forex beroperasi tanpa lokasi fisik dan sistem terpusat tunggal, pasar ini terdiri dari jaringan global yang saling terhubung yang melibatkan bank, dealer, platform perdagangan, dan broker.

Pasar forex sejauh ini menjadi pasar yang paling besar dan paling likuid di pasar keuangan global menurut Peshkova (2025) pada Bank for International Settlements (BIS) Triennial Central Bank Survey, nilai transaksi jual-beli mata uang per April 2025 mencapai rata-rata sebesar 9.5 miliar USD perhari, terjadi peningkatan sebesar 28% dengan nilai sebelumnya 7.5 miliar USD pada tahun 2022. Pasar ini juga beroperasi selama 24 jam pada 5 hari berturut-turut menjangkau berbagai pusat finansial di setiap zona waktu, yang menyebabkan pergerakan yang sangat dinamis dan volatil.

Pada data time-series di data finansial umumnya di representasikan dengan format Open-High-Low-Close-Volume (OHLCV). OHLCV Adalah Kesimpulan yang memiliki

bentuk candle stick yang berisi kesimpulan dari harga dan aktivitas jual-beli pada interval waktu tertentu (Tepelyan, 2025).. OHLCV dapat di definisikan sebagai:

1. Open Price (O): Harga pembukaan dalam interval waktu tertentu.
2. High Price (H): Harga tertinggi dalam interval waktu tertentu.
3. Low Price (L): Harga terendah dalam interval waktu tertentu.
4. Closing Price (C): Harga penutupan dalam interval waktu tertentu.
5. Volume (V): Volume transaksi dalam interval waktu tertentu.

2.2.1 Log Return

Pada analisis terhadap data time-series finansial, data Closing price mentah jarang digunakan sebagai *input* pada model karena bersifat non-stasioner dan mengikuti trend. Maka dari itu untuk mendapatkan data yang stasioner maka diterapkan *logarithmic return* (log return) pada closing price untuk dapat lebih sesuai dengan asumsi sebagian besar model (Malla et al., 2026; Stempień & Ślepaczuk, 2025; Sung et al., 2022).

Operasi Log Return dapat di representasikan sebagai:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1}) \dots\dots\dots(2. 1)$$

Berdasarkan **persamaan 2. 1**:

r_t = Satu periode log return dari Harga pada waktu t .

P_t = Harga pada interval waktu t .

$\ln(\cdot)$ = Logaritma natural.

2.2.2 Relative Strength Index (RSI)

Relative Strength Index (RSI) adalah salah satu indikator analisis teknikal yang berupa momentum oscillator yang diciptakan oleh J. Welles Wilder Jr. pada tahun 1978. RSI menghitung kecepatan dan besarnya perubahan harga sekarang untuk mengevaluasi apakah aset tersebut *Overbought* (Jenuh Beli) atau *Oversold* (Jenuh Jual). RSI cukup sering digunakan sebagai *input* fitur pada model Machine learning dan Deep learning karena dapat menangkap momentum dinamis jangka pendek yang melengkapi ketergantungan deret panjang saat melatih model dengan arsitektur Transformer (Saadati & Manthouri, n.d.; Stefaniuk & Ślepaczuk, 2025).

Operasi Relative Strength Index (RSI) dapat di representasikan sebagai berikut:

$$RSI_t = 100 - \frac{100}{1+RS_t} \dots\dots\dots(2. 2)$$

Di mana Relative Strength (**RS_t**) pada persamaan **2.2.2** dapat di representasikan sebagai:

$$RS_t = \frac{S \text{ Average Gain}_t}{S \text{ Average Loss}_t}$$

Berdasarkan **persamaan 2. 2**:

$S \text{ Average Gain}_t$ = Smoothed dari rata-rata keuntungan dalam periode waktu t

$S \text{ Average Loss}_t$ = Smoothed dari rata-rata kerugian dalam periode waktu t

Untuk mencari Smoothed rata-rata kerugian dan keuntungan dalam periode waktu t dengan cara:

$$S \text{ Average Gain}_t = \frac{(Average \text{ Gain}_{t-1} \times (n-1)) + Gain_t}{n} \dots\dots\dots(2. 3)$$

Dan:

$$S \text{ Average Loss}_t = \frac{(Average \text{ Loss}_{t-1} \times (n-1)) + Loss_t}{n} \dots\dots\dots(2. 4)$$

Yang di mana $Average \text{ Gain}$ dan $Average \text{ Loss}$ pada persamaan **2. 3** dan **2. 4** dapat dihitung dengan $Simple \text{ Average Gain}$ dan $Simple \text{ Average Loss}$ dengan cara:

$$Average \text{ Gain}_t = \frac{1}{t} \sum_{i=1}^t Gain_i, \text{ Average Loss}_t = \frac{1}{t} \sum_{i=1}^t Loss_i \dots\dots\dots(2. 5)$$

Berdasarkan **persamaan 2. 5** persamaan $Gain_i$ dan $Loss_i$ dapat dijabarkan sebagai:

$$Gain_t = \max(C_t - C_{t-1}, 0), \text{ Loss}_t = \max(C_{t-1} - C_t, 0) \dots\dots\dots(2. 6)$$

Berdasarkan persamaan **2. 6**:

$Gain_t$ = Keuntungan pada waktu t

$Loss_t$ = Kerugian pada waktu t

C_t = Harga penutupan pada waktu t

Karena RSI memiliki batasan yang jelas, bersifat stasioner, dan fokus pada momentum perubahan harga, membuatnya menjadi sangat baik saat melatih model dengan *input* yang panjang pada data OHLCV. Studi terbaru menunjukkan bahwa indikator seperti RSI meningkatkan performa dalam melakukan prediksi pada model dengan mengetahui jika pergerakan memiliki momentum yang kuat atau hanya fluktuasi kecil tanpa arah, sehingga akurasi meningkat(Mach et al., 2025; Saadati & Manthouri, n.d.; Stefaniuk & Ślepaczuk, 2025).

2.2.3 Average True Range (ATR)

Average True Range (ATR) adalah indikator volatilitas yang mengukur besarnya pergerakan harga dalam periode tertentu dan menghitung jarak antar sesi trading yang diciptakan oleh J. Welles Wilder Jr. pada tahun 1978. Dalam melakukan prediksi pada data timeseries forex ATR biasanya digunakan untuk membantu melatih model di ikuti dengan *input* fitur seperti OHLCV dan RSI. Karena ATR dapat memberi informasi terkait volatilitas pasar sehingga dapat mengenali pola volatilitas pada pasar (Mach et al., 2025; Saghafi et al., 2025; Stefaniuk & Ślepaczuk, 2025).

Operasi Average True Range (ATR) dapat di representasikan sebagai berikut:

$$ATR_t = \frac{(ATR_{t-1} \times (n-1)) + TR_t}{n} \dots\dots\dots (2. 7)$$

Berdasarkan pada **persamaan 2. 7** ATR_{t-1} didapatkan dengan **Simple Average** yang di representasikan sebagai:

$$ATR_t = \frac{1}{n} \sum_{i=1}^n TR_t \dots\dots\dots (2. 8)$$

Terlihat pada **persamaan 2. 7** dan **2. 8** True Range (TR) didapatkan dengan:

$$TR_t = \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|) \dots\dots\dots (2. 9)$$

Di mana pada persamaan 2. 9:

TR_t = True Range pada periode waktu t

H_t = Harga tertinggi pada waktu t

L_t = Harga terendah pada waktu t

C_t = Harga penutupan pada waktu t

Karena Average True Range (ATR) bersifat positif, stasioner, dan memiliki perubahan pelan yang di ukur secara langsung dengan OHLC, membuatnya cocok saat melakukan prediksi pasar dengan *input* yang panjang. Studi terbaru juga menunjukkan bahwa mengikutsertakan ATR pada *input* fitur dapat meningkatkan performa model dengan membantu model beradaptasi akan perubahan volatilitas (Mach et al., 2025; Saghafi et al., 2025).

2.2 Machine Learning dan Deep Learning

2.3.1 Machine Learning

Machine Learning adalah suatu konsep di mana sebuah program komputer dapat mempelajari dan beradaptasi dengan data yang baru tanpa intervensi manusia, berfokus pada melatih komputer belajar dari data dan berkembang dengan pengalaman pelatihan tanpa secara eksplisit di program untuk melakukannya (Sadiku et al., 2025). Tidak seperti program tradisional di mana dibutuhkan data dan program untuk mendapatkan hasil, pada Machine learning dibutuhkan data dan hasil untuk menghasilkan program.

Menurut Mienye & Swart (2024) Machine Learning juga terbagi menjadi 3 jenis utama berdasarkan bagaimana model akan belajar dari data:

1. Supervised Learning

Supervised learning adalah tipe yang cukup umum digunakan pada machine learning. Dengan pendekatan ini, model dilatih pada kumpulan data yang sudah memiliki jawaban (label) yang berisikan *input* fitur dan nilai hasil yang benar (Target). Algoritma ini mempelajari terkait pemetaan di antara *input* dan output yang akurat pada data yang belum pernah dilihat atau dipelajari sebelumnya.

2. Unsupervised Learning

Unsupervised learning belajar pada kumpulan data yang belum memiliki jawaban (label), yang berarti kumpulan data hanya berisikan *input* fitur tanpa nilai hasil yang benar (Target). Model akan belajar kumpulan data untuk mencari pola tersembunyi, struktur, atau pengelompokan data pada kumpulan data itu sendiri.

3. Reinforcement Learning

Reinforcement learning adalah ketika model belajar dengan cara membuat keputusan secara beruntutan di mana model akan mencoba-coba (Trial and Error) dan akan mendapatkan reward (Hadiah) atau penalty (Hukuman) dan akan belajar secara bertahap untuk mendapatkan optimisasi agar memaksimalkan akumulasi reward waktu ke waktu.

2.3.2 Deep Learning

Deep learning adalah spesialisasi pada cabang bidang dari Machine learning yang menggunakan jaringan syaraf tiruan (*Artificial Neural Network*) yang terdiri dari banyak bagian lapisan tersembunyi (*Hidden Layers*) agar secara otomatis mempelajari representasi hierarkis yang kompleks secara langsung dari data mentah. Ketika algoritma Machine learning tradisional biasanya membutuhkan seorang ahli manusia untuk mendesain dan mengekstraksi data mentah secara manual, model deep learning dapat menemukan pola rumit dan abstraksi tingkat tinggi melalui lapisan proses yang berurutan, meniru cara kerja otak manusia dalam memproses informasi (Mienye & Swart, 2024).

Pada Deep Neural Network, setiap lapisan mengubah data yang masuk menjadi representasi yang lebih abstrak. Pada lapisan pertama secara khusus menangkap fitur tingkat rendah dan akan di kombinasikan oleh lapisan yang lebih dalam untuk mengenali konsep dengan tingkat yang lebih tinggi. Dengan arsitektur ini membuat model deep learning dapat menangani kompleksitas dengan volume yang tinggi, tidak terstruktur, atau data dengan dimensi yang tinggi dibandingkan dengan pendekatan machine learning konvensional (Alzubaidi et al., 2021; Tian et al., 2025).

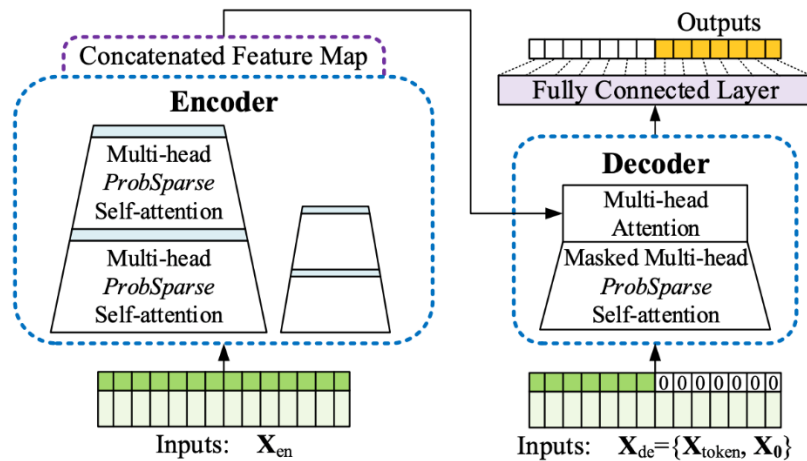
Perkembangan deep learning terjadi cepat karena 3 faktor kunci yaitu: 1). Terjadinya lonjakan perkembangan pada ketersediaan kumpulan data yang lebih besar. 2). Meningkatnya kemampuan komputasi secara signifikan khususnya pada perkembangan graphic processing units (GPU). 3). Berkembangnya efisiensi pelatihan algoritma dan teknik optimisasi. Pada akhirnya membuat deep learning mencapai performa yang luar biasa pada berbagai bidang, dan sekarang dikenali sebagai salah satu teknologi inti dari kecerdasan buatan modern.

2.4 Informer

Informer adalah model berbasis arsitektur Transformer yang dimodifikasi dan di desain untuk menangani prediksi deret panjang pada data timeseries (LSTF). Menurut Zhou et al. (2021) model Vanilla Transformer memiliki limitasi yang signifikan saat menangani masalah LSTF yaitu:

1. **Komputasi kuadrat pada mekanisme Self-Attention:** Dengan mekanisme self-attention yang menggunakan operasi *dot-product* antar semua pasangan token dalam sebuah *sequence*. Sehingga menghasilkan kompleksitas dan penggunaan memori per lapisannya (layer) menjadi $O(L^2)$.
2. **Memory bottleneck pada lapisan bertumpuk (Stacked layers) :** Jika sebuah model memiliki stacked layer *encoder/decoder* sebanyak J dengan setiap layer memiliki kompleksitas $O(L^2)$, maka membuat total penggunaan memorinya menjadi $O(J \times L^2)$ yang akan membatasi skalabilitas saat menerima *input* dengan *sequence* panjang.
3. **Lambat saat melakukan prediksi dengan output panjang:** Proses decoding pada Transformer standar yang bersifat bertahap satu per satu (Autoregressive) membuatnya sebanding dengan model berbasis RNN pada kecepatan prediksinya yang secara alami bersifat sekuensial.

Maka dari itu model Informer seperti pada **gambar 2.1** di modifikasi untuk menangani kelemahan model standar Transformer dengan mengubah mekanisme Self-attention menjadi ProbSparse Self-Attention sehingga kompleksitasnya menjadi $O(L \log L)$. Lalu dengan mekanisme Distilling layer pada *encoder* dan *generative style decoder*, mengecilkan kompleksitasnya sembari mempertahankan atau meningkatkan akurasi prediksi pada data yang volatil (Zhou et al., 2021).



Gambar 2. 1 Arsitektur Informer

2.4.1 Probsparse Self-Attention Layer

Pada model Transformer standar, setiap vector *query* (q_i) dihitung dengan *dot-product* pada setiap vector *key* (k_j) menghasilkan $L_k \times L_1$ matrix Attention dengan kompleksitas $O(L^2)$. Observasi empiris pada data timeseries menunjukkan skor Attention diikuti dengan Distribusi Ekor Panjang (Long-tail Distribution): Hanya sebagian kecil query aktif yang berkontribusi secara signifikan, sedangkan mayoritas query bersifat malas (Lazy).

Model Informer memperkenalkan sparsity measure untuk setiap query q_i yang dapat di representasikan dengan:

$$M(q_i, K) = \max_j \left\{ \frac{q_i k_j^T}{\sqrt{d}} \right\} - \frac{1}{L_{Ki}} \sum_{j=1}^{L_K} \frac{q_i k_j^T}{\sqrt{d}} \dots \dots \dots (2. 10)$$

Di mana d pada **persamaan 2. 10** adalah dimensi dari query atau key. Ini mendekati informasi yang didapatkan dari skor Attention yang dominan. Hanya top- u query yang dipertahankan, di mana:

$$u = c \cdot \ln L_Q \dots \dots \dots (2. 11)$$

Berdasarkan **persamaan 2. 11** u adalah sampling factor, sparse query pada metrik $\bar{Q}$ berisikan hanya baris u dan akan dihitung dengan:

$$A(\bar{Q}, K, V) = \text{Softmax} \left(\frac{\bar{Q} K^T}{\sqrt{d}} \right) V \dots \dots \dots (2. 12)$$

Berdasarkan **persamaan 2. 12**:

$\bar{Q}$ = Metrik query yang hanya berisi query paling aktif sebanyak u setelah perhitungan sparsity.

K^T = Metrik key yang di transpose.

$\sqrt{d}$ = Dimensi vektor query/key.

V = Metrik value.

$\text{Softmax}(\cdot)$ = Softmax function pada **persamaan 2. 13**.

Operasi softmax digunakan untuk memastikan bahwa semua *weight* pada attention tidak bernilai negatif dan totalnya menjadi 1, yang dapat direpresentasikan sebagai:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^m e^{z_j}} \dots \dots \dots (2. 13)$$

Di mana pada persamaan **2. 13**:

z_i = Skor Attention pada iterasi ke i .

e^{z_i} = Nilai eksponensial dari z_i .

m = Panjang dari *input* key.

Dengan ProbSparse Self-Attention pada **persamaan 2. 12** mengurangi kompleksitasnya menjadi $O(L \log L)$ sembari memberikan ketergantungan kritis jangka panjang paling banyak. Dengan pengurangan kompleksitas melalui sparse attention ini tidak membuat model lebih efisien, tetapi juga lebih sesuai dengan karakteristik data Forex yang tidak merata dan di dominasi kejadian-kejadian penting tertentu (Mach et al., 2025; Zhou et al., 2021).

2.4.2 Distilling Layer

Untuk semakin mengurangi kompleksitas dengan *input* yang sangat panjang tanpa kehilangan informasi. Oleh karena itu digunakan distilling layer yang secara progresif mengurangi panjang *sequence* di setiap *layer encoder*.

Operasi pada Distilling layer dapat di representasikan sebagai:

$$X_t^{j+1} = \text{MaxPoll} \left(\text{ELU} \left(\text{Conv1d} \left([X_t^j]_{AB} \right) \right) \right) \dots\dots\dots (2. 14)$$

Berdasarkan **persamaan 2. 14**:

X_t^{j+1} = Output dari tensor pada operasi Distillation di layer $j+1$ dan timestep t .

X_t^j = *Input* tensor yang berasal dari blok Attention pada layer j dan timestep t sebelumnya.

$[X_t^j]_{AB}$ = Output dari blok Attention (AB) pada layer j .

Conv1d = Operasi convolution 1 dimensi pada **persamaan 2.17**.

ELU = Operasi Exponential Linear Unit Activation function pada **persamaan 2. 16**.

MaxPoll = Operasi Maximum Pooling pada persamaan **2. 15**.

Berdasarkan **persamaan 2. 14**, $[X_t^j]_{AB}$ adalah output dari blok attention j , *Conv1d* menggunakan kernel dengan size 3, dan *MaxPoll* memiliki stride 2. Activation dengan *ELU* digunakan karena dapat memberikan nilai negatif yang mungkin membawa sinyal

volatilitas yang penting. Dengan ini, membuat struktur seperti piramida pada *encoder* yang akan mengurangi total dari penggunaan memori. Pada pengaplikasian data Forex, Distilling layer membantu menyaring *noise* dari data *tick* berfrekuensi tinggi sembari berfokus pada tren yang berkelanjutan di seluruh sesi *trading* (Stefaniuk & Ślepaczuk, 2025; Zhou et al., 2021).

1. Maxpoll

Maxpooling secara agresif melakukan *downsample* pada *sequence* dengan tetap menjaga fitur terkuat dari tahap sebelumnya. Dibandingkan dengan *Average Pooling*, *Max Pooling* berfokus pada menemukan sinyal puncak, di mana dalam Forex sesuai dengan peristiwa pada pasar yang signifikan. Hal ini memastikan ketergantungan jarak jauh yang paling informatif tetap terjaga setelah kompresi (Stefaniuk & Ślepaczuk, 2025; Zhou et al., 2021).

$$MaxPoll(x)_i = \max_{k \in [0, p-1]} x_{i \cdot s + k} \dots\dots\dots (2. 15)$$

Berdasarkan **persamaan 2. 15**:

$MaxPoll(x)_i$ = Nilai hasil dari pooling pada posisi i .

x = *Input sequence* setelah ELU activation.

i = Index posisi output saat ini.

k = Index yang berjalan di dalam Pooling window.

p = Ukuran Pooling window.

s = *Stride*

$x_{i \cdot s + k}$ = Nilai *input* sebenarnya pada posisi yang sedang dipertimbangkan di dalam *window*

2. ELU (Exponential Linear Unit) Activation

ELU dapat memberi nilai negatif agar bisa lewat tidak seperti ReLU, di mana tidak dapat memberikan nilai negatif. Ini dapat meningkatkan *Gradient Flow* saat *back-propagation* dan menghindari masalah neuron mati (Dying Neuron) pada tumpukan (stack) yang lebih dalam. Elu dapat memberikan distribusi penuh pada sinyal volatilitas sebelum downsampling (Mach et al., 2025).

Di mana ELU dapat di representasikan sebagai:

$$ELU(x) = \begin{cases} x & \text{jika } x > 0, \\ \alpha(e^x - 1) & \text{jika } x \leq 0 \end{cases} \dots\dots\dots(2. 16)$$

Berdasarkan **persamaan 2. 16**:

$ELU(x)$ = Hasil dari *activation function* untuk nilai *input* x .

x = Nilai *input* dari layer Conv1d

α = Hyperparameter yang mengontrol slope pada bagian negatif.

e^x = fungsi eksponensial yang diterapkan pada x (berbasis logaritma natural ≈ 2.718).

1 = Konstanta yang dikurangi pada regional negatif.

3. Conv1d (One-Dimensional Convolution)

Berbeda dengan model Transformer Standar yang tidak memiliki ekstraksi fitur lokal yang eksplisit sebelum Attention, Conv1d berperan sebagai ekstraktor fitur lokal yang ringan. Dalam data Forex, ini dapat membantu mengidentifikasi momentum harga jangka pendek dan pola volatilitas intra-bar yang sangat krusial sebelum *sequence* di kompresi .

Di mana Conv1d dapat di representasikan sebagai:

$$(Conv1d(X))_i = \sum_{k=1}^K w_k \cdot x_{i+k-1} + b \dots\dots\dots(2. 17)$$

Di mana pada persamaan **2. 17**:

$(Conv1d(X))_i$ = Nilai output covolution pada posisi i di *sequence*.

X = *Input* tensor dengan bentuk $L \times d_{model}$.

i = Indeks posisi output saat ini.

k = Indeks dari berat (weight) kernel yang berjalan dari 1 hingga ukuran kernel K .

K = Ukuran kernel.

w_k = Bobot (weight) yang dapat dipelajari dari posisi ke- k pada kernel Convolution.

x_{i+k-1} = Nilai *input* pada posisi $i+k-1$.

b = Bias term, konstanta yang dapat dilatih dan ditambahkan ke setiap output.

Dengan opesai di atas yang dimulai dari Conv1d ke ELU dan diakhiri dengan Maxpool mengkompresi *sequence* sembari tetap mempertahankan fitur yang informatif. Pada prediksi deret panjang Forex, operasi ini memberikan model kemampuan untuk memproses ribuan data OHLCV atau log-returns sebelumnya secara efisien tanpa

kehilangan ketergantungan jangka panjang yang kritis seperti pengelompokan volatilitas pada setiap sesi trading (Zhou et al., 2021).

2.4.3 Feed Forward Network (FFN)

Feed Forward Network di aplikasikan pada setiap blok Probsparse Self-Attention dan setelah layer Distilling pada *stack encoder* dan *stack decoder*. Saat mekanisme Attention menangkap dependensi di antara setiap timestep, FFN melakukan transformasi non linear secara independen pada setiap timestep. Sub-layer ini meningkatkan model dalam memahami pola data lebih kuat, khususnya pada pola yang non linear (Zhou et al., 2021).

FFN pada Informer mirip dengan yang digunakan pada model Transformer standar, lalu dikombinasikan dengan residual connection dan layer normalization. FFN dapat di representasikan dengan:

$$FFN(x) = x + LayerNorm(W_2 \cdot GELU(W_1 x + b_1) + b_2) \dots\dots\dots(2. 17)$$

Berdasarkan **persamaan 2. 17**:

$FFN(x)$ = Output akhir dari Feed Forward Network untuk *input x*.

x = *Input* fitur dari vector/tensor yang berasal dari sub layer sebelumnya.

$+$ = Residual connection.

$LayerNorm(\cdot)$ = Operasi Layer Normalization.

W_1 = Weight dari metriks transformasi linear pertama.

W_2 = Weight dari metriks transformasi linear kedua.

b_1 = Bias vector dari layer linear pertama.

b_2 = Bias vector dari layer linear kedua.

$GELU(\cdot)$ = Gaussian Error Linear Unit activation function.

Di mana persamaan GELU pada persamaan **2. 17** dapat di representasikan sebagai:

$$GELU(x) = x \cdot \phi(x) \dots\dots\dots(2. 18)$$

Di mana $\phi(x)$ pada persamaan **2. 18** adalah fungsi distribusi kumulatif dari distribusi normal standar $N(0, 1)$, dalam praktiknya sering diperkirakan sebagai:

$$GELU(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} + (x + 0.044715x^3) \right) \right) \dots\dots\dots(2. 19)$$

Berdasarkan **persamaan 2. 19**:

x = Nilai *input* yang berasal dari layer linear pertama.

$\tanh(\cdot)$ = Hyperbolic tangent function.

Dengan FFN, model dapat menangkap intraksi non linear seperti relasi di antara data OHLCV, atau pola volatilitas pada log return yang tidak bisa jika hanya menggunakan Attention saja. Pada long *sequence* forecasting data forex, sub layer ini diperlukan untuk mengubah representasi temporal yang dihasilkan oleh Attention menjadi pemetaan yang lebih kompleks. Sehingga model dapat memahami perubahan market, menangkap dinamika pada volatilitas, sembari meningkatkan akurasi prediksi pada kondisi pasar yang fluktuatif (Mach et al., 2025; Stefaniuk & Ślepaczuk, 2025).

2.4.4 Masked ProbSparse Self-Attention Layer

Pada stack *Decoder* di model informer, kausalitas harus dipertahankan agar model tidak dapat melihat *timestep* masa depan saat pelatihan maupun prediksi. Masked ProbSparse Self-Attention memiliki mekanisme sama seperti ProbSparse pada *encoder*, tetapi ditambahkan *causal mask* yang membatasi setiap *timestep* agar hanya dapat mengakses informasi dari masa lalu dan saat ini saja. Dengan ini maka model tetap bersifat autoregresif sekaligus menjaga efisiensi komputasi pada $O(L \log L)$ sehingga model tetap skalabilitas pada *sequence* panjang (Zhou et al., 2021).

Di mana Masked ProbSparse Self-Attention Layer dapat di representasikan sebagai:

$$A(\bar{Q}, K, V) = \text{Softmax} \left(\text{Mask} \left(\frac{\bar{Q}K^T}{\sqrt{d}} \right) \right) V \dots\dots\dots (2. 20)$$

Di mana Masked ProbSparse Self-Attention memiliki operasi yang sama seperti ProbSparse Self-Attention standar, tetapi mengganti seluruh posisi masa depan dari skor metriks Attention dengan $-\infty$. Setelah dilakukan operasi Softmax, posisi ini nilainya menjadi 0, memastikan model hanya menggunakan informasi dari *timestep* sebelumnya.

Masked ProbSparse Self-Attention mengizinkan *decoder* untuk menghasilkan seluruh *sequence* masa depan dalam sekali *forward pass* serta menjaga kasualitas. Dengan memastikan setiap *timestep* hanya mengakses informasi masa lalu dan tidak terjadi kebocoran data masa depan saat pelatihan model. Mekanisme ini dapat menangkap

dependensi jangka panjang seperti pengelompokan volatilitas antar sesi *trading* sekaligus efisien dan realistis pada skenario prediksi di dunia nyata (Niu, 2024).

2.4.5 Cross-Attention Layer

Cross-Attention Layer Adalah bagian penting dari model Informer. Saat Masked ProbSparse Self-Attention hanya melihat *input* sebelumnya, Cross-Attention layer dapat membuat *decoder* melihat balik pada data yang telah diekstraksi pada *encoder*. *Decoder* akan membuat query yang akan dibandingkan dengan key pada *encoder* untuk menentukan *sequence* historis paling relevan. Mekanisme ini membantu *decoder* mengkoneksi prediksinya sendiri dengan dependensi jangka panjang yang telah dipelajari *encoder*.

CrossAttention dikomputasikan menggunakan dot product antara query pada *decoder* dan key dan value *encoder*, yang dapat di representasikan sebagai:

$$CrossAttention(Q_{de}, K_{en}, V_{en}) = Softmax \left(Mask \left(\frac{Q_{de} K_{en}^T}{\sqrt{d}} \right) \right) V_{en} \dots\dots\dots (2. 21)$$

Berdasarkan **persamaan 2. 21**:

$CrossAttention(Q_{de}, K_{en}, V_{en})$ = Output akhir dari layer Cross-Attention.

Q_{de} = Metriks query dari *Decoder*.

K_{en} = Metriks key dari output akhir *Encoder*.

V_{en} = Metriks value dari output akhir *Encoder*.

$Softmax(\cdot)$ = Fungsi Softmax untuk mengubah setiap baris metriks menjadi distribusi probabilitas.

$\frac{Q_{de} K_{en}^T}{\sqrt{d}}$ = Scaled dot-product pada skor Attention.

T = Operator transpose pada metriksk key.

d = Dimensi pada setiap vektor query atau key.

Dengan Masked Self-Attention, Cross-Attention memperkuat model menggabungkan prediksi autoregresifnya sendiri dengan dengan data masa lalu dari *encoder*. Cross-Attention membuat model efektif dalam menggunakan konteks historis saat melakukan predikisi masa depan. Hal ini membuat model informer baik dalam menangkap pola

kelanjutan tren atau pengelompokan volatilitas yang mengarah pada prediksi jangka yang lebih akurat (Niu, 2024).

2.4.6 Linear Projection Layer

Linear Projection Layer adalah komponen akhir pada *decoder* Informer. Setelah *decoder* memproses informasi melewati banyak layer masked self-attention dan cross-attention, Layer ini akan mengkonversi representasi tersembunyi dari *decoder* menjadi nilai aktual prediksi (Mach et al., 2025). Linear Projection layer berperan untuk mengubah fitur berdimensi tinggi yang dipelajari model menjadi format output yang diinginkan. Karena Informer menggunakan generative-style *decoder*, Linear Projection layer menghasilkan seluruh *sequence* masa depan dalam sekali forward pass dibandingkan dengan 1 step per waktunya (Zhou et al., 2021).

Di mana Linear Projection Layer dapat direpresentasikan sebagai:

$$\hat{Y} = H_{dec}W_{proj} + b_{proj} \dots\dots\dots(2. 22)$$

Berdasarkan **persamaan 2. 22**:

$\hat{Y}$ = Prediksi final output *sequence*.

H_{dec} = States/output tersembunyi yang datang dari layer terakhir pada *decoder*.

W_{proj} = Weight dari metriks yang dapat dipelajari linear projection.

b_{proj} = Bias dari vector yang dapat dipelajari yang dapat ditambahkan pada linear projection.

Layer ini memberikan model Informer untuk menghasilkan seluruh *sequence* masa depan dalam sekali *forward pass*, di mana akan sangat berguna saat pengaplikasian prediksi data Forex.

2.5 Pelatihan Model

Pada prediksi data timeseries, model belajar dengan proses yang sistematis karena itu membutuhkan partisi data untuk memastikan model belajar pola yang bermakna sembari menghindari *overfitting* dan memberikan evaluasi yang tepat. Data biasanya dibagi menjadi 3 yaitu, training set, validation set, dan test set. Pembagian ini diperlukan karena data yang memiliki bentuk timeseries memperlihatkan dependensi sekuensial, dan

random shuffling akan merusak urutan alami dari peristiwa di dalam data (Liu et al., 2024).

1. Training Set

Training set digunakan oleh model untuk mempelajari *input* fitur dengan optimisasi bobot. Training set memiliki proporsi data paling besar, pada penelitian ini memiliki 70% dari keseluruhan data. Data ini digunakan secara khusus untuk memperbarui parameter internal model yaitu *weight* dan *bias*. Pada pelatihan model akan memproses data berulang secara maju dan mundur, menggunakan algoritma optimisasi untuk meminimalkan *loss function* yang dipilih. Proporsi yang besar ini bertujuan untuk mempelajari tren pasar, pola pergerakan pasar dalam periode tertentu, dependensi jangka panjang, dan sifat non stasioner pada data historis (Cui et al., 2023).

2. Validation Set

Validation Set atau dikenal sebagai tuning set digunakan untuk memantau performa model saat pelatihan dan optimisasi *hyperparameter* seperti *learning rate*, *batch size*, jumlah *layer*, atau *dropout rate* (Hassani et al., 2025). Pada penelitian ini menggunakan 15% dari total data.

3. Test Set

Test Set adalah komponen terakhir yang digunakan untuk melihat data yang belum terlihat setelah pelatihan dan *hyperparameter tuning* selesai. Ini bertujuan untuk mensimulasikan kondisi dunia nyata dan memberikan kemampuan prediksi model yang tidak bias (Hall & Rasheed, 2025). Pada penelitian ini menggunakan 15% dari total data.

2.5.1 Adam (Adaptive Moment Estimation)

Adam optimizer digunakan sebagai algoritma optimisasi saat pelatihan pada *training set*. Adam menggabungkan kelebihan dari 2 metode populer yang lain yaitu Momentum dan RMSProp, dan mengadaptasi *learning rate* pada setiap parameter secara individu berdasarkan momen pertama dan kedua dari *gradient*. Dengan ini membuat Adam efektif saat melatih model *deep learning* pada data dengan banyak *noise*, non

stasioner, dan mampu mencapai konvergensi yang lebih cepat sembari menjaga kestabilan pelatihan (Cui et al., 2023).

Algoritma Adam dilakukan dengan 5 langkah pada setiap iterasi pelatihan t :

Pertama, model menghitung *gradient* dari *loss function* terhadap parameter saat ini menggunakan sebagian kecil data pada *training set*. Ini dapat direpresentasikan sebagai:

$$g_t = \nabla_{\theta} J(\theta_{t-1}) \dots \dots \dots (2.23)$$

Berdasarkan **persamaan 2.23**:

g_t = Gradient Vector pada timestep t .

$\nabla_{\theta} J(\cdot)$ = Operator *gradient* terhadap parameter model θ .

$J(\cdot)$ = Loss function yang digunakan.

θ_{t-1} = Parameter model dari timestep sebelumnya.

Setelah itu mencari estimasi momen pertama yaitu moving average dari gradien, yang dapat direpresentasikan sebagai:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \dots \dots \dots (2.24)$$

Berdasarkan **persamaan 2.24**:

m_t = Estimasi momen pertama dari *gradient* pada iterasi ke t .

β_1 = Koefisien decay untuk momen pertama (biasanya bernilai 0.9).

m_{t-1} = Estimasi momen pertama pada iterasi sebelumnya.

g_t = Gradien pada iterasi ke t .

Setelah itu mencari estimasi momen kedua (Variance) yaitu moving average dari gradien yang kuadratkan, dapat direpresentasikan dengan:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \dots \dots \dots (2.25)$$

Berdasarkan **persamaan 2.25** :

v_t = Estimasi momen kedua dari *gradient* pada iterasi ke t .

v_{t-1} = Estimasi momen kedua pada iterasi sebelumnya.

β_2 = Koefisien decay untuk momen kedua (biasanya bernilai 0.999).

g_t^2 = Gradien pada iterasi ke t yang di kuadratkan (element-wise).

karena nilai rata-rata bergerak (moving averages) diawali dari nol, nilainya cenderung bias ke nol terutama di iterasi awal, sehingga digunakan *bias correction* untuk mengoreksi dan membuat estimasinya lebih akurat, yang dapat direpresentasikan dengan:

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1-\beta_2^t} \dots\dots\dots(2. 26)$$

Berdasarkan **persamaan 2. 26**:

$\widehat{m}_t$ = Estimasi rata-rata gradien yang sudah dikoreksi

$\widehat{v}_t$ = Estimasi variance gradien yang sudah dikoreksi.

$\beta_1^t, \beta_2^t = \beta_1$ dan β_2 yang dipangkatkan sebanyak iterasi ke t .

Dan diakhir dengan memperbarui parameter model dengan nilai yang sudah diperbarui, yang dapat di representasikan dengan:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\widehat{v}_t} + \epsilon} \widehat{m}_t \dots\dots\dots(2. 27)$$

Berdasarkan **persamaan 2. 7**:

θ_t = Parameter model yang telah diperbarui pada timestep ke t .

θ_{t-1} = Parameter model sebelumnya.

η = Learning rate.

$\sqrt{\widehat{v}_t}$ = Variance yang telah dikoreksi yang diakar kuadratkan.

ϵ = Konstanta kecil untuk stabilitas numerik (biasanya 10^{-8})

$\widehat{m}_t$ = Momen pertama yang telah dikoreksi.

Dengan *Adaptive Scaling* pada *learning rate* untuk setiap parameter berdasarkan Momentum dan Variance pada gradien, Adam dapat dengan cepat mencapai konvergensi walaupun dengan data yang *noisy* dan non stasioner. Dengan ini membutuhkan lebih sedikit *tuning* manual pada *learning rate* dibandingkan *Gradient descent* standar, membuatnya pilihan utama pada kebanyakan model untuk melakukan prediksi pada data timeseries (Hall & Rasheed, 2025).

2.6 Evaluasi Model

Setelah proses pelatihan model, performa pada model prediksi data timeseries harus diukur dengan menggunakan metrik error yang terstandarisasi. Metrik ini bertujuan untuk mengukur perbedaan antara nilai prediksi model dan nilai observasi yang aktual,

dengan itu dapat membandingkan model secara objektif dan untuk memahami kekuatan praktikalnya serta limitasinya. Pada literatur prediksi dengan data timeseries, 3 metrik yang sering digunakan adalah Mean Absolute Error (MAE), Mean Squared Error (MSE), dan Mean Absolute Percentage Error (MAPE). Ketiga metrik ini digunakan bersamaan agar dapat memberikan perspektif yang berbeda terhadap akurasi, kestabilan model, dan interpretabilitas .

2.6.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) menghitung rata-rata kesalahan antar nilai prediksi dan nilai sebenarnya tanpa memperhatikan positif atau negatif. Metrik ini cukup mudah dipahami karena dapat memperlakukan semua error individu secara adil dan relatif tidak sensitif pada *outliers* dibanding metrik dengan *squared error*. MAE sangat berguna karena hasil error-nya tetap dalam satuan yang sama dengan data asli, sehingga mudah dipahami dan diinterpretasikan .

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \dots \dots \dots (2. 28)$$

Berdasarkan **persamaan 2. 28**:

n = Nilai total dari observasi pada Evaluation Set.

y_i = Nilai observasi aktual pada time-step t .

$\hat{y}_i$ = Nilai prediksi pada time-step t .

MAE mudah dipahami dan tidak terlalu dipengaruhi oleh error yang ekstrim. Karena MAE tidak mengkuadratkan error, semua error diberi bobot yang sama, sehingga dapat menjadi kelemahan ketika didapatkan error yang sangat besar .

2.6.2 Mean Squared Error (MSE)

Mean Squared Error (MSE) menghitung rata-rata kesalahan nilai prediksi dan nilai sebenarnya lalu dikuadratkan. Karena error dikuadratkan, kesalahan yang besar akan diberi penalti lebih besar dibandingkan kesalahan yang kecil. Karena itu membuat MSE sangat sensitif terhadap nilai *outliers*, tetapi juga sangat berguna saat harus menghindari kesalahan yang besar atau ketika metrik ini digunakan sebagai *loss function* saat pelatihan model.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \dots \dots \dots (2. 29)$$

MSE mudah digunakan sebagai optimisasi yang berbasis gradien karena bersifat diferensibel, tetapi karena error dikuadratkan, hasilnya akan lebih sulit untuk diinterpretasikan. MSE juga lebih sensitif terhadap *outliers* dibandingkan dengan MAE .

2.6.3 Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error (MAPE) mengukur error dalam bentuk persentase terhadap nilai sebenarnya, sehingga tidak bergantung kepada skala data. Karena itu MAPE cocok untuk membandingkan performa antar dataset dengan satuan atau magnitudo yang berbeda, serta mudah dipahami karena memiliki hasil berbentuk persen.

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \dots\dots\dots(2. 30)$$

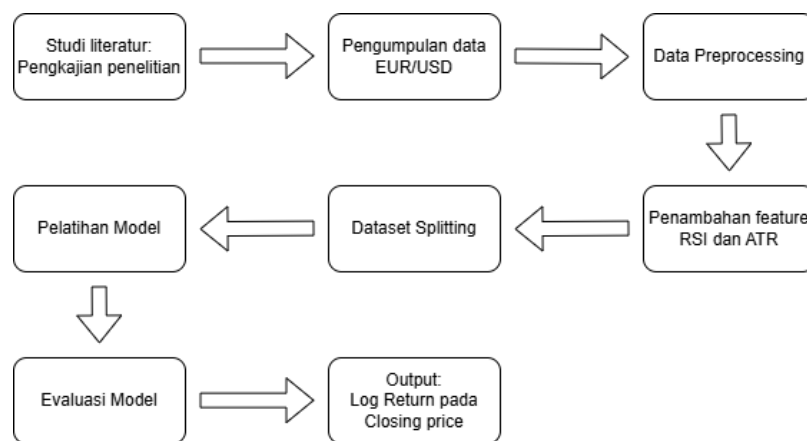
MAPE mudah dipahami dan cocok Ketika membandingkan model dengan skala yang berbeda, tetapi kelemahannya adalah bisa bermasalah saat nilai aktual mendekati 0, serta cenderung memberi penalti lebih besar pada *under-prediction* dibandingkan dengan *over-prediction*.

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Pelaksanaan Penelitian

Penelitian ini dilakukan melalui beberapa tahap seperti yang ditunjukkan pada **gambar 3.1**. Tahapan ini telah disusun untuk memastikan bahwa proses pengumpulan data, pelatihan model, hingga evaluasi dapat berjalan secara sistematis selaras dengan tujuan penelitian.



Gambar 3. 1 Tahapan Pelaksanaan Penelitian

1. Studi Literatur

Studi literatur dilakukan untuk memperoleh landasan teori dan juga sebagai acuan dalam menentukan metode penelitian. Kajian literatur ini mencakup konsep Informer, arsitektur Informer, pengolahan dataset timeseries, dan juga pemilihan metrik evaluasi. Hasil pada studi literatur, digunakan sebagai dasar dalam perancangan eksperimen, penetapan batasan penelitian, serta strategi evaluasi model.

2. Pengumpulan Data

Tahap pengumpulan data dilakukan dengan mengunduh data pasar EUR/USD di platform forexsb.com, dengan periode waktu 6 April 2010 hingga 17 April 2026. Data diambil dengan interval waktu 1 jam, lalu zona waktu Singapura, dan tidak menyertakan data pada Sabtu dan Minggu.

3. Data Preprocessing

Pada tahap preprocessing data akan diberi penamaan kolom agar jelas dan konsisten. Lalu data akan diperiksa apakah memiliki *missing value*, duplikasi data, dan anomali. Lalu mengubah tipe data agar sesuai dengan format agar memastikan data siap dalam melakukan proses selanjutnya.

4. Penambahan fitur RSI, ATR, dan Log return

Penambahan fitur untuk memperkaya data dengan menambahkan indikator teknikal seperti Relative Strength Index (RSI) dan Average True Range (ATR). Di mana penambahan fitur ini akan menambah konteks pada data yang akan meningkatkan performa model. Penambahan fitur RSI dan ATR menggunakan *window* sebanyak 14, di mana akan menggunakan data dari 14 baris sebelumnya untuk mengetahui nilai RSI dan ATR nya.

Lalu akan ditambahkan log return, di mana fitur tersebut akan berfungsi sebagai fitur *input* dan juga target prediksi dari model agar lebih stasioner. Nilai log return juga akan digunakan untuk melakukan rekonstruksi menjadi harga close agar lebih mudah dimengerti.

5. Dataset Splitting

Dataset yang telah bersih dan sudah memiliki target yaitu Log return dari Closing price akan dibagi menjadi data latih, data validasi, data uji, dan data sampel cadangan untuk melihat bagaimana model melakukan prediksi pada data diluar pelatihan. Pembagian ini memiliki tujuan yaitu untuk menghindari kebocoran data (data leakage) dan memastikan evaluasi dilakukan pada data yang tidak digunakan pada saat pelatihan.

Data sebelumnya akan diambil sebanyak 491 baris terakhir sebagai data sampel cadangan. Di mana setiap skenarionya akan melakukan 100 poin prediksi. Pada kolom Close, data akan diduplikasi pada training. Penduplikasian ini dilakukan karena kolom close yang digunakan untuk pelatihan akan dinormalisasi sehingga skala nilainya akan diperkecil. Namun model tetap membutuhkan nilai aktual dari Close yang berguna untuk rekonstruksi log return kembali menjadi harga close.

6. Pelatihan model

Tahap pelatihan model dilakukan dengan menggunakan arsitektur Informer. Proses pelatihan dilakukan dengan menggunakan framework PyTorch dengan

memanfaatkan data pelatihan dan data validasi. Di tahap ini model akan dilatih untuk mempelajari pola pasar berdasarkan label yang telah ditentukan.

7. Evaluasi model

Model yang telah dilatih akan dievaluasi menggunakan data pengujian dan data sampel cadangan. Evaluasi akan dilakukan untuk mengukur performa model dalam melakukan prediksi pada data EUR/USD. Di mana evaluasi model akan menggunakan 3 metrik yaitu MAE, MSE, dan MAPE. Kedua pengujian akan diuji ketiga metrik tersebut.

8. Output

Output dari penelitian ini berupa hasil dari prediksi pada Log return dari Closing price. Lalu hasil prediksi tersebut akan direkonstruksi menjadi Close. Hasil prediksi tersebut diharapkan dapat memberikan gambaran bagaimana model Informer melakukan prediksi pada data EUR/USD.

3.2 Pengumpulan Data

Data diunduh melalui platform forexsb.com dan memiliki bentuk data time series dengan data tanggal dan data OHLCV seperti pada **gambar 3.2**. Data berisikan Tahun, Bulan, dan tanggal yang dalam satu kesatuan, lalu jam dengan interval 1 jam, Open price, High price, Low price, Closing price, dan Volume.

2010-04-06 10:00	1.342501	1.342771	1.341911	1.342481	1308
2010-04-06 11:00	1.342461	1.343331	1.341751	1.342553	329
2010-04-06 12:00	1.342701	1.343621	1.341231	1.341632	474
2010-04-06 13:00	1.341791	1.343601	1.341421	1.342772	438
2010-04-06 14:00	1.342751	1.343971	1.341291	1.343533	163
2010-04-06 15:00	1.343291	1.343631	1.340111	1.340983	952
2010-04-06 16:00	1.341121	1.342101	1.339871	1.340444	369
2010-04-06 17:00	1.340401	1.342161	1.340141	1.340975	004
2010-04-06 18:00	1.340931	1.341981	1.339661	1.340845	007
2010-04-06 19:00	1.340871	1.342011	1.339531	1.340133	596

Sumber: forexsb.com

Gambar 3. 2 Contoh 10 data pertama EUR/USD yang telah diunduh

Data diunduh dengan konfigurasi 1 jam untuk intervalnya, menggunakan zona waktu Singapura yaitu GMT + 8, dan tidak menyertakan data pada hari sabtu dan minggu. Dengan konfigurasi tersebut, data yang berhasil diunduh sebanyak 95.558 baris. Data

yang diunduh belum memiliki nama kolom dan hanya data berurut seperti pada **Gambar 3.2**, sehingga masih butuh preprocessing.

3.3 Perancangan Data

Penelitian ini akan menggunakan dataset bar EUR/USD yang diunduh dari platform forexsb.com. Data dikumpulkan seperti yang dijelaskan pada **Subbab 3.2**. Pada tahap ini, data mentah akan diubah menjadi struktur yang dapat digunakan untuk implementasi model. Dataset mentah tersebut selanjutnya akan diproses menjadi rangkaian data terstruktur tanpa menganggap adanya header, kemudian setiap kolom akan diberi nama agar lebih jelas. Proses dilakukan berdasarkan kriteria visual yang telah ditetapkan pada **Tabel 3.1**.

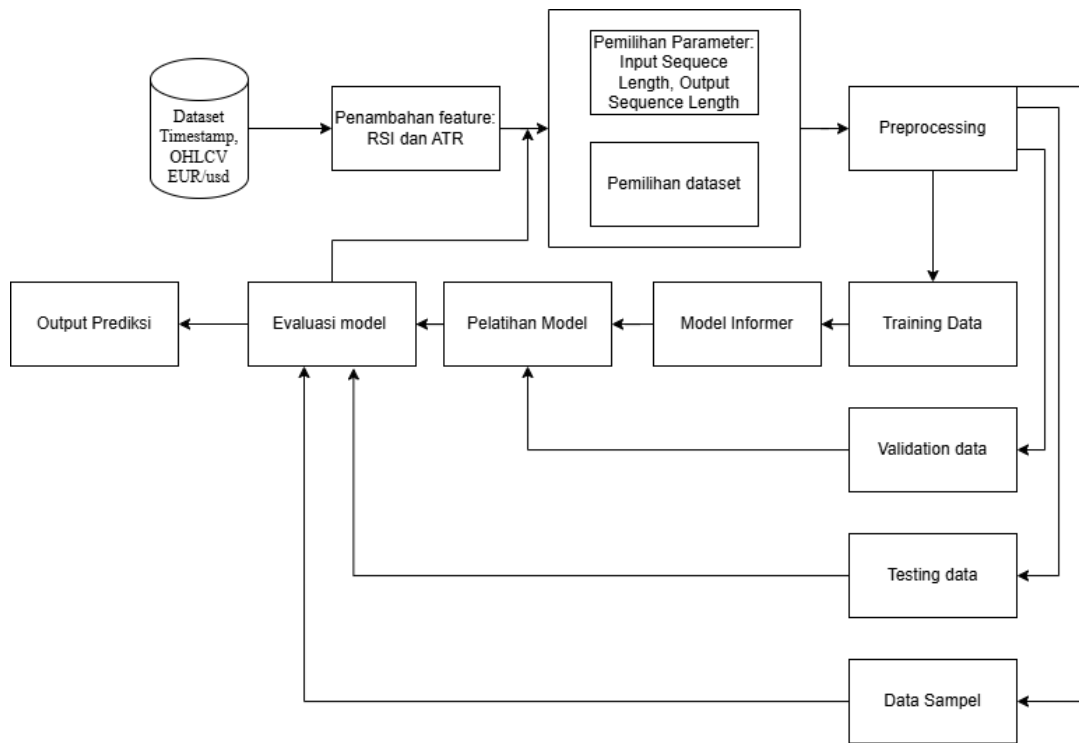
Tabel 3. 1 Skema struktur data EUR/USD

Datetime	Open	High	Low	Close	Volume
2010-04-06 10:00	1.34250	1.34277	1.34191	1.34248	1308
2010-04-06 11:00	1.34246	1.34333	1.34175	1.34255	3329
2010-04-06 12:00	1.34270	1.34362	1.34123	1.34163	2474
2010-04-06 13:00	1.34179	1.34360	1.34142	1.34277	2438
2010-04-06 14:00	1.34275	1.34397	1.34129	1.34353	3163
2010-04-06 15:00	1.34329	1.34363	1.34011	1.34098	3952
2010-04-06 16:00	1.34112	1.34210	1.33987	1.34044	4369
2010-04-06 17:00	1.34040	1.34216	1.34014	1.34097	5004
2010-04-06 18:00	1.34093	1.34198	1.33966	1.34084	5007
2010-04-06 19:00	1.34087	1.34201	1.33953	1.34013	3596

Tabel 3.1 menampilkan bagaimana contoh perancangan pada data mentah yang telah diunduh. Contoh perancangan ini digunakan untuk mempersiapkan data mentah menjadi data yang lebih siap digunakan, sehingga membantu dalam pemrosesan data lebih lanjut. Di mana pada setiap kolom akan digunakan sebagai fitur *input* pada model dan digunakan untuk melakukan pemrosesan penambahan fitur *input* baru seperti RSI, ATR, dan Log return.

3.4 Perancangan Proses/Algoritma

Penelitian ini menggunakan model Informer untuk melakukan prediksi pada pasar valuta asing dengan pasangan mata uang EUR/USD. Alur proses ini, secara keseluruhan ditunjukkan pada gambar 3.3



Gambar 3. 3 Flowchart Perancangan Proses/Algoritma

1. Dataset Timestamp dan OLCV EUR/US

Penelitian ini menggunakan dataset forex dengan pasangan mata uang EUR/USD dengan interval waktu 1 jam. Data yang digunakan pada model akan menggunakan data yang telah diolah seperti yang dipaparkan pada **Subbab 3.3**. Di mana dataset akan berisikan Open, High, Low, Close, Volume, dan Date. Di mana kolom date akan digunakan sebagai *time embedding* yang akan digunakan sebagai fitur *input* pada model.

Lalu kolom OHLCV akan digunakan untuk mengkomputasi fitur tambahan yaitu Log return, RSI, dan ATR. Setelah itu seluruh kolom tersebut akan digunakan sebagai fitur *input* untuk pelatihan pada model. Sehingga model akan menerima 9 fitur *input* untuk melakukan prediksi pada log return, yang di mana hasil prediksi akan dilakukan rekonstruksi kembali menjadi harga close.

2. Penambahan fitur Log Return, RSI, dan ATR

Dataset yang telah akan diolah terlebih dahulu agar bisa mendapat fitur baru sebelum proses preprocessing. Tahapan ini meliputi menghapus data delay karena ketiga fitur tersebut prosesnya membutuhkan data 2 hingga 14 window. Di mana dari 95.558 baris yang telah diunduh, karena log return terdapat data delay sebanyak 1, lalu RSI dan ATR terdapat sebanyak 14 data delay maka setelah penambahan ketiga fitur tersebut 14 baris pertama akan dihapus dari data. Sehingga data yang sudah siap digunakan tidak terdapat nilai kosong karena pengaruh data delay dari penambahan fitur tersebut.

Karena proses menghilangkan data kosong pada data yang sudah siap jadi maka jumlah baris pada data akhir akan sebanyak 95.544 baris. Seluruh baris data ini akan terbagi lagi pada tahap pembagian data agar setiap bagian mendapatkan porsi yang sesuai untuk menunjang performa model dalam melakukan prediksi.

3. Pemilihan Hyperparameter

Setelah tahap penambahan fitur akan dipilih salah satu nilai *Input Sequence Length* dan *Output Sequence Length*. Di mana *Input Sequence length* berfungsi untuk menentukan sejauh apa model akan melihat data sebelumnya sebagai konteks historis dalam melakukan prediksi. Lalu *Output Sequence Length* berfungsi untuk seberapa panjang model akan melakukan prediksi. *Input sequence length* yang diuji adalah 48, 96, 192, dan 384. Lalu *output sequence length* yang diuji adalah 1, 4, dan 8. Di mana model akan diuji seluruh kombinasinya yang menghasilkan 12 skenario pengujian.

4. Preprocessing

Setelah pemilihan hyperparameter dan dataset, preprocessing akan dilakukan untuk menyiapkan data sebelum proses *Forecasting*. Tahapan ini meliputi Zero-Mean Normalization. Proses normalisasi akan dilakukan hanya pada data pelatihan, hal ini dilakukan untuk mencegah terjadinya kebocoran data (*data leakage*) dan juga agar evaluasi model akan dilakukan tetap adil dengan mencerminkan kondisi nyata yang di mana model hanya memiliki data historis saat melakukan prediksi.

5. Pembagian data

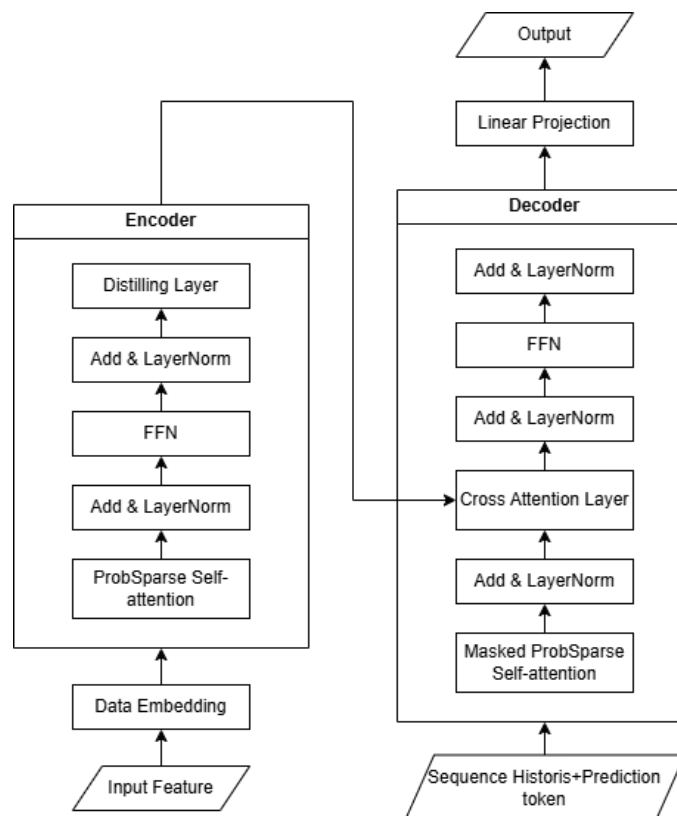
Setelah tahap preprocessing, data akan terlebih dahulu diambil sebagian kecilnya untuk melakukan prediksi sampel. Lalu sebagian besar data sisanya akan dibagi lagi menjadi 3 bagian yang akan digunakan sebagai data pelatihan, dan sebagiannya lagi

dibagi untuk data uji dan data validasi untuk keperluan pengujian performa model. Pembagian dilakukan dengan memisahkan data terlebih dahulu untuk data sampel cadangan sebanyak 491 baris. Pemilihan ini dilakukan agar setiap skenarionya melakukan 100 titik prediksi dengan memulai pada titik yang sama.

Lalu sisa dari data yang telah dipisahkan sebanyak 95.544 baris akan dibagi lagi menjadi tiga bagian yaitu pelatihan, evaluasi, dan pengujian. Rasio pembagian yang digunakan adalah 70:15:15. Pemilihan rasio tersebut karena rasio ini cukup umum digunakan dalam pelatihan model. Rasio ini dapat memberikan data pelatihan yang cukup besar dengan tetap memberikan jumlah data evaluasi dan pengujian yang memadai untuk memberikan model yang optimal.

6. Model Informer

Data yang telah disiapkan akan diproses menggunakan arsitektur Informer. Aliran data akan disesuaikan dengan target fitur dan *hyperparameter* yang ingin diuji. Arsitektur dari model dapat dilihat pada **Gambar 3. 4**.



Gambar 3. 4 Arsitektur Model Informer

Arsitektur dari informer dimulai dari *input feature* yang akan berisikan OHLCV, RSI, ATR, dan Log return. Sebelum masuk ke dalam model, data dari *input* akan diproses terlebih dahulu melalui lapisan *Data Embedding*, di mana akan mentransformasi nilai numerik mentah menjadi representasi fitur berdimensi tinggi (*high-dimensional*). Lapisan ini berisikan *Value Embedding*, yang akan mengkonversi nilai *input* menjadi menjadi representasi vektor. Lalu *Positional Embedding* yang akan memberikan urutan sekuensial dari data *time series*. Lalu yang terakhir adalah *Temporal Embedding* yang akan menggabungkan informasi sementara (*temporal*) seperti *timestamps*. Dengan menggabungkan ketiga itu, model dapat menunjukkan karakteristik numerik dan dependensi *temporal* dari *input sequence* sebelum ekstraksi fitur dimulai.

Nilai representasi dari lapisan *embedding* akan dilempar ke blok *encoder*, di mana blok tersebut akan mengekstraksi pola yang bermakna dari *sequence* historis. Pertama akan melewati mekanisme *Probsparse self-attention* untuk mengetahui dependensi yang paling informatif sekaligus mengurangi kompleksitas komputasi secara efisien. Hasil dari mekanisme tersebut akan diproses melalui *Add & LayerNorm*, di mana proses pelatihan akan distabilisasi dan memudahkan *gradient propagation* selama pelatihan. Fitur yang melewati *Add & LayerNorm* akan ternormalisasi akan disempurnakan oleh *Feed Forward Network* (FFN) untuk mempelajari representasi fitur di tingkat yang lebih tinggi kemudian diikuti oleh *Add & LayerNorm* lagi. Lalu diakhiri dengan *Distilling Layer* yang berisikan *one-dimensional convolution*, *batch normalization*, *ELU activation*, dan *max pooling*. Lapisan ini mengkompresi panjang *sequence* dengan tetap memberikan informasi yang penting, sehingga mengurangi penggunaan *memory* dan meningkatkan efisiensi komputasi sebelum representasi dari *encoder* dilempar ke blok *decoder*.

Lalu pada blok *decoder*, blok akan menerima *sequence* historis dan *prediction token* yang juga akan melalui *embedding* sebelum diproses. Lapisan pertama pada blok ini adalah *Masked ProbSparse Self-attention*. Untuk memastikan setiap prediksi hanya dapat bergantung pada observasi sebelumnya tanpa mengakses informasi dari masa depan. Hasil dari lapisan tersebut akan dinormalisasi melalui *Add & LayerNorm* sebelum memasuki *Cross Attention Layer*, di mana pada *Cross Attention Layer* informasi dari *encoder* akan diproses untuk diekstraksi pada *decoder*. Hal ini akan membuat model dapat memanfaatkan pola historis secara efektif ketika menghasilkan prediksi masa

depan. Hasil dari lapisan tersebut kemudian disempurnakan melalui *Add & LayerNorm* yang diikuti dengan lapisan FFN dan diakhiri juga dengan *Add & LayerNorm*. Lalu lapisan terakhir adalah *linear projection* yang akan memetakan representasi dari *decoder* menjadi dimensi dari target prediksi dan menghasilkan nilai akhir dari prediksi.

7. Pelatihan model

Tahap pelatihan akan dilakukan dengan menggunakan fitur yang sudah dipilih. Pada tahap ini, ditentukan konfigurasi hyperparameter yang telah dipilih meliputi panjang *input* dan output *optimizer*, fungsi *loss*, learning rate, jumlah *epoch*, dan *batch size*. Selama proses iterasi pelatihan, Data validasi digunakan sebagai acuan untuk memantau *loss*, serta menjadi dasar aktivasi mekanisme *Early Stopping* jika tidak terjadi peningkatan performa

8. Evaluasi model

Setelah model selesai dilatih, dilakukan evaluasi menggunakan data uji. Karena data ini tidak pernah dilibatkan dalam proses pelatihan maupun validasi, hasil evaluasi ini akan mencerminkan kinerja objektif model di dunia nyata. Metrik yang diukur meliputi MAE, MSE dan MAPE.

9. Output prediksi

Hasil akhir dari sistem adalah prediksi Log return dari closing price. Setelah itu pelatihan akan diulang dengan fitur dan *hyperparameter* yang berbeda.

Konfigurasi detail model Informer yang digunakan disajikan dalam **Tabel 3.2**, sedangkan konfigurasi *hyperparameter* pelatihan ditunjukkan pada **Tabel 3.3**.

3.5 Perancangan Pengujian

Pengujian pada penelitian ini dilakukan untuk mengukur kinerja model Informer dalam melakukan prediksi Log return dari closing price pada pasar valuta asing dengan pasangan mata uang EUR/USD. Dataset yang digunakan telah dibagi menjadi data latih, data validasi, dan data uji sesuai dengan **Subbab 3.3**, sehingga evaluasi dilakukan dengan data uji yang bersifat independen dan tidak digunakan saat proses pelatihan. Konfigurasi model dan pengaturan pelatihan yang digunakan dalam pengujian ditunjukkan pada **Tabel 3.2** dan **3.1**. Evaluasi performa model dilakukan menggunakan metrik MSE, MAE, dan MAPE untuk memberikan perspektif yang berbeda saat analisis hasil akhir.

Tabel 3. 2 Konfigurasi perancangan model

Komponen	Konfigurasi.
Environment	Google Collab
Runtime	T4 GPU
Arsitektur	Informer.
<i>Input</i> Fitur	Timestamp, Open, Low, High, Close, Volume, RSI, ATR, dan Log Return.
Output model	Log Return

Perangan data, model, dan evaluasi akan dilakukan pada Google Collab, lalu kode akan di eksekusi menggunakan *runtime* T4 GPU. Arsitektur Informer akan dirancang dan dilatih serta pengujian dalam *environment* Google Collab. Model akan dirancang untuk menerima 9 fitur *input* sebagai konteks historisnya untuk melakukan prediksi pada nilai masa depan dari *log return* yang akan dikonversikan kembali menjadi harga *close*.

Tabel 3. 3 Konfigurasi hyperparameter model

Parameter	Nilai
d_model (Dimensi Model)	64
n_heads (Attention Head)	4
<i>Encoder</i> layer	2
<i>Decoder</i> layer	1
Dimensi FFN	256
Dropout	0.01
Probsparse Factor	5 (c=5)
Panjang <i>input encoder</i>	48, 96, 192, dan 384.
Panjang output	1, 4, dan 8.
Activation	Gelu
Learning rate	0.0002
Maksimum Epoch	50
Early stopping	Patience = 5.

Prosedur pengujian dilakukan dengan melatih model menggunakan data latih dan memantau performa menggunakan data validasi pada setiap epoch. Selama pelatihan, mekanisme *early stopping* diterapkan dengan patience 10 untuk menghentikan pelatihan ketika performa validasi tidak mengalami peningkatan, sehingga mengurangi risiko *overfitting*. Lalu model akan digunakan untuk melakukan prediksi data uji, lalu hasil prediksi dibandingkan dengan data aktual untuk menghitung evaluasi metrik dengan menggunakan MAE, MSE, dan MAPE. Setelah itu menguji *Input* fitur, panjang *input* dan panjang output lainnya, lalu uji perbandingan model.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Penerapan/Pengolahan Data

Pada tahap penerapan dan pengolahan data menguraikan eksekusi dari serangkaian proses pematangan data sebelum masuk ke proses pelatihan model. Data diperoleh dari platform forexsb.com dengan pemilihan mata uang EUR/USD. Dengan beberapa konfigurasi pada platform data lalu diolah untuk memastikan tersusun dalam format dan struktur matematis yang sesuai dengan algoritma Informer agar dapat diproses secara optimal. Tahap ini menghasilkan data yang telah melalui pembersihan, transformasi, dan pembagian dataset.

4.1.1 Pengumpulan dan Penyaringan Data.

Sebagaimana yang telah dirancang pada **BAB III**, sumber data EUR/USD yang digunakan untuk pelatihan diperoleh dari forexsb.com dengan interval 1 jam dalam periode waktu 6 April 2010 hingga 17 April 2026. Tampilan halaman pengambilan data ditunjukkan pada **Gambar 4. 1**.

The screenshot displays the 'Download Historical Forex Data' interface on the forexsoftware.com website. The page includes a navigation bar with links like 'Forex Software', 'Products', 'Purchase', 'Docs & Guides', 'Forum', 'EA Studio', and 'Top 10'. Below the navigation bar, there's a section titled 'Download Historical Forex Data' with a sub-header 'Import historical data in MetaTrader, Excel, Forex Strategy Builder, Expert Advisor Studio.'.

The main form allows users to select a 'Symbol' (EURUSD - Euro / US Dollar) and a 'Format' (Forex Strategy Builder (CSV)). Below this, there's a table with columns: Symbol, Period, Bars, From time, To time, and Download.

Symbol	Period	Bars	From time	To time	Download
EURUSD	M1	0	-	-	
EURUSD	M5	0	-	-	
EURUSD	M15	0	-	-	
EURUSD	M30	0	-	-	
EURUSD	H1	0	-	-	
EURUSD	H4	0	-	-	
EURUSD	D1	0	-	-	

At the bottom of the page, there is a copyright notice: 'Copyright © 2026 Forex Software Ltd'.

Gambar 4. 1 Tampilan Halaman Pengambilan Data forexsb.com

Dataset diunduh dengan konfigurasi "Maximum bars" 100.000 pada zona waktu Singapura dan tidak meliputi data akhir pekan seperti pada **Gambar 4. 2**. Didapatkan data sebanyak 95.558 dengan format file csv tanpa penamaan kolom.

Gambar 4. 2 Konfigurasi Data yang Diunduh

Data mentah tersebut terdiri dari Datetime, Open, High, Low, Close, dan Volume dalam interval waktu 1 jam, tetapi data yang ter-unduh belum memiliki penamaan kolom dan terkumpul hanya pada 1 kolom yang dapat dilihat pada **Tabel 4. 1**. Karena itu dibutuhkan proses untuk pemisahan data dan penamaan kolom.

Tabel 4. 1 Sepuluh Baris Pertama Data Mentah EUR/USD

2010-04-06 10:00	1.34250	1.34277	1.34191	1.34248	1308
2010-04-06 11:00	1.34246	1.34333	1.34175	1.34255	3329
2010-04-06 12:00	1.34270	1.34362	1.34123	1.34163	2474
2010-04-06 13:00	1.34179	1.34360	1.34142	1.34277	2438
2010-04-06 14:00	1.34275	1.34397	1.34129	1.34353	3163
2010-04-06 15:00	1.34329	1.34363	1.34011	1.34098	3952
2010-04-06 16:00	1.34112	1.34210	1.33987	1.34044	4369

2010-04-06 17:00	1.34040	1.34216	1.34014	1.34097	5004
2010-04-06 18:00	1.34093	1.34198	1.33966	1.34084	5007
2010-04-06 19:00	1.34087	1.34201	1.33953	1.34013	3596

Sebagaimana yang telah dirancang pada **BAB III Subbab 3.3**, data mentah yang sebelumnya belum memiliki nama kolom, terkumpul dalam 1 baris, dan belum disertai dengan kolom RSI, ATR, dan *Log Return*.

```
df = pd.read_csv('/content/EURUSD60.csv', sep='\t', header=None,
                 names=['date', 'Open', 'High', 'Low', 'Close', 'Volume'])

# Koversi kolom 'date' menjadi tipe data date dan dengan format Tahun-
# Bulan-Hari Jam Menit
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d %H:%M')

# Mengkomputasikan kolom 'Close' untuk mendapatkan Log Return
df['log_return'] = np.log(df['Close'] / df['Close'].shift(1))

# Pemrosesan RSI(14)
df['RSI_14'] = ta.rsi(df['Close'], length=14)

# Pemrosesan ATR(14)
df['ATR_14'] = ta.atr(df['High'], df['Low'], df['Close'], length=14)

# Menghapus 14 baris pertama (NaNs dari RSI/ATR)
df = df.iloc[14:].reset_index(drop=True)

# Kolom final
final_columns = ['date', 'Open', 'High', 'Low', 'Close',
                 'Volume', 'log_return', 'RSI_14', 'ATR_14']

df = df[final_columns]
df.to_csv('/content/EURUSD60_processed.csv', index=False)
```

Gambar 4. 3 Kode Program Penamaan Kolom dan Penambahan RSI, ATR, dan *Log Return*

Berdasarkan **Gambar 4. 3** fungsi `pd.read_csv()` digunakan untuk membaca file mentah bernama `EURUSD60.csv` dari file yang sudah ditambahkan di Google Colab. Lalu karena file `EURUSD60.csv` setiap record nya tergabung dalam 1 baris dan setiap

kolomnya hanya terpisah oleh spasi, maka digunakan fungsi `sep='\t'` untuk memberikan penamaan kolom sesuai dengan variabel names. Lalu setelah diberi penamaan kolom, kolom 'date' dikonversi menjadi datetime pada tipe datanya agar memastikan terbaca sebagai datetime. Setelah itu data siap diolah lebih lanjut seperti **Tabel 4.2** dibandingkan data pada **Tabel 4.1**.

Tabel 4.2 Sepuluh Baris Pertama Pada Data yang Sudah Diberi Penamaan Kolom

Datetime	Open	High	Low	Close	Volume
2010-04-06 10:00	1.34250	1.34277	1.34191	1.34248	1308
2010-04-06 11:00	1.34246	1.34333	1.34175	1.34255	3329
2010-04-06 12:00	1.34270	1.34362	1.34123	1.34163	2474
2010-04-06 13:00	1.34179	1.34360	1.34142	1.34277	2438
2010-04-06 14:00	1.34275	1.34397	1.34129	1.34353	3163
2010-04-06 15:00	1.34329	1.34363	1.34011	1.34098	3952
2010-04-06 16:00	1.34112	1.34210	1.33987	1.34044	4369
2010-04-06 17:00	1.34040	1.34216	1.34014	1.34097	5004
2010-04-06 18:00	1.34093	1.34198	1.33966	1.34084	5007
2010-04-06 19:00	1.34087	1.34201	1.33953	1.34013	3596

Setelah data mentah diberi penamaan kolom, data ditambahkan kolom baru yaitu *Log Return*, RSI dengan *window* 14, dan ATR dengan *window* 14. Terlihat pada **Gambar 4.3** *Log Return* diolah dengan operasi yang telah dijelaskan pada **BAB II Subbab 2.2.1**. Lalu untuk mencari RSI dan ATR digunakan *library* `pandas_ta` yang langsung mengoprasikannya seperti yang sudah dijelaskan pada **BAB II Subbab 2.2.2** dan **Subbab 2.2.3**. Hasil dari operasi tersebut terlihat pada **Gambar 4.4**.

	date	log_return	RSI_14	ATR_14
2010-04-06	10:00:00	NaN	NaN	NaN
2010-04-06	11:00:00	0.000052	100.000000	NaN
2010-04-06	12:00:00	-0.000685	49.726776	NaN
2010-04-06	13:00:00	0.000849	69.911950	NaN
2010-04-06	14:00:00	0.000566	76.644485	NaN
2010-04-06	15:00:00	-0.001900	42.379450	NaN
2010-04-06	16:00:00	-0.000403	38.458422	NaN
2010-04-06	17:00:00	0.000395	43.940678	NaN
2010-04-06	18:00:00	-0.000097	42.930475	NaN
2010-04-06	19:00:00	-0.000530	37.816871	NaN
2010-04-06	20:00:00	-0.002638	23.090500	NaN
2010-04-06	21:00:00	0.000419	27.888008	NaN
2010-04-06	22:00:00	0.000748	35.611927	NaN
2010-04-06	23:00:00	-0.000344	33.817545	0.002502
2010-04-07	00:00:00	-0.000209	32.736277	0.002516
2010-04-07	01:00:00	-0.000576	29.904581	0.002447
2010-04-07	02:00:00	0.001928	46.578773	0.002532
2010-04-07	03:00:00	0.000814	51.795892	0.002489
2010-04-07	04:00:00	-0.000403	49.230784	0.002416
2010-04-07	05:00:00	-0.000470	46.346997	0.002348

Gambar 4. 4 Dua Puluh Baris Pertama Kolom Log Return, RSI, dan ATR

Terlihat pada **Gambar 4. 4**, karena RSI dan ATR yang digunakan membutuhkan 14 window sebelumnya, maka dari itu 13 record pertama nilainya menjadi NaN. Maka dari itu 14 baris pertama dihilangkan seperti yang terlihat pada **Gambar 4. 3** untuk memastikan tidak ada nilai kosong pada data hasilnya. Setelah itu data digabungkan dengan urutan kolom 'date', 'Open', 'High', 'Low', 'Close', 'Volume', 'log_return', 'RSI_14', dan 'ATR_14'. Setelah itu data disimpan dengan nama EURUSD60_processed.csv, cuplikan 20 baris pertama data yang sudah matang dapat dilihat pada **Tabel 4. 3**.

Tabel 4. 3 Dua Puluh Baris Pertama Data EURUSD Setelah Diproses

date	Open	High	Low	Close	Volume	log return	RSI 14	ATR 14
4/7/2010 0:00	1.33768	1.33805	1.33536	1.33742	4100	- 0.000209336	32.73628	0.002516
4/7/2010 1:00	1.33756	1.33758	1.33602	1.33665	3475	- 0.000575901	29.90458	0.002447
4/7/2010 2:00	1.33674	1.33995	1.33632	1.33923	3174	0.001928338	46.57877	0.002532
4/7/2010 3:00	1.33922	1.34086	1.33893	1.34032	3061	0.000813569	51.79589	0.002489
4/7/2010 4:00	1.34039	1.34057	1.3391	1.33978	3553	-0.00040297	49.23078	0.002416
4/7/2010 5:00	1.33975	1.3403	1.33883	1.33915	3542	- 0.000470337	46.347	0.002348
4/7/2010 6:00	1.33933	1.34003	1.33893	1.33988	1607	0.000544973	50.00167	0.002259
4/7/2010 7:00	1.33971	1.3408	1.33772	1.33811	1628	- 0.001321887	42.45111	0.002318
4/7/2010 8:00	1.33807	1.33854	1.33699	1.3374	2215	-0.00053074	39.8515	0.002263
4/7/2010 9:00	1.33747	1.33878	1.33701	1.33807	1996	0.000500847	43.37542	0.002228
4/7/2010 10:00	1.33827	1.33914	1.33711	1.33846	2557	0.000291422	45.38135	0.002214
4/7/2010 11:00	1.33835	1.33909	1.33772	1.3384	3458	-4.48E-05	45.11655	0.002153
4/7/2010 12:00	1.33844	1.33916	1.33752	1.33815	2591	- 0.000186808	43.96542	0.002117
4/7/2010 13:00	1.33811	1.33817	1.33574	1.33614	2532	- 0.001503203	36.01013	0.002139
4/7/2010 14:00	1.33605	1.3375	1.33544	1.33686	3128	0.000538721	40.1853	0.002133
4/7/2010 15:00	1.33723	1.33959	1.33692	1.33805	4155	0.00088975	46.40906	0.002176
4/7/2010 16:00	1.33808	1.3396	1.33746	1.33874	4605	0.000515543	49.67858	0.002174
4/7/2010 17:00	1.33866	1.34018	1.33772	1.33833	4949	- 0.000306305	47.81199	0.002194
4/7/2010 18:00	1.33837	1.33899	1.33434	1.33492	5010	- 0.002551203	35.77298	0.002369
4/7/2010 19:00	1.33499	1.33577	1.33309	1.33482	3386	-7.49E-05	35.49076	0.002392

4.1.2 Validasi Kualitas Data

Setelah data mentah EUR/USD diolah dan ditambahkan kolom tambahan *Log return*, RSI, dan ATR, tahap selanjutnya adalah melakukan validasi pada kualitas data EUR/USD yang telah diproses sebelum lanjut ke tahap permodelan. Tahap ini penting untuk memastikan jika data tidak memiliki nilai kosong, duplikasi pada kolom date, nilai

numerik yang tidak valid, atau data akhir pekan. Karena dataset berupa data time-series perjam, kualitas data menjadi sangat penting karena mempengaruhi formasi sliding window dan mengakibatkan pelatihan model yang tidak akurat. Proses validasi dilakukan pada data mentah dan data yang sudah diproses. Data mentah terdiri dari 6 kolom yaitu date, open, high, low, close, volume. Lalu setelah pemrosesan data menjadi 9 kolom dengan kolom tambahan yaitu log_return, RSI_14, dan ATR_14. Dataset yang telah diproses selanjutnya digunakan sebagai *input* dari model Informer.

Tahap pertama adalah melakukan pengecekan *missing value* pada setiap kolom. Hasil menunjukkan bahwa data mentah maupun data yang sudah diproses tidak terdapat *missing value*. Hasil ini dapat diartikan jika setiap *record* pada kedua data memiliki nilai OHLCV yang lengkap, pada data yang sudah terproses juga menunjukkan jika 14 baris pertama yang digunakan untuk pemrosesan 3 kolom baru sudah dihilangkan.

Tahap kedua adalah memastikan tidak ada duplikasi data khususnya pada kolom date. Tahap ini penting karena duplikasi pada kolom date dapat membuat model mempelajari kondisi pasar yang sama. Hasil menunjukkan jika tidak terdapat duplikasi pada setiap kolom khususnya pada kolom date di data mentah maupun data yang sudah terproses.

Lalu tahap ketiga adalah memastikan jika tidak terdapat *record* akhir pekan, dan didapatkan jika tidak terdapat data pada akhir pekan pada kedua *dataset*. Lalu tahap validasi terakhir adalah untuk memastikan jika tidak ada nilai yang tidak valid. Proses ini diperlukan karena saat proses perhitungan *log return* dapat menyebabkan potensi nilai tidak valid karena jika nilai *close* sebelumnya bernilai 0 dan menyebabkan nilai tak hingga. Hasil dari validasi data dapat dilihat pada **Tabel 4. 4**.

Tabel 4. 4 Hasil Validasi Kualitas Data

Aspek Validasi	Data Mentah	Data Terproses
Jumlah Baris	95, 558	95, 544
Jumlah Kolom	0	0
Missing Values	0	0
Duplicated Rows	0	0
Record Akhir Pekan	0	0
Nilai Invalid	0	0
Rentang Tanggal	2010-04-06 10:00 - 2026-04-17 23:00	2010-04-07 00:00 - 2026-04-17 23:00

4.1.3 Pembagian Data

Setelah data diproses dan divalidasi, tahap selanjutnya adalah pembagian data (*data splitting*) menjadi data untuk model dan sampel prediksi. Data yang telah terproses berisi sebanyak 95, 544 kolom dengan rentang waktu dari 04-07-2010 pukul 00:00 hingga 17-04-2026 pukul 23:00. Data ini tidak dibagi secara acak karena data EUR/USD adalah *time-series*, di mana dibutuhkan urutan kronologis untuk mencegah data masa depan masuk ke dalam proses pelatihan.

Dimulai dari membagi data menjadi 2 bagian yaitu data untuk pelatihan model dan data untuk prediksi sampel. Data kepada pelatihan model dibagi lagi menjadi 3 bagian yang digunakan untuk pelatihan, validasi, dan tes, sedangkan data prediksi sampel digunakan untuk sampel prediksi bergulir (*rolling*) pada tahap akhir. Pembagian ini dilakukan untuk memastikan jika evaluasi model dan visualisasi prediksi sampel dilakukan pada data setelah periode pelatihan dan tes.

```
def required_reserved_rows() -> int:
    # Untuk prediksi sampel yang disejajarkan:
    # jumlah baris = konteks seq_len terbesar + pred_len + jumlah prediksi
    yang diinginkan - 1
    return SAMPLE_ALIGNMENT_SEQ_LEN + MAX_PRED_LEN +
    SAMPLE_TARGET_PREDICTIONS - 1

def resolve_effective_modeling_end(full_df: pd.DataFrame):
    dates = pd.to_datetime(full_df[DATE_COL]).reset_index(drop=True)
    sample_end_ts = pd.Timestamp(SAMPLE_PREDICTION_END) if
    SAMPLE_PREDICTION_END else dates.iloc[-1]
    valid_end_positions = np.flatnonzero((dates <=
    sample_end_ts).to_numpy())
    if len(valid_end_positions) == 0:
        raise ValueError("Tidak ada baris data yang tersedia pada atau
    sebelum SAMPLE_PREDICTION_END.")
    sample_end_idx = int(valid_end_positions[-1])
    sample_start_idx = sample_end_idx - required_reserved_rows() + 1
    model_end_idx = sample_start_idx - 1
    if model_end_idx < 0:
        raise ValueError("Dataset terlalu pendek untuk ukuran sampel yang
    diminta.")

    return dates.iloc[model_end_idx], {
        "sample_start_idx": int(sample_start_idx),
        "sample_end_idx": int(sample_end_idx),
        "model_end_idx": int(model_end_idx),
        "reserved_rows_needed": int(required_reserved_rows()),
        "first_aligned_forecast_idx": int(sample_start_idx +
    SAMPLE_ALIGNMENT_SEQ_LEN),
    }
```

```

EFFECTIVE_MODELING_END, SPLIT_INFO = resolve_effective_modeling_end(df)
MODELING_END = str(EFFECTIVE_MODELING_END)

modeling_mask = (df[DATE_COL] >= pd.Timestamp(MODELING_START)) &
(df[DATE_COL] <= EFFECTIVE_MODELING_END)
sample_mask = df[DATE_COL] > EFFECTIVE_MODELING_END
if SAMPLE_PREDICTION_END is not None:
    sample_mask = sample_mask & (df[DATE_COL] <=
pd.Timestamp(SAMPLE_PREDICTION_END))

modeling_df = df[modeling_mask].copy().reset_index(drop=True)
sample_df = df[sample_mask].copy().reset_index(drop=True)

def split_indices(full_df: pd.DataFrame):
    dates = pd.to_datetime(full_df[DATE_COL]).reset_index(drop=True)
    model_positions = np.flatnonzero(((dates >=
pd.Timestamp(MODELING_START)) & (dates <=
EFFECTIVE_MODELING_END)).to_numpy())
    if len(model_positions) == 0:
        raise ValueError("Tidak ditemukan baris data untuk periode
modeling.")

    model_start = int(model_positions[0])
    model_end = int(model_positions[-1] + 1)
    n_model = model_end - model_start
    train_end = model_start + int(n_model * TRAIN_RATIO)
    val_end = model_start + int(n_model * (TRAIN_RATIO + VAL_RATIO))

    sample_positions = np.flatnonzero((dates >
EFFECTIVE_MODELING_END).to_numpy())
    if SAMPLE_PREDICTION_END is not None:
        sample_positions = np.flatnonzero(((dates >
EFFECTIVE_MODELING_END) & (dates <=
pd.Timestamp(SAMPLE_PREDICTION_END))).to_numpy())

    if len(sample_positions) == 0:
        sample_start = sample_end = None
    else:
        sample_start = int(sample_positions[0])
        sample_end = int(sample_positions[-1] + 1)

    return {
        "model_start": model_start,
        "model_end": model_end,
        "train_end": train_end,
        "val_end": val_end,
        "sample_start": sample_start,
        "sample_end": sample_end,
    }

```

Gambar 4. 5 Kode Program Pembagian Data Cadangan dan Model

Berdasarkan hasil eksekusi program pada **Gambar 4. 5**, periode sampel prediksi adalah 491 baris terakhir dari data yang telah di proses seperti pada **Subbab 4.1.1**. Angka ini dikalkulasikan berdasarkan *input sequence* terpanjang, panjang prediksi terpanjang, dan jumlah sampel prediksi berguling. *Input sequence* terpanjang adalah 384, lalu

panjang prediski maksimal adalah 8, dan jumlah sampel prediksi bergulir adalah 100. Karena itu jumlah baris data yang harus dicadangkan dapat dihitung sebagai berikut:

$$\text{Baris data cadangan} = 384 + 8 + 100 - 1 = 491$$

Data cadangan digunakan agar seluruh skenario dapat menghasilkan 100 titik prediksi bergulir dengan waktu prediksi yang sama. Berdasarkan perhitungan tersebut, data model berakhir pada 19-03-2026 pukul 16:00, sedangkan data cadangan dimulai dari 19-03-2026 pukul 17:00 hingga 17-04-2026 pukul 23:00. Lalu titik pertama setiap skenario dimulai pada 13-04-2026 pukul 13:00.

Setelah periode model ditentukan, data model dibagi menjadi data latih, validasi dan tes menggunakan rasio 70:15:15. Data latih digunakan untuk mempelajari parameter model, data validasi digunakan untuk mengamati performa model saat pelatihan model dan membantu *early stopping*, sedangkan data tes digunakan untuk mengevaluasi performa akhir model pada data yang belum terlihat (*unseen*).

Tabel 4. 5 Pembagian Dataset Untuk Model

Pembagian Data	Rasio	Tanggal Dimulai	Tanggal Berakhir	Jumlah Baris
Data Latih	70%	07-04-2010 00:00	04-06-2021 20:00	66, 537
Data Validasi	15%	04-06-2021 21:00	24-10-2023 11:00	14, 258
Data tes	15%	24-10-2023 12:00	19-03-2026 16:00	14, 258

Berdasarkan **Tabel 4. 5**. Data latih mendapatkan proporsi data terbanyak karena digunakan sebagai sumber data utama untuk pelatihan model. Data validasi mendapatkan data dengan periode waktu setelah data latih untuk merepresentasikan kondisi pasar terkini saat memantau proses pelatihan. Lalu dengan periode data tes setelah data variasi, agar evaluasi model dapat dilakukan pada data yang tidak terlihat terkini sebelum periode data sampel prediksi.

Pembagian secara kronologis ini sangat penting karena untuk melakukan peramalan finansial pada data *time-series* model harus hanya belajar dari data masa lalu dan dievaluasi pada data masa depan. Dengan mempertahankan urutan waktu, proses evaluasi menjadi realistis dan mendekati kondisi peramalan aktual, di mana data pasar masa depan tidak ada saat pelatihan.

4.1.4 Normalisasi Data dan Formasi *Sliding Window*

Setelah data untuk model dibagi menjadi data latih, validasi, dan tes, tahap selanjutnya adalah normalisasi dan formasi *sliding window*. Tahap ini penting karena fitur *input* memiliki rentang nilai yang berbeda. Contohnya nilai OHLC berada di sekitar 1.0 hingga 1.4, sedangkan kolom volume memiliki nilai yang lebih besar. Jika nilai ini digunakan secara langsung kepada model, fitur dengan rentang nilai yang lebih besar dapat mendominasi proses pelatihan model.

```
# =====
# 4. Scaling, fitur waktu, dataset, dan dataloader
# =====

class StandardScalerNP:
    def __init__(self):
        self.mean_ = None
        self.std_ = None

    def fit(self, values: np.ndarray):
        values = np.asarray(values, dtype=np.float32)
        if values.ndim == 1:
            values = values.reshape(-1, 1)
        self.mean_ = values.mean(axis=0)
        self.std_ = values.std(axis=0)
        self.std_[self.std_ == 0] = 1.0
        return self

    def transform(self, values: np.ndarray) -> np.ndarray:
        values = np.asarray(values, dtype=np.float32)
        if values.ndim == 1:
            values = values.reshape(-1, 1)
        return (values - self.mean_) / self.std_

    def inverse_transform(self, values: np.ndarray) -> np.ndarray:
        values = np.asarray(values, dtype=np.float32)
        return values * self.std_ + self.mean_

def make_time_features(dates: pd.Series) -> np.ndarray:
    d = pd.to_datetime(dates)
    return np.stack([
        d.dt.month.values,
        d.dt.day.values,
        d.dt.weekday.values,
        d.dt.hour.values,
    ], axis=1).astype(np.int64)
```

Gambar 4. 6 Kode Program Implementasi Standard Scaler dan Time Feature Extraction

Seperti pada **Gambar 4. 6**, normalisasi menggunakan *Standard Scaler*. *Standard Scaler* melakukan transformasi setiap fitur numerik dengan meng-subtraksi nilai mean dan membaginya dengan nilai standar deviasi. *Scaler* hanya diterapkan pada data latih

untuk menghindari kebocoran data (*data leakage*) dari data validasi, tes, dan data cadangan. Setelah mean dan standar deviasi didapatkan dari data latih, *scaler* yang sama digunakan untuk mentransformasi seluruh data.

Fitur *input* yang digunakan pada model berisikan 8 fitur historis yaitu Open, High, Low, Close, Volume, log_return, RSI_14, dan ATR_14. Sedangkan yang menjadi target pada model adalah log_return. Target juga dinormalisasi secara terpisah menggunakan *Standard Scaler*. Pemisahan ini dilakukan karena variabel target memiliki peran yang berbeda dari fitur *input* dan dibutuhkan untuk mengembalikan kembali ke skala asli sebelum dikonversi menjadi close.

Proses normalisasi diterapkan hanya pada fitur *input* numerik dan variabel target. Dapat dilihat pada **Gambar 4. 7** di bawah, fitur waktu tidak dilakukan penerapan normalisasi karena digunakan untuk penanda temporal kategorikal (*Categorical Temporal Marker*). Nilai close juga disimpan pada skala aslinya karena dibutuhkan pada rekonstruksi harga close dari log_return.

Setelah normalisasi, data ditransformasi menjadi format *supervised learning* menggunakan metode *sliding window*. Metode *sliding window* mengkonversi data berurutan (*sequential*) menjadi pasangan *input* dan output *sequence*. Pada setiap sampel, *encoder* menerima data *input* historis dengan panjang yang sama pada *seq_len*, sedangkan *decoder* menerima kombinasi dari nilai target yang diketahui dan posisi *zero-filled* masa depan. Lalu model melakukan prediksi nilai target masa depan berdasarkan panjang prediksi yang digunakan.

```
class InformerTimeSeriesDataset(Dataset):
    def __init__(self, feature_scaled, target_scaled, time_markers,
close_values, log_return_values, seq_len, label_len, pred_len):
        self.feature_scaled = feature_scaled.astype(np.float32)
        self.target_scaled = target_scaled.astype(np.float32)
        self.time_markers = time_markers.astype(np.int64)
        self.close_values = close_values.astype(np.float32)
        self.log_return_values = log_return_values.astype(np.float32)
        self.seq_len = int(seq_len)
        self.label_len = int(label_len)
        self.pred_len = int(pred_len)

    def __len__(self):
```

```

        return max(0, len(self.feature_scaled) - self.seq_len -
self.pred_len + 1)

    def __getitem__(self, index):
        s_begin = index
        s_end = s_begin + self.seq_len
        r_begin = s_end - self.label_len
        r_end = r_begin + self.label_len + self.pred_len

        return (
            torch.from_numpy(self.feature_scaled[s_begin:s_end]),
            torch.from_numpy(self.target_scaled[r_begin:r_end]),
            torch.from_numpy(self.time_markers[s_begin:s_end]),
            torch.from_numpy(self.time_markers[r_begin:r_end]),
            torch.tensor(self.close_values[s_end - 1],
dtype=torch.float32),
            torch.from_numpy(self.close_values[s_end +
self.pred_len].astype(np.float32)),
            torch.from_numpy(self.log_return_values[s_end +
self.pred_len].astype(np.float32)),
        )

def make_loader(dataset, batch_size, shuffle=False, drop_last=False):
    if dataset is None or len(dataset) == 0:
        return None

    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle,
drop_last=drop_last, num_workers=NUM_WORKERS)

```

Gambar 4. 7 Kode Program Implementasi Sliding Window Pada Formasi Dataset

Terlihat pada **Tabel 4. 6** di bawah, panjang *input sequence* yang digunakan adalah 48, 96, 192, dan 384. Nilai ini merepresentasikan *window* historis dari 2 hari, 4 hari, 8 hari, dan 16 hari karena data yang digunakan berada dalam *timeframe* 1 jam. Lalu panjang prediksi yang diuji adalah 1, 4, dan 8 jam ke depan. Karena itu, setiap skenario mengevaluasi model menggunakan kombinasi panjang konteks historis dan *horizon* peramalan yang berbeda. Panjang *label* adalah setengah dari panjang *input sequence*. Panjang *label* ini digunakan oleh blok *decoder* pada informer untuk mengetahui bagian historis yang diketahui sebelum melakukan peramalan pada nilai target selanjutnya.

Tabel 4. 6 Skenario Panjang Sequence yang Akan Diuji

No.	Panjang <i>Input Sequence</i> (SEQ_LEN)	Panjang <i>Output Sequence</i> (PRED_LEN)	Panjang Label (LABEL_LEN)
1	48	1	24
2	48	4	24
3	48	8	24
4	96	1	48
5	96	4	48
6	96	8	48
7	192	1	96
8	192	4	96
9	192	8	96
10	384	1	192
11	384	4	192
12	384	8	192

Banyaknya sampel yang dihasilkan dari *sliding window* tergantung dari panjang *input sequence* dan panjang dari *output sequence*. *Input sequence* yang panjang mengurangi sampel pelatihan yang tersedia karena semakin banyak juga data historis yang dibutuhkan pada setiap sampel prediksinya. Sama halnya dengan jika memanjangkan panjang dari output maka sedikit mengurangi sampel yang tersedia karena lebih banyak nilai target masa depan yang dibutuhkan.

Setelah sampel *sliding window* dibuat, data dimuat di pemuat data (*dataloader*) dengan *batch size* 32. *Dataloader* pada pelatihan menggunakan pengacakan (*shuffling*) karena sampel sudah membentuk kronologis (*Chronologically*) dan *shuffling* hanya mempengaruhi urutan *batch* pada saat optimisasi. Sedangkan data validasi, tes, dan cadangan sampel tidak menggunakan *shuffling* agar evaluasi dan *output* prediksi tetap dalam urutan kronologis.

4.2 Implementasi Proses Model Informer

Setelah data di normalisasi dan di transformasi menjadi sampel *sliding window*, thapa selanjutnya adalah implementasi model informer. Implementasi model menggunakan *Pytorch* dan melakukan peramalan pada log return dari EUR/USD. Log

return yang diprediksi selanjutnya dikonversi kembali menjadi harga close pada tahap evaluasi, jadi output dari model dapat mengintegrasikan pergerakan harga dari EUR/USD.

Model Informer dipilih karena diciptakan untuk peramalan dengan *sequence* panjang. Dalam pengujian ini model menangani *input sequence* hingga 384 jam *timesteps*. Karena itu, arsitektur berbasis *attention* dibutuhkan agar model dapat menangkap dependensi jangka panjang dengan lebih efisien. Implementasi arsitektur informer terdiri dari *embedding layer*, *ProbSparse self-attention*, *encoder*, *decoder*, *distilling*, dan *linear projection layer* seperti yang telah dijelaskan pada **BAB II Subbab 2.4**.

Pengujian menggunakan 1 target bernama `log_return_to_close`. Yang berarti bahwa jika model dilatih untuk melakukan prediksi pada log return yang dinormalisasi, dan hasil dari prediksi dikonversi kembali menjadi harga *close* setelahnya. Fitur *input* terdiri dari 8 variabel historis, sedangkan target pada *decoder* adalah *log return*.

4.2.1 Konfigurasi Arsitektur Model Informer

Konfigurasi arsitektur model yang digunakan pada pengujian ini sama di setiap skenarionya. Perbedaan di antara setiap skenario hanya pada panjang *input sequence* dan outputnya. Pendekatan ini dilakukan agar perbedaan panjang *window* historis dan *forecasting horizons* dapat dibandingkan secara adil.

Tabel 4. 7 Konfigurasi Arsitektur Model Informer

Parameter	Nilai
Dimensi <i>input encoder</i> (<code>enc_in</code>)	8
Dimensi <i>input decoder</i> (<code>dec_in</code>)	1
Dimensi output	64
Banyak attention head	4
<i>Encoder Layer</i> (<code>e_layer</code>)	2
<i>Decoder Layer</i> (<code>d_layer</code>)	1
Dimensi Feed-forward (<code>d_ff</code>)	256
Sampling factor (<code>factor</code>)	5
Dropout	0.01

Berdasarkan **Tabel 4. 7**, model informer memiliki arsitektur yang kecil (*compact*) dengan menggunakan dimensi model (d_model) 64 dan dengan 4 attention head (n_head). Konfigurasi ini digunakan agar model mempelajari pola sementara dan biaya (*cost*) komputasi pelatihan setiap skenarionya seimbang. Karena pengujian memiliki 12 skenario, model yang *compact* dibutuhkan agar setiap kombinasi *input* dan output *sequence* dapat dilatih secara konsisten.

Encoder menerima 8 fitur *input* yaitu Open, High, Low, Close, Volume, Log return, RSI(14), dan ATR(14). Fitur tersebut merepresentasikan harga historis, volume trading, pergerakan return, momentum, dan volatilitas pada pasar EUR/USD. *Decoder* akan menerima target *sequence* berdasarkan log return yang telah dinormalisasi. Pada pelatihan, *decoder* akan menerima *input* yang terbagi menjadi 2 bagian yaitu, label *sequence* yang diketahui dan posisi *zero-filled* masa depan. Struktur ini memberikan model kemampuan untuk mempelajari pemetaan (*mapping*) di antara informasi target historis dan nilai target dari masa depan.

Pada tahap *embedding* fitur *input* dikonversi menjadi sesuai dengan dimensi model. Implementasi *embedding* meliputi *token embedding*, *positional embedding*, dan *temporal embedding*. Pada *token embedding* nilai *input* numerik ditransformasi menjadi *latent vector*. *Positional embedding* menambahkan informasi urutan *sequence*. Lalu *temporal embedding* menambahkan informasi terkait *timestamp*, yang berisikan Bulan, Hari, dan Jam. Karena data EUR/USD yang digunakan berada pada interval waktu 1 jam, penanda jam menjadi sangat penting untuk menjaga pola *intraday* sementara.

```
class ProbAttention(nn.Module):
    """Attention ProbSparse yang digunakan pada Informer."""
    def __init__(self, mask_flag=True, factor=5, scale=None,
attention_dropout=0.1, output_attention=False):
        super().__init__()
        self.factor = factor
        self.scale = scale
        self.mask_flag = mask_flag
        self.output_attention = output_attention
        self.dropout = nn.Dropout(attention_dropout)

    def _prob_QK(self, Q, K, sample_k, n_top):
        # Q, K: [B, H, L, D]
        B, H, L_K, E = K.shape
        _, _, L_Q, _ = Q.shape

        sample_k = min(sample_k, L_K)
        n_top = min(n_top, L_Q)
```

```

        # Mengambil sampel key secara acak untuk setiap query.
        index_sample = torch.randint(L_K, (L_Q, sample_k), device=Q.device)
        K_sample = K[:, :, index_sample, :] # [B, H, L_Q, sample_k, E]
        Q_K_sample = torch.matmul(Q.unsqueeze(-2), K_sample.transpose(-2,
-1)).squeeze(-2)

        # Pengukuran sparsity: nilai maksimum dikurangi rata-rata.
        M = Q_K_sample.max(dim=-1)[0] - Q_K_sample.mean(dim=-1)
        M_top = M.topk(n_top, sorted=False)[1] # [B, H, n_top]

        # Menggunakan top queries terpilih untuk menghitung QK secara penuh.
        Q_reduce = Q.gather(dim=2, index=M_top.unsqueeze(-1).expand(-1, -
1, -1, E))
        Q_K = torch.matmul(Q_reduce, K.transpose(-2, -1))
        return Q_K, M_top

    def _get_initial_context(self, V, L_Q):
        B, H, L_V, D = V.shape
        if not self.mask_flag:
            V_sum = V.mean(dim=-2)
            context = V_sum.unsqueeze(-2).expand(B, H, L_Q, D).clone()
        else:
            # Pada self-attention decoder, L_Q == L_V.
            assert L_Q == L_V, "Causal ProbAttention requires L_Q == L_V"
            context = V.cumsum(dim=-2)
        return context

    def _update_context(self, context_in, V, scores, index, L_Q,
attn_mask):
        B, H, L_V, D = V.shape

        if self.mask_flag:
            if attn_mask is None:
                attn_mask = ProbMask(B, H, L_Q, index, scores,
device=V.device)
                scores = scores.masked_fill(attn_mask.mask, -np.inf)

            attn = torch.softmax(scores, dim=-1)
            attn = self.dropout(attn)
            context_update = torch.matmul(attn, V)
            context_in.scatter_(dim=2, index=index.unsqueeze(-1).expand(-1, -
1, -1, D), src=context_update)

            if self.output_attention:
                attns = torch.ones((B, H, L_Q, L_V), device=attn.device) / L_V
                attns.scatter_(dim=2, index=index.unsqueeze(-1).expand(-1, -1,
-1, L_V), src=attn)
                return context_in, attns
            return context_in, None

    def forward(self, queries, keys, values, attn_mask=None):
        # Bentuk input: [B, L, H, D]
        B, L_Q, H, D = queries.shape
        _, L_K, _, _ = keys.shape

        Q = queries.transpose(2, 1) # [B, H, L_Q, D]
        K = keys.transpose(2, 1) # [B, H, L_K, D]
        V = values.transpose(2, 1) # [B, H, L_K, D]

        U_part = self.factor * math.ceil(math.log(L_K + 1))

```

```

        u = self.factor * math.ceil(math.log(L_Q + 1))
        U_part = min(U_part, L_K)
        u = min(u, L_Q)

        scores_top, index = self._prob_QK(Q, K, sample_k=U_part, n_top=u)
        scale = self.scale or 1.0 / math.sqrt(D)
        scores_top = scores_top * scale

        context = self._get_initial_context(V, L_Q)
        context, attn = self._update_context(context, V, scores_top, index,
        L_Q, attn_mask)

        return context.transpose(2, 1).contiguous(), attn

class AttentionLayer(nn.Module):
    def __init__(self, attention, d_model, n_heads, d_keys=None,
d_values=None, mix=False):
        super().__init__()
        d_keys = d_keys or (d_model // n_heads)
        d_values = d_values or (d_model // n_heads)

        self.inner_attention = attention
        self.query_projection = nn.Linear(d_model, d_keys * n_heads)
        self.key_projection = nn.Linear(d_model, d_keys * n_heads)
        self.value_projection = nn.Linear(d_model, d_values * n_heads)
        self.out_projection = nn.Linear(d_values * n_heads, d_model)
        self.n_heads = n_heads
        self.mix = mix

    def forward(self, queries, keys, values, attn_mask=None):
        B, L, _ = queries.shape
        _, S, _ = keys.shape
        H = self.n_heads

        queries = self.query_projection(queries).view(B, L, H, -1)
        keys = self.key_projection(keys).view(B, S, H, -1)
        values = self.value_projection(values).view(B, S, H, -1)

        out, attn = self.inner_attention(queries, keys, values, attn_mask)
        if self.mix:
            out = out.transpose(2, 1).contiguous()
        out = out.view(B, L, -1)
        return self.out_projection(out), attn

```

Gambar 4. 8 Implementasi ProbSparse Self-Attention dan Attention Layer

Dapat dilihat pada **Gambar 4.8**, setelah tahap *embedding*, *encoder* memproses *sequence* historis menggunakan *Probsparse self-attention*. Mekanisme *Probsparse attention* memilih interaksi *query* dan *key* paling informatif atau aktif alih-alih menghitung penuh *attention* untuk setiap posisinya. Ini penting karena dengan pengujian panjang *input sequence* yang mencapai 384 *timestep*. Dengan mekanisme *Probsparse attention* model dapat mengurangi komputasi *attention* yang tidak terlalu penting dengan tetap mempertahankan dependensi *sequence* panjang yang penting.

```

class ConvLayer(nn.Module):
    def __init__(self, c_in):
        super().__init__()
        self.downConv = nn.Conv1d(
            in_channels=c_in,
            out_channels=c_in,
            kernel_size=3,
            padding=1,
            padding_mode="circular",
        )
        self.norm = nn.BatchNorm1d(c_in)
        self.activation = nn.ELU()
        self.maxPool = nn.MaxPool1d(kernel_size=3, stride=2, padding=1)

    def forward(self, x):
        x = self.downConv(x.permute(0, 2, 1))
        x = self.norm(x)
        x = self.activation(x)
        x = self.maxPool(x)
        x = x.transpose(1, 2)
        return x

class EncoderLayer(nn.Module):
    def __init__(self, attention, d_model, d_ff=None, dropout=0.1,
activation="relu"):
        super().__init__()
        d_ff = d_ff or 4 * d_model
        self.attention = attention
        self.conv1 = nn.Conv1d(in_channels=d_model, out_channels=d_ff,
kernel_size=1)
        self.conv2 = nn.Conv1d(in_channels=d_ff, out_channels=d_model,
kernel_size=1)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
        self.activation = F.relu if activation == "relu" else F.gelu

    def forward(self, x, attn_mask=None):
        new_x, attn = self.attention(x, x, x, attn_mask=attn_mask)
        x = x + self.dropout(new_x)

        y = self.norm1(x)
        y = self.dropout(self.activation(self.conv1(y.transpose(-1, 1))))
        y = self.dropout(self.conv2(y).transpose(-1, 1))
        return self.norm2(x + y), attn

class Encoder(nn.Module):
    def __init__(self, attn_layers, conv_layers=None, norm_layer=None):
        super().__init__()
        self.attn_layers = nn.ModuleList(attn_layers)
        self.conv_layers = nn.ModuleList(conv_layers) if conv_layers is
not None else None
        self.norm = norm_layer

    def forward(self, x, attn_mask=None):
        attns = []
        if self.conv_layers is not None:

```

```

        for attn_layer, conv_layer in zip(self.attn_layers[:-1],
self.conv_layers):
            x, attn = attn_layer(x, attn_mask=attn_mask)
            x = conv_layer(x)
            attns.append(attn)
            x, attn = self.attn_layers[-1](x, attn_mask=attn_mask)
            attns.append(attn)
        else:
            for attn_layer in self.attn_layers:
                x, attn = attn_layer(x, attn_mask=attn_mask)
                attns.append(attn)

        if self.norm is not None:
            x = self.norm(x)
        return x, attns

class DecoderLayer(nn.Module):
    def __init__(self, self_attention, cross_attention, d_model,
d_ff=None, dropout=0.1, activation="relu"):
        super().__init__()
        d_ff = d_ff or 4 * d_model
        self.self_attention = self_attention
        self.cross_attention = cross_attention
        self.conv1 = nn.Conv1d(in_channels=d_model, out_channels=d_ff,
kernel_size=1)
        self.conv2 = nn.Conv1d(in_channels=d_ff, out_channels=d_model,
kernel_size=1)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
        self.activation = F.relu if activation == "relu" else F.gelu

    def forward(self, x, cross, x_mask=None, cross_mask=None):
        x = x + self.dropout(self.self_attention(x, x, x,
attn_mask=x_mask)[0])
        x = self.norm1(x)

        x = x + self.dropout(self.cross_attention(x, cross, cross,
attn_mask=cross_mask)[0])
        y = self.norm2(x)
        y = self.dropout(self.activation(self.conv1(y.transpose(-1, 1))))
        y = self.dropout(self.conv2(y).transpose(-1, 1))
        return self.norm3(x + y)

class Decoder(nn.Module):
    def __init__(self, layers, norm_layer=None, projection=None):
        super().__init__()
        self.layers = nn.ModuleList(layers)
        self.norm = norm_layer
        self.projection = projection

    def forward(self, x, cross, x_mask=None, cross_mask=None):
        for layer in self.layers:
            x = layer(x, cross, x_mask=x_mask, cross_mask=cross_mask)

        if self.norm is not None:
            x = self.norm(x)

        if self.projection is not None:

```

```

        x = self.projection(x)
    return x

```

Gambar 4. 9 Kode Program Implementasi Encoder, Decoder, dan Distilling

Pada **Gambar 4. 9**, *encoder* juga menggunakan *distilling layer* melalui *convolutional layer* yang diikuti dengan *maxpooling*. Lapisan ini mengurangi panjang *sequence* di dalam *encoder*. Tujuan dari tahap ini adalah untuk membuat model tetap efisien dalam melakukan proses dengan *input sequence* yang sangat panjang. Pada implementasi arsitektur ini, blok *encoder* terdiri dari 2 *encoder layer* dan *distilling layer* di antara keduanya. Pada blok *decoder*, blok menggunakan *masked Probsparse self-attention* dan *cross-attention*. *Masked self-attention* digunakan dengan bertujuan untuk mencegah *decoder* untuk melihat nilai target masa depan selama pelatihan model. Sedangkan *cross-attention* menghubungkan representasi dari *decoder* dengan *output* yang dihasilkan dari blok *encoder*, sehingga memberikan informasi kepada *decoder* terkait *input sequence* historis ketika melakukan prediksi masa depan.

```

class Informer(nn.Module):
    def __init__(
        self,
        enc_in,
        dec_in,
        c_out,
        seq_len,
        label_len,
        out_len,
        factor=5,
        d_model=512,
        n_heads=8,
        e_layers=2,
        d_layers=1,
        d_ff=2048,
        dropout=0.0,
        attn="prob",
        embed="fixed",
        activation="gelu",
        output_attention=False,
        distil=True,
        mix=True,
        use_time_features=True,
    ):
        super().__init__()
        self.pred_len = out_len
        self.attn = attn
        self.output_attention = output_attention

        Attn = ProbAttention if attn == "prob" else FullAttention

```

```

        self.enc_embedding = DataEmbedding(enc_in, d_model, embed,
dropout, use_time_features=use_time_features)
        self.dec_embedding = DataEmbedding(dec_in, d_model, embed,
dropout, use_time_features=use_time_features)

        self.encoder = Encoder(
            [
                EncoderLayer(
                    AttentionLayer(
                        Attn(False, factor, attention_dropout=dropout,
output_attention=output_attention),
                        d_model,
                        n_heads,
                        mix=False,
                    ),
                    d_model,
                    d_ff,
                    dropout=dropout,
                    activation=activation,
                )
                for _ in range(e_layers)
            ],
            [ConvLayer(d_model) for _ in range(e_layers - 1)] if distil
else None,
            norm_layer=nn.LayerNorm(d_model),
        )

        self.decoder = Decoder(
            [
                DecoderLayer(
                    AttentionLayer(
                        Attn(True, factor, attention_dropout=dropout,
output_attention=False),
                        d_model,
                        n_heads,
                        mix=mix,
                    ),
                    AttentionLayer(
                        FullAttention(False, factor,
attention_dropout=dropout, output_attention=False),
                        d_model,
                        n_heads,
                        mix=False,
                    ),
                    d_model,
                    d_ff,
                    dropout=dropout,
                    activation=activation,
                )
                for _ in range(d_layers)
            ],
            norm_layer=nn.LayerNorm(d_model),
            projection=nn.Linear(d_model, c_out, bias=True),
        )

        def forward(self, x_enc, x_mark_enc, x_dec, x_mark_dec,
enc_self_mask=None, dec_self_mask=None, dec_enc_mask=None):
            enc_out = self.enc_embedding(x_enc, x_mark_enc)
            enc_out, attns = self.encoder(enc_out, attn_mask=enc_self_mask)

```



```

dec_out = self.dec_embedding(x_dec, x_mark_dec)
dec_out = self.decoder(dec_out, enc_out, x_mask=dec_self_mask,
cross_mask=dec_enc_mask)

if self.output_attention:
    return dec_out[:, -self.pred_len:, :], attns
return dec_out[:, -self.pred_len:, :]

```

Gambar 4. 10 Kode Program Implementasi Arsitektur Model Informer

Dan tahap terakhir adalah *linear projection layer*, di mana pada lapisan akhir ini melakukan tranformasi dimensi model dari output *decoder* menjadi dimensi *output* target. Karena target hanya memiliki 1 variabel yaitu *log return*, maka dimensi akhirnya adalah 1. Lalu output dari model dibandingkan nilai aktual dari *log return* yang telah dinormalisasi menggunakan *loss function mean squared error* selama pelatihan.

4.2.2 Konfigurasi Pelatihan Model dan Proses Pelatihan Model

Setelah arsitektur informer diimplementasikan, tahap selanjutnya adalah konfigurasi pelatihan dan melakukan pelatihan pada model. Proses pelatihan dilakukan secara langsung terhadap seluruh skenario. Karena pada pengujian ini memakai 4 panjang *input sequence* dan 3 panjang output *sequence* maka total pelatihan adalah 12. Setiap skenario menggunakan arsitektur dan konfigurasi yang sama. Perbedaan hanya terdapat pada nilai *seq_len*, *label_len*, dan *pred_len*. Sedangkan *label_len* didapatkan dari setengah nilai *seq_len*.

Tabel 4. 8 Konfigurasi Hyperparameter Pelatihan

Parameter	Nilai	Deskripsi
Batch Size	32	Banyaknya sampel diproses dalam 1 iterasi pelatihan
Maximum Epoch	50	Nilai maksimum dari banyaknya epoch dalam pelatihan
Optimizer	Adam	Algoritma optimisasi untuk pelatihan
Learning Rate	0.0002	Besar langkah yang digunakan Optimizer
Loss Function	Mean Squared Error	Loss function pelatihan
Early Stopping Patience	5	Banyak epoch yang diperbolehkan tanpa perkembangan di validasinya
Minimum Delta	0.000001	Besaran minimal perkembangan validation loss
Random Seed	42	Untuk mendukung reproduktifitas
Restore Best Weight	True	Untuk mengembalikan weight model dari epoch terbaik

Berdasarkan **Tabel 4. 8**, proses pelatihan menggunakan MSE sebagai *loss function*. *Loss function* ini dipilih karena model dilatih sebagai model regresi untuk melakukan prediksi nilai log return masa depan. *Adam optimizer* digunakan karena dapat beradaptasi pada *learning rate* untuk setiap parameter pada saat pelatihan dan juga cukup sering digunakan pada model deep learning.

```
# Hyperparameter arsitektur Informer.
D_MODEL = 64
N_HEADS = 4
E_LAYERS = 2
D_LAYERS = 1
D_FF = 256
FACTOR = 5
DROPOUT = 0.01
ATTN = "prob"
EMBED = "fixed"
ACTIVATION = "gelu"
DISTIL = True
OUTPUT_ATTENTION = False
```

Gambar 4. 11 Kode Program Konfigurasi Hyperparameter Pelatihan

Mekanisme *early stopping* digunakan untuk mengurangi risiko *overfitting*. Pada masa pelatihan model, *loss* pada validasi terus dipantau hingga akhir pada setiap *epoch*-nya. Jika *loss* pada validasi tidak meningkat setidaknya sebesar 0.000001 sebanyak 5 *epoch* berturut, proses pelatihan dihentikan sebelum mencapai *epoch* maksimal. Setelah *early stopping* dipicu, model mengembalikan (*restored*) *weight* dari *epoch* dengan nilai *loss* pada validasi terendah. Implementasi kode program mekanisme *earlystopping* dapat dilihat pada **Gambar 4. 12** di bawah.

```
class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.0, checkpoint_path=None):
        self.patience = int(patience)
        self.min_delta = float(min_delta)
        self.checkpoint_path = checkpoint_path
        self.best_loss = np.inf
        self.best_epoch = 0
        self.counter = 0
        self.best_state = None

    def step(self, val_loss, model, epoch):
        if val_loss < self.best_loss - self.min_delta:
            self.best_loss = float(val_loss)
            self.best_epoch = int(epoch)
            self.counter = 0
            self.best_state = copy.deepcopy(model.state_dict())
            if self.checkpoint_path:
                os.makedirs(os.path.dirname(self.checkpoint_path),
                    exist_ok=True)
```

```

        torch.save({"epoch": self.best_epoch, "best_val_loss":
self.best_loss, "model_state_dict": self.best_state},
self.checkpoint_path)
        return False
        self.counter += 1
        return self.counter >= self.patience

    def restore(self, model):
        if self.best_state is not None:
            model.load_state_dict(self.best_state)
            return True
        if self.checkpoint_path and os.path.exists(self.checkpoint_path):
            checkpoint = torch.load(self.checkpoint_path,
map_location=DEVICE)
            model.load_state_dict(checkpoint["model_state_dict"])
            return True
        return False

```

Gambar 4. 12 Kode Program Implementasi Early Stopping

Terlihat pada Gambar 4. 13, proses pelatihan terjadi berulang hingga seluruh skenario mendapatkan epoch terbaik. Pada setiap skenario, *dataloader* dibangun berdasarkan *seq_len*, *label_len*, dan *pred_len* yang dipilih. Model di inisialisasi, dilatih, divalidasi, dan dievaluasi. Setiap *weight* terbaik di setiap skenario disimpan sebagai *file checkpoint*, sedangkan historis pelatihan, metrik evaluasi, dan hasil prediksi sampel disimpan dengan format *csv*.

```

def run_epoch(model, loader, optimizer, criterion, cfg: ExperimentConfig,
train=True):
    model.train(train)
    losses = []
    iterator = enumerate(loader)
    if train:
        iterator = tqdm(iterator, total=len(loader), leave=False)

    with torch.set_grad_enabled(train):
        for batch_i, batch in iterator:
            if train and MAX_TRAIN_BATCHES is not None and batch_i >=
MAX_TRAIN_BATCHES:
                break
            if (not train) and MAX_EVAL_BATCHES is not None and batch_i
>= MAX_EVAL_BATCHES:
                break

            batch_x, batch_y, batch_x_mark, batch_y_mark, *_ =
move_batch(batch)
            dec_zeros = torch.zeros_like(batch_y[:, -cfg.pred_len:, :])
            dec_inp = torch.cat([batch_y[:, :cfg.label_len, :], dec_zeros],
dim=1)

            if train:
                optimizer.zero_grad(set_to_none=True)

            outputs = model(batch_x, batch_x_mark, dec_inp, batch_y_mark)

```

```

        target = batch_y[:, -cfg.pred_len:, :]
        loss = criterion(outputs, target)

        if train:
            loss.backward()
            optimizer.step()

        losses.append(loss.item())

    return float(np.mean(losses)) if losses else np.nan

for target_mode in TARGET_MODES:
    for seq_len in SEQ_LENS:
        for pred_len in PRED_LENS:
            scenario_no += 1
            cfg = ExperimentConfig(target_mode=target_mode,
seq_len=seq_len, pred_len=pred_len, label_len=seq_len // 2)
            print(f"\nSkenario {scenario_no}/{total}:
{scenario_label(cfg)}")
            try:
                model, hist, row, test_pred, sample_pred, meta =
run_experiment(cfg, verbose=True)
                row["error"] = ""
                all_results.append(row)
                all_loss_history.append(hist)
                all_sample_rows.extend(make_sample_rows(sample_pred,
cfg, meta))
            except Exception as e:
                if isinstance(e, RuntimeError) and "out of memory" in
str(e).lower():
                    torch.cuda.empty_cache()
                all_results.append({
                    "scenario_label": scenario_label(cfg),
                    "target_mode": target_mode,
                    "seq_len": seq_len,
                    "label_len": seq_len // 2,
                    "pred_len": pred_len,
                    "error": repr(e),
                })
                print("ERROR:", repr(e))

            results_df = pd.DataFrame(all_results)
            results_df.to_csv(RESULTS_CSV_PATH, index=False)
            if all_loss_history:
                loss_history_df = pd.concat(all_loss_history,
ignore_index=True)
                loss_history_df.to_csv(LOSS_HISTORY_CSV_PATH,
index=False)
            if all_sample_rows:
                sample_predictions_df = pd.DataFrame(all_sample_rows)
                sample_predictions_df.to_csv(PREDICTIONS_CSV_PATH,
index=False)

print("Hasil tersimpan:", RESULTS_CSV_PATH)
print("Riwayat loss tersimpan:", LOSS_HISTORY_CSV_PATH)
print("Prediksi sampel tersimpan:", PREDICTIONS_CSV_PATH)
display(pd.DataFrame(all_results))

```

Gambar 4. 13 Kode Program Implementasi Skenario Pelatihan

Perulangan pelatihan terdiri dari 3 tahap utama, pertama model melakukan *forward propagation* dengan memproses *input encoder*, *input decoder*, dan fitur *time marker*. Kedua, prediksi *log return* yang ternormalisasi dibandingkan dengan *log return* ternormalisasi aktual menggunakan *loss* MSE. Ketiga, *backpropagation* dilakukan untuk memperbarui parameter dari model. Proses ini diulang hingga mencapai *epoch* maksimum atau *early stopping* dipicu.

Tabel 4. 9 Ringkasan Pelatihan Setiap Skenario

No.	Skenario	Panjang <i>input</i>	Panjang output	Epoch terbaik	Epoch berjalan	Loss terbaik
1	LR → Close S48 P1	48	1	6	11	0.968582
2	LR → Close S48 P4	48	4	3	8	0.969071
3	LR → Close S48 P8	48	8	1	6	0.969293
4	LR → Close S96 P1	96	1	7	12	0.968569
5	LR → Close S96 P4	96	4	2	7	0.969598
6	LR → Close S96 P8	96	8	1	6	0.969443
7	LR → Close S192 P1	192	1	1	6	0.968818
8	LR → Close S192 P4	192	4	1	6	0.969027
9	LR → Close S192 P8	192	8	2	7	0.970218
10	LR → Close S384 P1	384	1	11	16	0.968644
11	LR → Close S384 P4	384	4	1	6	0.969434
12	LR → Close S384 P8	384	8	2	7	0.969737

Berdasarkan **Tabel 4.9**, seluruh 12 skenario berhasil dilakukan pelatihan. Banyak *epoch* dari setiap skenario lebih rendah dari nilai maksimal *epoch* karena mekanisme *early stopping* dipicu. Hasil ini menunjukkan jika loss pada validasi mencapai terbaik

sebelum menyentuh nilai maksimal pelatihan. Skenario dengan durasi terpanjang dalam hal epoch adalah skenario LR $\rightarrow$ Close S384 P1, yang di mana skenario tersebut selama 16 *epoch* dengan *loss* terbaik di *epoch* ke 11.

Loss validasi terbaik pada setiap skenario relatif cukup mirip dengan nilai berkisar di angka 0.968 hingga 0.970. Skenario dengan *loss* validasi terbaik didapatkan oleh skenario LR $\rightarrow$ Close S96 P1 dengan nilai 0.968569. Namun, nilai dari *loss* validasi sendiri tidak dapat menjadi kesimpulan final dari performa model karena nilai tersebut dihitung dari skala target yang dinormalisasi. Karena itu, perbandingan akhir juga dievaluasi menggunakan metrik harga *close* dengan MAE, MSE, dan MAPE pada data tes dan data sampel cadangan.

4.3 Hasil Pengujian

Setelah seluruh skenario pelatihan dilakukan, langkah selanjutnya adalah melakukan pengujian pada model informer yang dilatih. Pengujian mencakup mengukur bagaimana kinerja model dalam melakukan prediksi data EUR/USD masa depan pada data yang belum pernah dilihat (*unseen*). Walau model dilatih untuk melakukan prediksi pada *log return*, evaluasi akhir dilakukan dengan merekonstruksi ulang menjadi harga *close*. Ini dilakukan karena harga *close* lebih muda diinterpretasikan dibandingkan dengan *log return* yang ternormalisasi, khususnya dalam hal peramalan data *time-series* finansial.

Proses pengujian menggunakan 2 sumber evaluasi. Sumber pertama berasal dari data tes, yang menjadi bagian dari *modelling* dan tidak digunakan dalam pelatihan model. Data ini digunakan untuk mengevaluasi performa secara umum untuk setiap skenario pada data *unseen* historis. Sumber kedua berasal dari data cadangan yang telah dipisahkan pada **Subbab 4.1.3**, yang terdiri dari 491 baris data dan digunakan untuk menghasilkan 100 titik *rolling prediction* pada setiap skenario.

Tahap evaluasi ini dibuat untuk mengetahui apakah model informer dapat menghasilkan prediksi harga *close* yang mendekati data aktual harga *close* dari EUR/USD. Karena itu, evaluasi berfokus pada metrik regresi seperti MAE, MSE, dan MAPE. Ketiga metrik tersebut dihitung dengan data *log return* yang telah di rekonstruksi kembali menjadi harga *close*. Proses konversi terdiri dari 2 tahap utama. Tahap pertama

adalah melakukan normalisasi terbalik (*inverse*), di mana prediksi *log return* yang ternormalisasi di transformasi kembali menjadi skala log return asli menggunakan *target scaler*. Tahap kedua adalah melakukan rekonstruksi harga *close*, di mana prediksi *log return* dikonversi menjadi harga *close* menggunakan harga *close* diketahui sebelumnya sebagai basis (*base*) harga.

Rekonstruksi harga *close* dilakukan dengan:

$$\text{Prediksi Harga Close} = \text{Base Close} \times \exp(\text{Prediksi Log Return Kumulatif})$$

Untuk prediksi dengan panjang 1, prediksi *close* dihitung secara langsung dari basis harga *close* dan *log return* yang diprediksi. Untuk prediksi dengan panjang 4 dan 8, prediksi dari log return diakumulasikan terlebih dahulu. Ini penting karena log return bersifat aditif seiring waktunya. Karena itu, prediksi dengan panjang lebih dari 1 atau *multisteps*, prediksi harga *close* pada langkah terakhir dihitung dengan prediksi log return kumulatif dari tahap pertama hingga akhir.

Tabel 4. 10 Contoh Rekonstruksi Harga Close

Skenario	Tanggal	Prediksi ke-	Tahap ke-	Base Close	Log return aktual kumulatif	Log return prediksi kumulatif	Close Aktual	Close Prediksi	Error Absolut
LR → Close S96 P4	4/13/2026 13:00	1	1	1.16860998	0.00012832	0.00000122	1.16875994	1.168611	0.00014853
LR → Close S96 P4	4/13/2026 14:00	1	2	1.16860998	-0.00005988	0.00001336	1.16854	1.168626	0.00008559
LR → Close S96 P4	4/13/2026 15:00	1	3	1.16860998	0.00093234	0.00004478	1.16970003	1.168662	0.00103772
LR → Close S96 P4	4/13/2026 16:00	1	4	1.16860998	-0.00001714	0.00005743	1.16858995	1.168677	0.00008714

Dapat dilihat pada **Tabel 4. 10**, contoh dari rekonstruksi menggunakan skenario dengan panjang prediksi lebih dari 1 yaitu pada skenario LR → Close S96 P4. Pada skenario ini, model menggunakan 96 data historis sebagai *input* dan menghasilkan prediksi 4 jam ke depan. 4 baris pada tabel adalah urutan prediksi yang sama yaitu prediksi pertama. Nilai *base close* adalah 1.16860998 dan tetap sama untuk peramalan pada langkah pertama hingga keempat karena keempat prediksi dihasilkan dalam 1 *window* peramalan yang sama. Model tidak melakukan rekonstruksi setiap langkah secara independen dari harga *close* aktual sebelumnya. Tetapi, log return yang diprediksi

diakumulasikan dari langkah prediksi pertama hingga akhir. Setelah itu, prediksi log return yang telah diakumulasikan dikonversikan menjadi prediksi harga close menggunakan fungsi eksponensial.

Sebagai contoh, pada peramalan pertama, kumulatif dari prediksi log return adalah 0.00000122, menghasilkan prediksi harga close dengan nilai 1.16861141. Pada tahap langkah kedua, prediksi log return kumulatif meningkat menjadi 0.00001336, menghasilkan nilai close 1.16862559. Proses ini berlanjut hingga peramalan keempat, di mana prediksi log return kumulatif 0.00005743 dan hasil harga closenya menjadi 1.16867709. Nilai aktual harga close juga dibandingkan dengan nilai *base* close. Pada peramalan tahap ketiga, nilai aktual close meningkat ke 1.16970003, sedangkan prediksi harga close adalah 1.16866231. Ini menghasilkan error absolut terbesar, yaitu 0.00103772. Hasil ini menunjukkan jika prediksi dengan langkah lebih dari 1 (*Multi-step*), error dari model sangat bervariasi di setiap langkah prediksi, karena setiap titik masa depan memiliki pergerakan pasar aktual yang berbeda.

Contoh di atas menunjukkan ketika *pred_len* lebih dari 1, hasil dari prediksi harus diinterpretasikan sebagai nilai masa depan sekuensial yang dihasilkan dari *window* historis yang sama. Karena itu, proses rekonstruksi menggunakan log return kumulatif dibandingkan memproses setiap langkah prediksi berbeda.

Tabel 4. 11 Hasil Evaluasi Seluruh Skenario

No	Skenario	Test			Sampel Prediksi		
		MAE	MSE	MAPE	MAE	MSE	MAPE
1	LR→Close S48 P1	0.000654	1.06E-06	0.058935	0.000516	4.87E-07	0.043851
2	LR→Close S48 P4	0.001028	2.54E-06	0.092669	0.000761	1.15E-06	0.064609
3	LR→Close S48 P8	0.001401	4.58E-06	0.126267	0.001193	3.18E-06	0.101264
4	LR→Close S96 P1	0.000654	1.06E-06	0.058924	0.000516	4.84E-07	0.043848
5	LR→Close S96 P4	0.001028	2.54E-06	0.092676	0.000758	1.16E-06	0.064419
6	LR→Close S96 P8	0.001395	4.55E-06	0.125747	0.001178	3.14E-06	0.10003
7	LR→Close S192 P1	0.000656	1.06E-06	0.059088	0.000526	5.03E-07	0.044675
8	LR→Close S192 P4	0.001028	2.54E-06	0.092699	0.000781	1.23E-06	0.066369
9	LR→Close S192 P8	0.001396	4.56E-06	0.125848	0.001096	2.65E-06	0.093048
10	LR→Close S384 P1	0.000654	1.06E-06	0.058907	0.000514	4.82E-07	0.043712
11	LR→Close S384 P4	0.001031	2.55E-06	0.092909	0.000797	1.27E-06	0.067728
12	LR→Close S384 P8	0.001396	4.55E-06	0.125843	0.001164	3.08E-06	0.098837

Berdasarkan hasil pengujian yang dapat dilihat pada **Tabel 4. 11**, metrik MAPE tes dengan nilai terendah didapatkan oleh skenario LR → Close S384 P1 dengan nilai tes MAPE 0.058907%. Skenario ini menggunakan *input sequence* terpanjang yaitu 384 jam dan melakukan prediksi 1 jam ke depan. Skenario inipun mendapatkan nilai MAPE terendah pada prediksi di data cadangan dengan nilai 0.043712%. Ini menunjukkan dengan prediksi 1 jam ke depan, konteks historis yang panjang memberikan performa terbaik pada seluruh skenario yang diuji.

Untuk skenario dengan panjang prediksi 4 jam ke depan, performa prediksi pada data cadangan terbaik didapatkan oleh skenario LR → Close S96 P4 dengan nilai MAPE 0.064419%. Sedangkan nilai tes MAPE pada panjang prediksi 4 jam ke depan memiliki nilai yang cukup mirip di setiap panjang *sequence input* yang berbeda. Nilai MAPE pada tes di panjang prediksi tersebut berada pada rentang 0.092669% hingga 0.092909%, menunjukkan jika perbedaan panjang *sequence input* tidak menyebabkan perbedaan performa yang signifikan di panjang prediksi 4 jam ke depan.

Pada prediksi dengan panjang 8 jam ke depan, performa pada sampel cadangan terbaik dimiliki oleh skenario LR→Close S192 P8 dengan nilai MAPE 0.093048%. Namun, performaa pada data tes terbaik dimiliki pada skenario LR→Close S96 P8 dengan nilai MAPE 0.125747%. Perbedaan ini menunjukkan panjang *sequence input* terbaik dapat bervariasi bergantung pada apakah model dievaluasi menggunakan data tes atau data sampel cadangan. Hasil juga menunjukkan panjang prediksi memiliki dampak yang lebih dibandingkan panjang *sequence input*. Nilai error meningkat ketika panjang prediksi diperpanjang dari 1 ke 4 dan dari 4 ke 8. Pola ini terjadi karena semakin panjang prediksi semakin sulit juga model dalam melakukan peramalan. Ketika model melakukan prediksi lebih jauh, ketidakpastian semakin besar, dan error dari peramalan sebelumnya terakumulasi di tahap prediksi selanjutnya.

Tabel 4. 12 Rata-rata Hasil Evaluasi Berdasarkan Panjang Prediksi

Prediction Length	Rata-rata					
	Test			Sampel Prediksi		
	MAE	MSE	MAPE (%)	MAE	MSE	MAPE (%)
1	0.000654	1.06E-06	0.058964	0.000518	4.89E-07	0.044022
4	0.001029	2.55E-06	0.092738	0.000774	1.20E-06	0.065781
8	0.001397	4.56E-06	0.125926	0.001158	3.01E-06	0.098295

Berdasarkan **Tabel 4. 12**, nilai rata-rata pada tes MAPE meningkat dari 0.058964% pada panjang prediksi 1 jam ke 0.092738% di skenario dengan prediksi 4 jam ke depan, lalu meningkat juga pada prediksi 8 jam ke depan dengan nilai 0.125926%. Pola yang sama juga terjadi di data sampel cadangan dengan nilai 0.044022% pada prediksi 1 jam menjadi 0.065781% pada prediksi 4 jam ke depan, dan meningkat menjadi 0.098295% pada prediksi 8 jam ke depan.

Hasil tersebut menunjukkan bahwa model informer dapat menghasilkan prediksi yang akurat pada panjang prediksi yang pendek. Ketika panjang prediksi diperpanjang, model tetap dapat menghasilkan persentase error yang relatif cukup kecil, tetapi nilai error terjadi peningkatan. Oleh karena itu, panjang prediksi dapat dipertimbangkan sebagai faktor yang paling berperan pada pengujian ini.

Efek dari *sequence input* berperan lebih kecil dibandingkan peran dari panjang prediksi. Pada skenario dengan prediksi dengan panjang 1 jam, panjang *sequence input* dengan hasil terbaik adalah dengan panjang 384. Untuk prediksi dengan panjang 4 jam, hasil sampel terbaik didapatkan pada panjang *sequence input* 96. Lalu dengan panjang prediksi 8 jam, *sequence input* terbaik didapatkan oleh panjang 192. Ini menunjukkan jika menggunakan *window* historis yang panjang belum pasti memberikan performa yang lebih baik. *Sequence input* yang panjang dapat memberikan informasi historis yang lebih banyak, tetapi juga dapat memberikan model pola lama yang belum tentu relevan terhadap kondisi pasar.

Hasil dari tes menunjukkan bahwa model informer dapat melakukan peramalan pada harga close EUR/USD dengan nilai MAPE yang relatif kecil terhadap seluruh skenario. Dari seluruh skenario, skenario LR→Close S384 P1 menjadi yang terbaik, dengan menghasilkan nilai MAPE terendah pada data tes dan sample. Namun, untuk

panjang prediksi panjang *sequence input* terbaik berubah tergantung panjang prediksinya. Karena itu, pemilihan panjang *sequence input* harus disesuaikan dengan panjang prediksi yang digunakan.

4.4 Pembahasan

Bedasarkan hasil pengujian pada **Subbab 4.3**, model informer dapat melakukan peramalan pada harga close EUR/USD dengan error yang relatif kecil pada seluruh skenario yang diuji. Model diuji dengan 4 *sequence input* yang berbeda dan 3 panjang prediksi yang berbeda. Hasil menunjukkan performa peramalan dipengaruhi oleh kedua *hyperparameter* tersebut, tetapi performa model lebih dipengaruhi oleh panjang prediksi dibandingkan dengan *sequence input*.

Pola yang paling penting yang di observasi pada hasil evaluasi adalah error dari prediksi meningkat ketika panjang prediksi diperpanjang. Pola ini dapat dilihat pada **Tabel 4. 11**, di mana panjang prediksi 1 jam ke depan memiliki rata-rata MAPE pada tesnya sebesar 0.058964% dan 0.044022% pada rata-rata MAPE di data sampel cadangannya. Untuk panjang prediksi 4 jam ke depan, nilai rata-rata MAPE pada data tes meningkat menjadi 0.092738% dan 0.065781% pada data sampel cadangan. Lalu pada panjang prediksi 8 jam ke depan meningkatkan nilai MAPE pada tes menjadi 0.125926% dengan 0.098295% pada data sampel cadangan.

Hasil tersebut menunjukkan jika peramalan dengan *horizon* pendek lebih mudah dibandingkan dengan *horizon* panjang. Pada prediksi 1 jam ke depan, model cukup perlu melakukan estimasi pergerakan selanjutnya secara langsung setelah *input window* historis. Sebaliknya, pada prediksi 4 dan 8 jam ke depan model perlu melakukan estimasi *sequence* pergerakan masa depan yang lebih panjang. Karena target model *adalah log return*, *log return* yang diprediksi harus diakumulasikan terlebih dahulu sebelum dikonversikan menjadi harga *close*. Karena itu, ketika panjang prediksi diperpanjang, error dari prediksi sebelumnya dapat mempengaruhi proses rekonstruksi harga *close*.

Pada metrik evaluasi MAE, panjang prediksi 1 jam ke depan mendapatkan nilai MAE terendah dengan nilai berkisar 0.00065. Sedangkan pada panjang prediksi 8 jam ke depan, nilai MAE berkisar pada 0.001395 hingga 0.001401. Pada metrik MSE, nilai pada pengujian relatif kecil karena harga berada pada rentang 1.0. Contohnya di rata-rata nilai

MSE pada data tes nilai pada prediksi dengan panjang 1 jam ke depan adalah 1.059×10^{-6} yang meningkat menjadi 4.559×10^{-6} pada panjang prediksi 8 jam ke depan. Peningkatan ini tidak hanya menunjukkan rata-rata *error* yang lebih tinggi, tetapi juga nilai deviasi yang lebih besar pada beberapa titik prediksi.

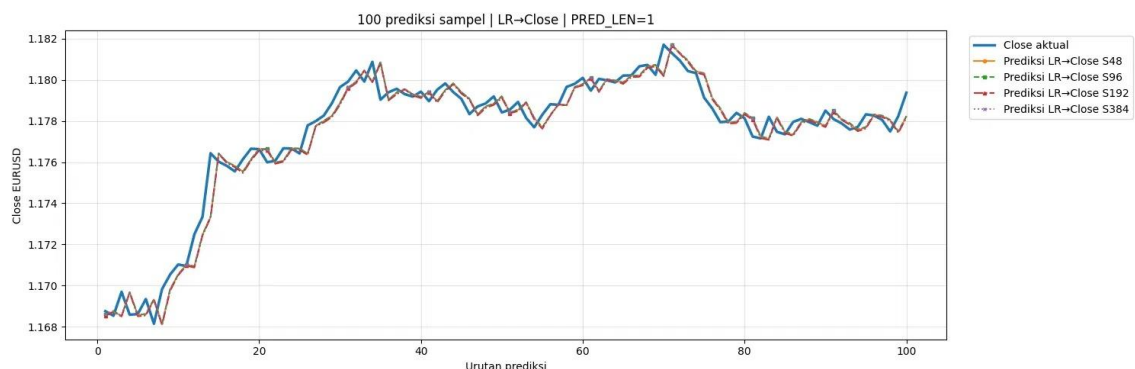
Pada penelitian ini di dapatkan nilai MAPE yang lebih rendah dibandingkan dengan penelitian Wang et al. (2021), dimana pada penelitian tersebut memiliki nilai MAPE sebesar 0.18945% dengan menggunakan model CNN-TLSTM untuk peramalan pada mata uang USD/CNY. Dibandingkan dengan penelitian ini, yang menggunakan skenario LR→Close S384 P1 didapatkan nilai MAPE sebesar 0.043712% pada sampel prediksi EUR/USD, nilai tersebut lebih rendah sebesar 76.9% dibandingkan dengan hasil dari Wang et al. (2021). Nilai MAPE yang lebih rendah bisa didapatkan dengan model Informer yang dapat menangkap konteks historis yang lebih panjang, lalu rendahnya nilai MAPE tersebut diduga karena karakteristik mata uang EUR/USD yang relatif lebih stabil dibandingkan dengan USD/CNY.

Hasil menunjukkan jika penggunaan *sequence input* yang lebih panjang tidak selalu memberikan hasil terbaik pada setiap panjang prediksi. Untuk panjang prediksi 4 jam ke depan, hasil pada data sampel cadangan terbaik didapatkan oleh skenario LR→Close S96 P4 dengan nilai MAPE sebesar 0.064419%. Untuk panjang prediksi 8 jam ke depan, dengan data sampel cadangan mendapatkan nilai MAPE terbaik sebesar 0.093048% pada skenario LR→Close S192 P8. Hasil tersebut berarti panjang *sequence input* paling optimal berubah bergantung pada panjang prediksi yang digunakan. Penggunaan *sequence input* yang panjang dapat memberikan informasi historis pada model lebih banyak, tetapi juga dapat memberikan model pola lama pasar yang tidak relevan pada kondisi market.

Berdasarkan **Tabel 4. 11**, panjang *sequence input* terbaik tidak sama pada setiap panjang prediksi yang berbeda. Pada panjang prediksi 1 jam ke depan, model dapat keuntungan dengan *sequence input* yang panjang yaitu 384 pada data tes dan data sampel cadangan. Lalu pada panjang prediksi 4 jam ke depan, *sequence input* terbaik adalah 48 pada data tes dan 96 pada data sampel cadangan. Sedangkan pada panjang prediksi 8 jam ke depan, pada data tes dengan *sequence input* 96 menjadi yang terbaik dan panjang 192 menjadi yang terbaik untuk data sampel cadangan. Hasil tersebut dapat dibandingkan

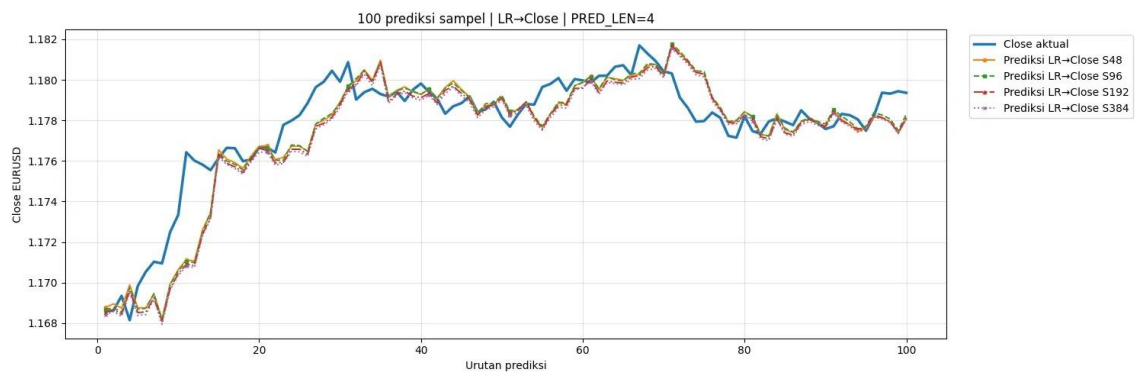
dengan dengan penelitian Ghahremani & Nguyen (2025), yang mengembangkan model ALFA dan menunjukkan pentingnya identifikasi informasi yang relevan untuk peramalan mata uang asing. Penelitian ini menunjukkan hubungan panjang *sequence input* dan panjang prediksi tidak linear. Meningkatkan panjang *input* tidak berarti meningkatkan performa model.

Kemungkinan pola ini terjadi karena perubahan alami dari pasar EUR/USD. Pada data *time-series* finansial, pola yang terjadi sekarang terkadang lebih berguna dibandingkan pola lama yang terjadi di pasar. *Sequence input* yang pendek dapat membuat model berfokus pada kondisi pasar yang sedang terjadi, meskipun penggunaan *sequence input* yang panjang dapat memberikan konteks historis yang lebih luas. Jika kondisi pasar relatif stabil, *sequence input* yang lebih panjang dapat membuat model lebih memahami pergerakan umum yang terjadi di pasar. Namun jika kondisi pasar berubah, informasi lampau dapat menjadi sedikit berguna atau bahkan menurunkan akurasi prediksi. Karena itu, panjang *sequence input* terbaik bergantung pada seberapa banyak konteks historis yang relevan pada panjang prediksi yang dipilih.



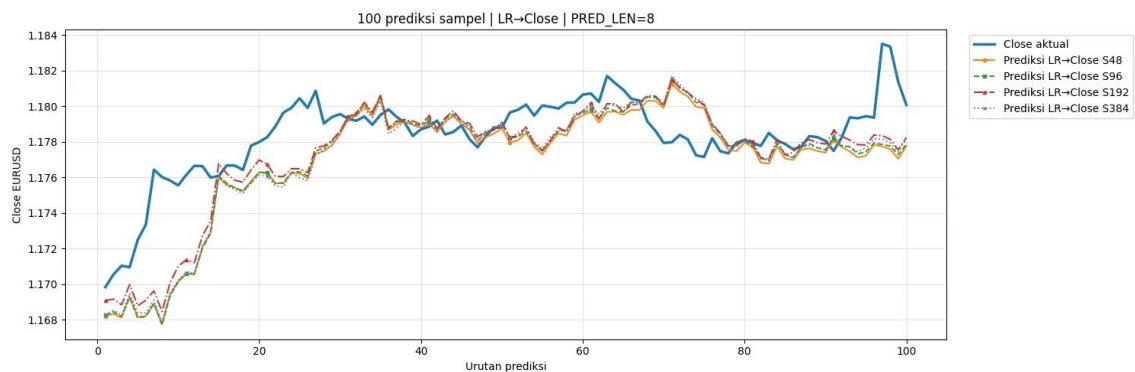
Gambar 4. 14 Seratus Prediksi Sampel Dengan Panjang Prediksi 1 Jam

Dapat dilihat pada **Gambar 4. 14**, dengan menggunakan data sampel sebagai pengujian dan panjang prediksi 1 jam ke depan menunjukkan jika prediksi mendekati pergerakan dari harga aktual selama 100 titik prediksi. Pada kondisi pasar naik dan turun, nilai prediksi berhasil menangkap *trend* dari pasar dengan tetap mempertahankan perbedaan yang kecil dari observasi nilai aktual. Perbedaan di antara keempat panjang *sequence* yang berbeda cukup minim, menunjukkan jika model informer dapat memanfaatkan informasi historis secara efektif dalam melakukan peramalan jangka pendek tanpa memedulikan panjang *input* dari *encoder*.



Gambar 4. 15 Seratus Prediksi Sampel Dengan Panjang Prediksi 4 Jam

Lalu pada **Gambar 4. 15**, dengan panjang prediksi 4 jam ke depan kurva prediksi tetap mengikuti *trend* pasar secara keseluruhan tetapi mulai menunjukkan deviasi yang lebih besar dibandingkan harga aktualnya. Deviasi tersebut menjadi lebih mudah terlihat ketika pasar berada di periode naik dan turun dengan cepat, di mana nilai prediksi merespons dengan lebih bertahap dibandingkan dengan nilai aktual pasar. Pola ini mencerminkan meningkatnya ketidakpastian terkait prediksi lebih jauh ke depan. Walau error dari prediksi meningkat, konfigurasi dengan panjang prediksi 4 jam tetap menghasilkan kemiripan pada kurva prediksi yang menunjukkan jika pemilihan panjang *sequence input* memiliki pengaruh yang relatif kecil dibandingkan panjang *output* yang digunakan. *Trend* harga secara keseluruhan tetap terjaga dengan baik selama periode prediksi, yang membuktikan jika model informer berhasil menangkap dependensi jangka panjang sementara bahkan dengan prediksi 4 jam ke depan.



Gambar 4. 16 Seratus Prediksi Sampel Dengan Panjang Prediksi 8 Jam

Sementara dapat dilihat pada **Gambar 4. 16**, dengan pengujian yang sama tetapi dengan panjang prediksi 8 jam ke depan menunjukkan ketidaksesuaian prediksi dan nilai

aktual yang lebih besar, khususnya mendekati bagian urutan terakhir dari prediksi. Meski kurva prediksi masih mengikuti arah pergerakan pasar, tetapi prediksi cenderung semakin menghaluskan pergerakan dan gagal menangkap perubahan harga yang tajam pada harga aktual dari data EUR/USD. Pola ini terjadi karena tingkat akurasi dari prediksi semakin menurun seiring dengan semakin panjangnya panjang prediksi yang digunakan.

Nilai validasi *loss* pada **Tabel 4. 9**, relatif mirip pada seluruh skenario yang diuji, berkisar pada rentang 0.968 hingga 0.970. Namun hasil akhir nilai MAPE pada data tes dan sampel cadangan menunjukkan perbedaan yang signifikan pada panjang prediksi yang berbeda. Ini terjadi karena *loss* pada validasi dihitung menggunakan log return yang dinormalisasi sebagai target model, sedangkan evaluasi akhir dihitung menggunakan harga *close* yang direkonstruksi. Karena itu *loss* validasi berguna untuk memantau proses pelatihan dan *earlystopping*, tetapi belum cukup untuk menentukan skenario yang terbaik.

Hasil dari prediksi dari sampel cadangan memiliki nilai yang lebih rendah dibandingkan pada data test. Contohnya pada skenario LR→Close S384 P1 menghasilkan nilai MAPE 0.058907% pada data tes dan 0.043712% pada data sampel cadangan. Pola yang sama juga dapat ditemukan di skenario yang lain. Perbedaan ini dapat terjadi karena periode pada sampel cadangan lebih pendek dibandingkan data tes dan hanya terdapat 100 *rolling* prediksi pada setiap skenarionya. Data tes mencakup periode yang lebih panjang sehingga mendapatkan kondisi market yang lebih beragam. Sedangkan sampel cadangan hanya memberikan periode yang lebih sempit, yang memungkinkan model untuk lebih mudah dalam melakukan prediksi.

Penggunaan 100 prediksi *rolling* penting karena memberikan sampel prediksi dari skenario yang berbeda untuk dibandingkan dengan rentang prediksi yang sama. Tanpa ini, perbedaan *sequence input* dan panjang prediksi dapat dimulai pada *timestamp* yang berbeda. Ini membuat perbandingan skenario tidak adil karena dievaluasi pada bagian pasar yang berbeda. Dengan memberikan sampel cadangan sebanyak 491 baris dan mensejajarkan tanggal peramalan pertamanya, perbandingan prediksi pada data sampel cadangan dapat dilakukan secara konsisten pada setiap skenarionya.

Proses rekonstruksi juga berperan penting pada tahap evaluasi. Karena model melakukan prediksi pada nilai log return dan bukan secara langsung pada nilai harga *close*. Untuk prediksi dengan panjang lebih dari 1, prediksi log return

diakumulasikan terlebih dahulu sebelum dirubah menjadi harga close. Hal ini berarti jika prediksi harga close langkah terakhir sangat bergantung pada total pergerakan dari langkah pertama hingga langkah ke n . Karena itu prediksi *multi-step* lebih sulit karena model harus mempertahankan prediksi log return yang akurat di setiap langkahnya.

Hasil juga menunjukkan jika prediksi model memiliki pergerakan yang lebih halus (*smooth*) di beberapa titik pada pergerakan harga close. Hal ini umum terjadi pada peramalan di data *time-series* berbasis regresi karena model cenderung memprediksi nilai close yang mendekati rata-rata (*central tendency*) dari distribusi data pelatihan. Ketika pasar aktual bergerak secara volatil, model tidak dapat mengikuti pergerakan yang ekstrim. Ini dapat dilihat pada contoh rekonstruksi, di mana harga close aktual pada prediksi pertama bergerak lebih kuat dibandingkan prediksi. Hasil tersebut menyebabkan error absolut yang lebih besar pada langkah tersebut.

Seperti yang dikemukakan pada tujuan penelitian, model berhasil diimplementasikan dengan *log return* sebagai target peramalan. Lalu *output* model direkonstruksi menjadi harga *close* agar dapat diinterpretasikan pada skala asli harga EUR/USD. Lalu model juga berhasil dievaluasi dengan 12 skenario berbeda dengan 4 panjang *sequence input* dan 3 panjang prediksi yang berbeda. Hasil juga menunjukkan jika parameter panjang prediksi berperan lebih signifikan dibandingkan panjang *sequence input*, di mana panjang prediksi yang lebih pendek memberikan *error* yang lebih kecil dibanding panjang prediksi yang lebih panjang.

Lalu pada rumusan masalah yang telah dikemukakan, implementasi model informer dapat dilakukan dengan tahap melakukan transformasi *sliding window*, normalisasi fitur *input* dan variabel target, penambahan *temporal marker*, dan pelatihan model menggunakan arsitektur *encoder* dan *decoder*. Model menggunakan data historis OHLCV, log return, RSI, dan ATR sebagai fitur *input* dengan log return sebagai target prediksi model. Lalu pada rumusan masalah yang kedua, evaluasi model menunjukkan jika model berjalan terbaik pada panjang prediksi 1 jam ke depan, khususnya dengan panjang *sequence input* 384. Namun, pada prediksi 4 dan 8 jam ke depan, *sequence input* terbaik berubah-ubah. Karena itu dalam pengujian ini, panjang prediksi harus menjadi variabel dalam mempertimbangkan panjang *sequence input*. *Sequence input* yang panjang dapat

berguna di beberapa kasus, tetapi tidak selalu opsi terbaik untuk setiap panjang prediksi yang digunakan.

Secara keseluruhan, hasil pengujian menunjukkan jika model dapat melakukan peramalan pada EUR/USD dengan *sequence* yang panjang dengan nilai error yang rendah. Model mendapatkan performa terbaiknya pada skenario LR→Close S384 P1, namun panjang *sequence input* terbaik bervariasi pada panjang prediksi lainnya. Hasil juga membuktikan jika penggunaan panjang prediksi lebih dari 1 menyebabkan model kesulitan dalam melakukan prediksi. Karena itu model berdasarkan pengujian yang telah dilakukan, model lebih cocok dalam melakukan peramalan dengan data EUR/USD jangka pendek, khususnya dengan panjang prediksi 1 jam ke depan. Untuk panjang prediksi yang lebih panjang, dibutuhkan penambahan optimisasi atau peningkatan pada fiturnya agar mengurangi akumulasi error pada prediksi.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, rumusan masalah yang telah dikemukakan pada BAB I dapat dijawab melalui kesimpulan berikut:

1. Model dengan arsitektur informer untuk melakukan peramalan pada data *timeseries* EUR/USD dengan *sequence* yang panjang dapat diimplementasikan. Dataset yang digunakan berisikan OHLCV dengan interval waktu 1 jam. Setelah tahap *preprocessing* data berisikan sebanyak 95,544 baris dengan periode waktu 2010-04-07 00:00:00 hingga 2026-04-17 23:00:00. Setiap baris datanya berisikan 9 kolom yaitu Date, Open, High, Low, Close, Volume, RSI, ATR, dan Log Return.
2. Model diimplementasikan melakukan konversi data *timeseries* menjadi *supervised learning* menggunakan metode *sliding window*. Panjang *sequence input* yang diuji adalah 48, 96, 192, dan 384. Lalu panjang prediksi yang diuji adalah 1, 4, dan 8. Model dilatih dengan log return yang sudah dinormalisasi sebagai target prediksi, lalu hasil prediksi tersebut dikonversi kembali menjadi harga close dari EUR/USD dengan rekonstruksi harga.
3. Hasil penelitian menunjukkan jika model informer dapat menghasilkan error prediksi yang rendah pada setiap skenario yang diuji. Skenario terbaik dihasilkan pada skenario LR→Close S384 P1 dengan panjang prediksi 1 jam ke depan dan *sequence input* 384. Skenario tersebut mendapat nilai evaluasi terendah pada seluruh evaluasi metrik yang digunakan, dengan nilai MAPE 0.058907% pada data tes dan 0.058907% pada data sampel cadangan. Hasil tersebut menunjukkan *sequence input* terpanjang yang diuji menghasilkan performa terbaik pada prediksi 1 jam ke depan.
4. Panjang prediksi memiliki peran yang lebih kuat dan konsisten dibandingkan panjang *sequence input*. Nilai rata-rata nilai MAPE pada data tes meningkat dari 0.058964% pada panjang prediksi 1 jam menjadi 0.092738% pada panjang prediksi 4 jam, lalu meningkat menjadi 0.125926% pada panjang prediksi 8 jam. Pola tersebut terjadi juga pada pengujian data sampel cadangan dengan nilai rata-rata MAPE 0.044022% pada

panjang prediksi 1 jam, menjadi 0.065781% pada panjang prediksi 4 jam, lalu meningkat pada panjang prediksi 8 jam dengan nilai 0.098295%. Hasil tersebut menunjukkan jika model menghasilkan hasil terbaik dengan panjang prediksi yang pendek, dan penggunaan panjang prediksi yang panjang menyebabkan model kesulitan dalam melakukan prediksi.

5. Meningkatkan panjang *sequence input* tidak selalu meningkatkan performa model. Untuk panjang prediksi 1 jam ke depan, model terbaik didapatkan menggunakan panjang *sequence input* 384. Pada panjang prediksi 4 jam, pada data sampel cadangan panjang *sequence input* sebesar 96, sedangkan pada panjang prediksi 8 jam ke depan, panjang 192 menjadi yang terbaik. Karena itu, panjang *sequence input* yang optimal bergantung pada panjang prediksi yang digunakan. *Input sequence* yang panjang dapat memberikan konteks historis yang lebih luas, tetapi juga dapat memberikan pola lama dari pasar yang belum tentu relevan pada kondisi pasar yang di prediksi.

5.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan, berikut adalah beberapa saran untuk pengembangan penelitian selanjutnya:

1. Pada penelitian selanjutnya dapat melakukan perbandingan model Informer dengan model peramalan *timeseries* seperti LSTM, GRU, Transformer, AutoFormer, atau NSTransformer. Perbandingan ini dapat menentukan apakah model Informer dapat memberikan performa yang lebih baik dibandingkan arsitektur lainnya pada data EUR/USD dengan menggunakan pengaturan experiment yang sama.
2. Penelitian selanjutnya dapat melakukan penambahan indikator teknikal sebagai fitur *input*. Dapat ditambahkan indikator seperti Moving Average (MA), Exponential Moving Average (EMA), Moving Average Convergence Divergence (MACD), Stochastic Oscillator, atau indikator berbasis volatilitas lainnya. Penambahan fitur dapat membantu model menangkap *trend*, momentum, dan pola volatilitas lebih efektif.
3. Pada penelitian selanjutnya dapat melakukan evaluasi model dengan metrik lainnya. Penambahan tersebut dapat meliputi dengan menambahkan metrik Directional Accuracy, Hit Ratio, Profit Simulation, Sharpe Ratio, atau backtesting pada model.

Metrik tersebut dapat membantu melakukan evaluasi apakah hasil peramalan model berguna untuk *trading* atau pembantu pengambil keputusan sebelum membuka posisi pada pasar.

DAFTAR PUSTAKA

1. Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(1). <https://doi.org/10.1186/s40537-021-00444-8>
2. Ayitey Junior, M., Appiahene, P., Appiah, O., & Bombie, C. N. (2023). Forex market forecasting using machine learning: Systematic Literature Review and meta-analysis. *Journal of Big Data*, 10(1). <https://doi.org/10.1186/s40537-022-00676-2>
3. Chaboud, A., Rime, D., & Sushko, V. (2023). *The foreign exchange market*. www.bis.org
4. Chika-Obi, K. (2026). Predictive Analysis of Forex Market Trends Using Machine Learning Algorithms. *International Journal of Computer Science and Mathematical Theory*, 12(1). <https://doi.org/10.56201/ijcsmt.vol.12.no1.2026.pg19.33>
5. Cui, Y., Li, Z., & Zhang, P. (2023). *Informer Model with Season-Aware Block for Efficient Power Series Forecasting*. <https://doi.org/10.20944/preprints202312.1377.v1>
6. Fischer, T., Sterling, M., & Lessmann, S. (2024). Fx-spot predictions with state-of-the-art transformer and time embeddings. *Expert Systems with Applications*, 249. <https://doi.org/10.1016/j.eswa.2024.123538>
7. Ghahremani, S., & Nguyen, U. T. (2025). Prediction of foreign currency exchange rates using an attention-based long short-term memory network. *Machine Learning with Applications*, 20, 100648. <https://doi.org/10.1016/j.mlwa.2025.100648>
8. Hall, T., & Rasheed, K. (2025). A Survey of Machine Learning Methods for Time Series Prediction. In *Applied Sciences (Switzerland)* (Vol. 15, Number 11). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/app15115957>
9. Hassani, A., Javadi, M., & Naisipour, M. (2025). *The Time Series Informer Model for Stock Market Prediction*. <https://doi.org/10.20944/preprints202510.1842.v1>
10. Jazayeri, K. (2024). Predicting multi-horizon currency exchange rates using a stacked ensemble of random forest and SVR. *Intelligent Decision Technologies*, 18(1), 297–325. <https://doi.org/10.3233/IDT-230194>
11. Kong, Y., Wang, Z., Nie, Y., Zhou, T., Zohren, S., Liang, Y., Sun, P., & Wen, Q. (2025). *Unlocking the Power of LSTM for Long Term Time Series Forecasting*. <http://arxiv.org/abs/2408.10006>

12. Liu, B., Li, Z., Li, Z., & Chen, C. (2024). CL-Informer: Long time series prediction model based on continuous wavelet transform. *PLoS ONE*, 19(9 September). <https://doi.org/10.1371/journal.pone.0303990>
13. Lu, Y., Zhang, H., & Guo, Q. (2023). *Stock and market index prediction using Informer network*. <http://arxiv.org/abs/2305.14382>
14. Mach, B.-Ngoc., Nguyen, H.-Cuc., & Nguyen, T. Q. (2025). *Enhancing long-term forex market forecasting using transformer-based time series models: A comparative analysis with ensemble tree methods*. <https://doi.org/10.21203/rs.3.rs-7733769/v1>
15. Malla, S. R., Kayastha, S., Suwal, R., Bhandari, H. C., & Adhikari, R. (2026). *XGBoost Forecasting of NEPSE Index Log Returns with Walk Forward Validation*. <http://arxiv.org/abs/2601.08896>
16. Mienye, I. D., & Swart, T. G. (2024). A Comprehensive Review of Deep Learning: Architectures, Recent Advances, and Applications. *Information (Switzerland)*, 15(12). <https://doi.org/10.3390/info15120755>
17. Niu, Y. (2024). Forecasting logarithmic returns on bitcoin based on the Informer model. *ACM International Conference Proceeding Series*, 255–260. <https://doi.org/10.1145/3705618.3705662>
18. Peshkova. (2025). *Triennial Central Bank Survey OTC foreign exchange turnover in April 2025*. www.bis.org
19. Saadati, S., & Manthouri, M. (n.d.). *Forecasting Foreign Exchange Market Prices Using Technical Indicators with Deep Learning and Attention Mechanism*.
20. Sadiku, M. N. O., Ajayi, S. A., & Sadiku, J. O. (2025). Machine Learning: An Overview the Creative Commons Attribution License (CC BY 4.0). In *International Journal of Trend in Scientific Research and Development*. www.ijtsrd.com/papers/ijtsrd97693.pdf
21. Saghaei, A., Bagherian, M., & Shokoohi, F. (2025). Forecasting Forex EUR/USD Closing Prices Using a Dual-Input Deep Learning Model with Technical and Fundamental Indicators. *Mathematics*, 13(9). <https://doi.org/10.3390/math13091472>
22. Stefaniuk, F., & Ślepaczuk, R. (2025). *Informer in Algorithmic Investment Strategies on High Frequency Bitcoin Data*. <http://arxiv.org/abs/2503.18096>
23. Stempień, D., & Ślepaczuk, R. (2025). *Hybrid Models for Financial Forecasting: Combining Econometric, Machine Learning, and Deep Learning Models*. <http://arxiv.org/abs/2505.19617>
24. Sung, S. H., Kim, J. M., Park, B. K., & Kim, S. (2022). A Study on Cryptocurrency Log-Return Price Prediction Using Multivariate Time-Series Model. *Axioms*, 11(9). <https://doi.org/10.3390/axioms11090448>
25. Tepelyan, R. (2025). *Enhancing OHLC Data with Timing Features: A Machine Learning Evaluation*. <http://arxiv.org/abs/2509.16137>

26. Tian, C., Cheng, T., Peng, Z., Zuo, W., Tian, Y., Zhang, Q., Wang, F. Y., & Zhang, D. (2025). A survey on deep learning fundamentals. *Artificial Intelligence Review*, 58(12). <https://doi.org/10.1007/s10462-025-11368-7>
27. Wang, J., Wang, X., Li, J., & Wang, H. (2021). A Prediction Model of CNN-TLSTM for USD/CNY Exchange Rate Prediction. *IEEE Access*, 9, 73346–73354. <https://doi.org/10.1109/ACCESS.2021.3080459>
28. Wu, Y. (2023). Comparison between transformer, informer, autoformer and non-stationary transformer in financial market. *Applied and Computational Engineering*, 29(1), 68–78. <https://doi.org/10.54254/2755-2721/29/20230874>
29. Yıldırım, D. C., Toroslu, I. H., & Fiore, U. (2021). Forecasting directional movement of Forex data using LSTM with technical and macroeconomic indicators. *Financial Innovation*, 7(1). <https://doi.org/10.1186/s40854-020-00220-2>
30. Zhao, L., & Yan, W. Q. (2024). Prediction of Currency Exchange Rate Based on Transformers. *Journal of Risk and Financial Management*, 17(8). <https://doi.org/10.3390/jrfm17080332>
31. Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting*. www.aiai.org
32. Zitis, P. I., Potirakis, S. M., & Alexandridis, A. (2024). Forecasting Forex Market Volatility Using Deep Learning Models and Complexity Measures. *Journal of Risk and Financial Management*, 17(12). <https://doi.org/10.3390/jrfm17120557>

LAMPIRAN

Lampiran 1 *Source Code* Model Informer



Lampiran 2 *Source Code* Pemrosesan Dataset



Lampiran 3 *Dataset Mentah*



Lampiran 4 *Dataset Terproses*



Lampiran 5 Hasil Pengujian

